



TECHNICAL UNIVERSITY

OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

Real-Time Light Dosimetry for Heritage Digital Twins via Texture Space Ray Tracing

LICENSE THESIS

Graduate: **Sebastian Duică**

Supervisor: **Assoc. Prof. Victor Ioan Băcu**

2026



TECHNICAL UNIVERSITY

OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

DEAN,
Prof. dr. ing. Vlad MUREȘAN

HEAD OF DEPARTMENT,
Prof. dr. ing. Rodica POTOLEA

Graduate: **Sebastian Duică**

Real-Time Light Dosimetry for Heritage Digital Twins via Texture Space Ray Tracing

1. **Project proposal:** Development of ChronoLux, an interactive Digital Twin for real-time light dosimetry using Texture Space Ray Tracing in DirectX 12.
2. **Project contents:** Introduction, Project Objectives, Bibliographic Research, Analysis and Theoretical Foundation, Detailed Design and Implementation, Testing and Validation, User's Manual, Conclusions, Bibliography.
3. **Place of documentation:** Technical University of Cluj-Napoca, Computer Science Department
4. **Consultants:**
5. **Date of issue of the proposal:** October 1, 2025
6. **Date of delivery:** July 10, 2026

Graduate: _____

Supervisor: _____



TECHNICAL UNIVERSITY

OF CLUJ-NAPOCA, ROMANIA

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

**Declarație pe proprie răspundere privind
autenticitatea lucrării de licență**

Subsemnatul(a) _____, _____ legitimat(ă) cu

_____ seria _____ nr. _____

(conform copiei anexate prezentei declarații, copie semnată și certificată "conform cu originalul"), autorul lucrării

elaborată în vederea susținerii examenului de finalizare a studiilor la Facultatea de Automatică și Calculatoare, Programul de studiu _____ din cadrul Universității Tehnice din Cluj-Napoca, sesiunea _____ a anului universitar _____, declar pe proprie răspundere, că această lucrare este rezultatul propriei activități intelectuale, pe baza cercetărilor mele și pe baza informațiilor obținute din surse care au fost citate, în textul lucrării și în bibliografie.

Declar, că această lucrare nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor.

Declar, de asemenea, că această lucrare nu a mai fost prezentată în fața unei alte comisii de examen de licență.

În cazul constatării ulterioare a unor declarații false, voi suporta sancțiunile administrative, respectiv, *anularea examenului de licență*. Sunt de acord ca, pe tot parcursul vieții, în cazul în care este necesar și se va dori verificarea autenticității lucrării mele să fiu identificat și verificat în baza datelor declarate de mine și conform copiei documentului de identitate menționat mai sus.

Data

Nume, Prenume

Semnătura

Contents

Chapter 1	Introduction	1
1.1	Context and Motivation	1
1.2	Limitations of Existing Solutions	1
1.3	The Proposed Solution: ChronoLux	2
1.4	Thesis Structure	2
Chapter 2	Project Objectives	4
2.1	Technical and Architectural Objectives	4
2.2	Metrological and Validation Objectives	4
Chapter 3	Bibliographic Research	6
3.1	Preventive Conservation and Empirical Dosimetry	6
3.2	The Metrology of Global Illumination	7
3.2.1	Rasterization vs. Ray Tracing in Scientific Simulation	7
3.2.2	The Rendering Equation and Monte Carlo Integration	7
3.2.3	Physical Material Simulation and Energy Conservation	8
3.2.4	Limitations of Offline Rendering	9
3.3	Hardware Acceleration and Heritage Digital Twins	9
3.3.1	Bounding Volume Hierarchies and Ray Traversal	9
3.3.2	DirectX Raytracing (DXR)	9
3.3.3	Variance Reduction and Texture Space Rendering	10
3.3.4	Solar Telemetry and Ephemeris Models	10
3.3.5	The Perez All-Weather Sky Model	11
Chapter 4	Analysis and Theoretical Foundation	12
4.1	High-Level Theoretical Pipeline	12
4.1.1	The Initialization Domain	12
4.1.2	The Radiometric Integration Domain	13
4.1.3	The Temporal Accumulation Domain	13
4.2	The Texture Space Ray Tracing Paradigm	13
4.2.1	The Mathematical Flaws of Screen Space Metrology	13
4.2.2	Texture Space Mapping	14
4.3	Dosimetric Path Tracing Algorithm	14
4.3.1	Stochastic Hemisphere Sampling and Energy Attenuation	14
4.3.2	The Physics of Refractive Transmission	15
4.3.3	Next Event Estimation (NEE)	15
4.3.4	Miss Evaluation and Perez Integration	16
4.4	Mathematical Dose Accumulation	16
4.4.1	Temporal Integration Logic	16
4.4.2	Floating-Point Precision and Heatmap Generation	17
4.5	Physical Light Attenuation in Atmosphere	17
4.5.1	Rayleigh and Mie Scattering	17
4.5.2	The Need for Anisotropic Sky Models	18
4.5.3	The Perez Scattering Indicatrix Formulation	18
4.6	The Theory of Solar Ephemeris	18

4.6.1	Astronomical Time and Julian Epoch	19
4.6.2	The Equation of Time and the Analemma	19
4.6.3	Solar Zenith and Air Mass Attenuation	19
4.7	The Rendering Equation and Thermodynamic Conservation	21
4.7.1	The Integral Formulation	21
4.7.2	Helmholtz Reciprocity and Energy Conservation	21
4.8	Theoretical Limitations of Photochemical Degradation	22
4.8.1	Photon Energy and Molecular Disruption	22
4.8.2	The Law of Reciprocity in Photochemical Damage	23
4.9	Conclusion	23
Chapter 5	Detailed Design and Implementation	24
5.1	Software Architecture Overview	24
5.2	Core Simulation Engine (C# CPU Orchestration)	24
5.2.1	The Chronological Engine and Time Stepping	26
5.2.2	Implementation of Mathematical Astrometry	26
5.3	Scene Management and Hardware Interfaces	27
5.3.1	Asynchronous Runtime Mesh Parsing	27
5.3.2	Dynamic Material Serialization and Hierarchy Keying	27
5.3.3	Spatial Normalization	28
5.3.4	Deterministic BVH Building	28
5.4	Hardware-Accelerated Path Tracing (HLSL / DXR)	29
5.4.1	Memory Management and DXR Acceleration Structures	29
5.4.2	Texture Space Dispatch Paradigm	29
5.4.3	UV Space Baking Matrix and Projection Anomaly	31
5.4.4	Pseudo-random Sequence Generation and Hashing	31
5.4.5	Direct Sunlight and Stochastic Multi-Sample Paths	32
5.5	Advanced Mathematical Models and Optimization	32
5.5.1	Implementation of Perez Sky Model in HLSL	32
5.5.2	Hardware Traversal Mechanics (DXR and Face Culling)	33
5.6	Asynchronous Data Telemetry and UI Integration	33
5.6.1	Automated Sensor Placement	33
5.6.2	Metrological Analytics Pipeline	33
5.6.3	User Interface Toolkit and JSON Serialization Limitations	35
5.6.4	Interactive Object Inspection and Navigation	36
5.6.5	Heatmap Visualization Pipeline	36
5.7	Deep Engine Optimizations and Memory Integrity	37
5.7.1	The Parallel Determinism Problem	37
5.7.2	Ray-Origin Epsilon Offsetting and Floating Point Precision Limitations	37
5.7.3	Helmholtz Reciprocity and Energy Conservation Enforcement	38
5.7.4	Thread Synchronization and CPU-GPU Data Dependencies	38
5.7.5	VRAM Memory Integrity and Cleaning Protocol	38
5.7.6	Data Export Payload Calculation and Mathematical Variance	39

Chapter 6	Testing and Validation	40
6.1	Introduction to the Metrological Validation Framework	40
6.2	Experimental Setup and Simulation Parameters	40
6.2.1	The Virtual Environment and Target Artifact	40
6.2.2	Meteorological and Astrometric Configuration	40
6.2.3	Atmospheric Modeling via the Perez Model	41
6.2.4	Hardware and Execution Profiling	41
6.3	Data Telemetry and the Simulation Output	42
6.3.1	Downsampled Dosemap and GPU Readback	42
6.3.2	The Telemetry CSV Format	42
6.4	Overview of Conservation Scenarios	43
6.5	Scenario A: The Baseline Exposure	43
6.5.1	Configuration and Untreated Conditions	43
6.5.2	Empirical Analysis of the Catastrophic Dose	43
6.5.3	Diurnal Fluctuation and Hourly Stress	43
6.6	Scenario B: Material Intervention	45
6.6.1	Simulating Physical Filtration	45
6.6.2	The Physics of Ray Attenuation	45
6.6.3	Results of the Material Intervention	46
6.7	Scenario C: Positional Optimization and Geometric Occlusion	46
6.7.1	The Role of Architectural Shadows	46
6.7.2	Phenomenological Analysis: Window vs Corner Geometry	46
6.8	Scenario D: Combined Optimal Conservation	47
6.8.1	Synergistic Interventions	47
6.9	Scientific Validation: Variance and Convergence	48
6.9.1	The Necessity of Mathematical Integrity	48
6.9.2	Spatial Dose Variance Tracking	48
6.9.3	Analytical Percentage Error and Convergence	49
6.9.4	Asynchronous Telemetry Pipeline Integrity	50
6.10	Conclusion of the Empirical Validation	50
Chapter 7	User's Manual	51
7.1	Introduction	51
7.2	Installation and System Requirements	51
7.2.1	Hardware Prerequisites	51
7.2.2	Software Installation and Artifact Import	51
7.3	The Project Launcher Dashboard	52
7.4	Viewport Navigation and Interaction	52
7.5	The Laboratory HUD and Telemetry	53
7.6	Material Configuration and Catalog	53
7.7	Simulation Configuration and Execution	55
7.8	Heatmap Visualization and Data Export	55
7.8.1	Viewport Heatmap	55
7.8.2	Headless Telemetry Export	55
Chapter 8	Conclusions	57
	Bibliography	59

Chapter 9	Appendix	61
9.1	Core HLSL Path Tracing Implementation	61
9.2	C# Orchestrator Implementations	66
9.3	Mathematical Radiometry and Ephemeris Models	68
9.4	Additional C# Application Logic	70
9.5	Python Data Processing Pipeline	86
9.6	Published Paper	91

Chapter 1. Introduction

1.1. Context and Motivation

The concept of cultural heritage conservation is one that is highly complex, being highly scientific and involving the balancing act between access to the public and the protection of artifacts. In museums, it is common practice to face a situation where there is a need to display old paintings, which could be as old as 16th century oil paintings, delicate pieces of cloth and even water colors without damaging them. The most ubiquitous source of degradation of such artifacts, however, is photochemical degradation caused by the presence of light.

Light Dosimetry is the metrological way of quantifying the radiative energy, measured in Lux Hours, acting upon an artifact within a specific period. Given that this damage is strictly cumulative and irreversible, it becomes imperative that preventive conservation measures be taken. For this purpose, guidelines set forth by international bodies like the International Commission on Illumination (CIE) are used to govern preventive conservation of artifacts. For highly susceptible items, the recommendation made by CIE is that the maximum limit of energy to which the artifact may be exposed should not exceed 50,000 Lux Hours annually.

1.2. Limitations of Existing Solutions

Up to now, the management of the stress induced by the light had been relying on two separate paradigms. However, both have their respective limitations when applied to today's workflows in the conservation of valuable heritage.

According to the first approach, physical light measurement can be carried out by the means of IoT sensor networks and photometers. While these devices indeed provide the essential ground truth data, they cannot help predict a threat before its occurrence; therefore, there is always going to be some delay, which makes these sensors unable to provide any proactive measure prior to a new exhibition. Besides, while measuring irradiance at a single point in space, it would take a huge amount of sensors deployed at high density in order to fully cover all possible angles of light incidence in a large object. Not only it would be expensive and unattractive, but mounting these devices on fragile pieces would simply cause irreversible damage. Finally, as every other device with a physical sensor inside, it suffers from calibration error, which needs to be compensated for by regular maintenance.

According to the second one, the issue of predicting is approached by using offline rendering engines like Radiance or Precomputed Radiance Transfer (PRT) lightmaps, which calculate the lighting distribution mathematically and in an extremely reliable way. Yet their inability to exploit hardware acceleration capabilities and being completely CPU-bound renders them impractical for real-time applications due to their poor

performance, which results in hours’ worth of calculation just for a single, one-year exposure simulation done with hourly precision (which is required in order to achieve proper global illumination).

That is precisely why this offline approach does not allow any interactive exploratory process to be carried out, meaning that conservators can neither explore the possibility of introducing some interventions into the environment—like altering transmittance of window glass, changing room albedo or moving objects around—nor try some optimal combination thereof.

1.3. The Proposed Solution: ChronoLux

For mitigating the issue of the interactivity and fidelity gap, this research will introduce ChronoLux - an interactive Digital Twin platform designed specifically for near-real-time dosimetry. Making use of modern GPU acceleration capabilities through DirectX 12 Raytracing (DXR), ChronoLux will transform passive artifact three-dimensional representations into interactive, predictive, and accurate simulations.

As opposed to camera-oriented screen-space rendering through camera-to-image ray-casting performed by current generation interactive graphics platforms, ChronoLux introduces a novel architectural approach - the Texture Space Ray Tracing. Regular renderers conduct frustum culling to exclude rays that are cast beyond the view range of the camera. Such an approach is insufficient when simulating light interaction for scientific dosimetry since measurements need to be conducted persistently throughout the entire geometrical surface of an object. Instead, using the artifact’s UV coordinates, ChronoLux will cast rays from those points thus baking any cumulative exposure permanently into the object’s surface.

In order to guarantee accuracy and metrological precision, the simulation will run under the Perez All-Weather Sky Model that will provide exact positioning of sunlight along with the skylight irradiance. The core of the algorithm will use the stochastic Next Event Estimation (NEE) Monte Carlo path tracer that will allow sampling separate components of illumination such as direct solar beams and diffuse scatterings independently. Overall, this approach provides curators with a non-destructive sandbox environment that enables instant dosimetric measurements without any drop in performance at approximately 60 fps rate.

1.4. Thesis Structure

Subsequently, the body of this paper is arranged in such a way that all the necessary information regarding ChronoLux will be delivered, from theory to experimental confirmation.

In Chapter 2, the main goals of this study will be described, namely, both interactive goals and metrological ones. Chapter 3 represents an extended review of the bibliography and describes recent achievements in the areas of physical dosimetry, offline ray tracing, and hardware accelerated global illumination. Chapter 4 contains the analysis of the theory of light transport, including the explanation of Lambert’s Cosine Law, and Monte Carlo integrals. Chapter 5 describes the process of creation of the Digital Twin and focuses on the Texture Space Ray Tracing algorithm, as well as Next Event Estimation kernel. In Chapter 6, the experimental validation and testing of the proposed

framework is described in detail, including different conservation case studies and proving the mathematical consistency of the Monte Carlo convergence. In addition, Chapter 7 presents an elaborate user guide to using the software, and in Chapter 8 - conclusions and future perspectives are stated.

Chapter 2. Project Objectives

The main focus of this thesis is to create a hardware-accelerated Digital Twin, called *ChronoLux*, which will provide an interactive and physically-based lighting simulation for the conservation of cultural heritage. This solution is designed to fill the important missing link between computationally-heavy offline rendering systems (such as Radiance), which can be proven mathematically, and real-time IoT sensing devices.

In order to achieve this high-level goal and provide the necessary combination of interactivity and scientific accuracy, the development process is carried out based on certain technical and metrological goals. The high-level architecture related to these goals is outlined in Figure 2.1.

2.1. Technical and Architectural Objectives

- 01. Texture Space Ray Tracing Integration:** To address the drawbacks associated with traditional screen space algorithms, such as frustum culling that results in critical out-of-screen lighting information being removed from calculations, a special compute shader for DirectX 12 Raytracing (DXR) would have to be used to launch rays directly from the unwrapped UV texture map of 3D models in order to permanently bake the cumulative radiometric energy into all of its geometric surface regardless of the location of the observer.
- 02. Physical Environmental Modeling:** In order to combine the Perez All-Weather Sky Model with a special Next Event Estimation (NEE) compute algorithm to create a scientifically rigorous method to decouple the direct beams of sunlight from the diffuse ambient lighting, thus generating physically correct global illumination without loss of real-time processing ability.
- 03. Asynchronous Compute Architecture:** For a proper separation of concerns between the user interface layer and computationally demanding GPU path tracing engine, the program should allow users to modify physical properties (transmission coefficients of window glazing or wall reflectance, for instance) via an interactive UXML/USS user interface without breaking the continuity of the 60 FPS viewport interaction.

2.2. Metrological and Validation Objectives

- 04. Headless Statistical Telemetry:** In this study, an automated approach has been developed for virtual point-probe Lux sensors which operate purely in the background. The sensors are created for the purpose of computing spatial variance (σ^2), maximum exposure dose, and standard Monte Carlo convergence error boundaries while not interfering with the primary rendering process.

O5. Empirical Scenario Validation: Explicit validation of this approach is achieved by running simulations of an entire year of meteorology (i.e., 8,760 hours of continual processing) for various preventive conservation scenarios. It is essential for establishing the efficacy of dynamic environmental control actions like the application of UV-absorbent windows or optimizing geometric occlusion of susceptible artworks.



Figure 2.1: High-Level Concept Architecture of the ChronoLux framework, illustrating the pipeline from raw heritage asset ingestion to real-time dosimetric data export.

The subsequent chapters of this thesis will methodically detail the design, theoretical foundation, programmatic implementation, and scientific validation of these stated objectives.

Chapter 3. Bibliographic Research

The creation of a hardware-accelerated Digital Twin for online light dosimetry is the result of the intersection of several complex scientific fields. The following section will provide an in-depth analysis of the literature related to preventive conservation metrology, mathematics of global illumination, and modern graphics processing technology. By analyzing the various approaches used today, the theoretical basis of this thesis will be defined, together with the limitations of current methods to be overcome by *ChronoLux*.

3.1. Preventive Conservation and Empirical Dosimetry

The need for light dosimetry is necessitated by the photochemically reactive nature of heritage objects. According to Michalski [1], both visible light and ultraviolet radiation are responsible for causing permanent molecular decay in natural and synthetic materials. In terms of photon energy, there is an inverse relationship between wavelength and energy level. The quantum energy of ultraviolet light, which occurs at wavelengths between 10 and 400 nm, is enough to break covalent bonds between pigment molecules, binding agents, and structural fibers.

Since light-induced deterioration is entirely accumulative, healing or reversing any damage is impossible simply by putting the artwork in darkness because the bonds have already been broken at a molecular level. To mitigate such problems effectively, the CIE set rigorous conservation limits and classified materials based on their varying degrees of vulnerability to light [2].

- **Insensitive Materials:** Artifacts are sensitive to exposure to light to differing extents. Materials like metals, stone, and ceramics are not vulnerable to photodegradation and, hence, have no special lighting requirements.
- **Low Sensitivity Materials:** Oil and tempera paintings, frescoes, and wood used in structures are capable of tolerating moderate exposure, up to 600,000 Lux-Hours per year.
- **High Sensitivity Materials:** Water colors, pastels, tapestries, and old papers, on the other hand, are extremely delicate and can only tolerate 50,000 Lux-Hours exposure in a year.
- **Extreme Sensitivity Materials:** Silk, certain unstable dyes, and early photos must be protected against all dangers posed by excess light, which is usually restricted to 15,000 Lux-Hours annually.

Going beyond these levels will hasten the process of entropy of artifacts, making it mandatory to establish accurate forecasting processes.

Traditionally, the approach that has been used extensively for evaluating such exposure is the measurement of incident light intensity. According to Silva et al. [3], the use of affordable data loggers to measure the microclimate in these locations is emphasized. In addition to this, Chianese et al. [4] describe the concept of “Internet of Cultural

Things” by suggesting the presence of dense IoT-based sensors within museums.

Discussion: While empirical sensor networks provide irrefutable real-world data, there exist several operational constraints associated with such systems that significantly limit their potential for use in modern conservation practices. First, it should be noted that these devices are purely reactive, with a physical sensor sending a signal when light interaction occurs, thus not enabling any preventive action before an exhibit is opened to light exposure. Second, the accurate simulation of the spatial distribution of irradiance at a huge three-dimensional surface would require installing an unrealistic amount of physical sensors on the subject. For instance, using thousands of sensors to cover a Renaissance sculpture would be both costly and visually unappealing. It could also be dangerous due to the fragility of such objects. As a result, to shift from reaction to prevention and simulation, mathematical models are required.

3.2. The Metrology of Global Illumination

3.2.1. Rasterization vs. Ray Tracing in Scientific Simulation

The visualization of virtual worlds in computer graphics has traditionally been implemented using rasterization algorithms. Rasterization maps 3D polygons to the 2D surface of the screen, computing their visibility through depth buffering. While extremely fast and highly suited for interactive purposes like games, rasterization is essentially incapable of scientific dosimetry. As the algorithm only takes into account direct visibility of objects (frustum culling), it heavily relies on ad hoc methods to create indirect lighting, including Screen Space Ambient Occlusion (SSAO), shadow mapping, and baked light probes, introducing significant bias.

Ray tracing, on the other hand, explicitly traces the actual path taken by the photons as they emanate from the light source, reflect from the geometry based on its material properties, and refract if necessary. Traditionally used for producing photorealistic images, ray tracing is the de facto standard for creating visual effects (VFX) for films and, most recently, top-tier video games due to the ability to trace perfectly accurate mirror reflections, soft shadows, and transparent materials’ refractions. In both of the cases, however, the goal remains that of deceiving human eyes and presenting realism without paying too much attention to the physical aspects in order to reduce rendering time.

Conversely, in order to achieve scientifically accurate dosimetry, one would need to simulate purely physical behavior of light with no regard to any virtual cameras or human perception involved. For that purpose, simulations need to be performed using unbiased and purely mathematical ray tracing which will allow the traced photons to serve as purely radiometric data. This technique is mathematically provable and thus represents the only possible algorithm for the measurement of the actual irradiation and Lux-Hour accumulation in a virtual world.

3.2.2. The Rendering Equation and Monte Carlo Integration

The key model that forms the backbone of physically based ray tracing is known as the Rendering Equation and was first introduced by Kajiya [5]. It represents a balance between the amount of radiance leaving a surface point through all directions, and the

summation of two quantities – radiance coming from emitting objects and scattered radiance from the hemisphere around the surface point.

Due to the fact that solving this infinite-dimensional equation is impossible for any graphics engine, an approximate method called Monte Carlo integration must be used. It entails sampling a random subset of rays in order to estimate the outcome of the integral. In other words, the more samples used, the higher the precision of the estimate – a principle defined by the Law of Large Numbers.

However, uniformly sampling random rays proves to be rather wasteful because many of them will not interact with any source of illumination, hence producing a noisy picture with high variance. In order to overcome this issue, the importance sampling technique can be applied. This approach uses a probability density function (PDF) that defines where rays should be cast based on the expected light distribution (e.g., cosine-weighted importance sampling for Lambertian surfaces).

In case the environment is enclosed and bounded, stochastic termination methods, like Russian Roulette, need to be applied in order to avoid infinite bouncing of rays. Instead of limiting their path length with an artificially chosen number of bounces, Russian Roulette ends the path when encountering dark surfaces which absorb more light.

3.2.3. Physical Material Simulation and Energy Conservation

Fidelity of the illumination is based on the rigour of material definitions. When a photon hits a geometrical surface, the physics behind what happens next can be described by the surface's Bidirectional Reflectance Distribution Function (BRDF) or Bidirectional Transmittance Distribution Function (BTDF). The combined effect describes the distribution function for the way the irradiance is reflected, absorbed, and transmitted through the hemisphere defined by the surface normal.

Modern computer graphics often use elaborate PBR materials that can be described by the state-of-the-art microfacet models such as the GGX distribution. According to Pharr et al. [6], these microfacet equations are employed to simulate the fine details of surface roughness and metals to achieve photorealistic visuals. However, their complex nature comes at an enormous computational cost while making a relatively minor contribution to the estimation of radiometric energy density on flat architectural surfaces.

ChronoLux does not use complex visuals of modern PBR materials but focuses on thermodynamically rigorous calculations. In metrological terms, according to the First Law of Thermodynamics (Energy Conservation), the sum of Reflectance (R), Transmittance (T), and Absorptance (A) of any material must be equal to one ($R + T + A = 1$). Thus, the reflectance of any hypothetical material reflecting 60% and transmitting 30% of the incident irradiation, like a tinted window glass, must have 10% absorptance.

Correctly accounting for these factors is critical for preventive conservation purposes. Walls are normally Lambertian, which means the photon is equally distributed in all directions after interaction with the wall surface, affecting the level of indirect illumination in the room. By contrast, windows imply complex interactions of photons and materials including transmission and refraction. A correct estimation of the transmittance graph of the UV-blocking glass or reflectance factor of polished marble flooring is crucial as it determines how the Lux-Hour dosage will accumulate on the artifacts.

3.2.4. Limitations of Offline Rendering

The foremost example of these approaches in architecture and conservation is the Radiance program [7]. The Radiance approach combines backwards ray tracing together with irradiance caching to solve the Rendering Equation with extremely high metrological precision.

Discussion: In contrast to other approaches to rendering, Radiance allows for the creation of lighting distributions that are provably correct from a mathematical standpoint. For this reason, it has become the gold standard for offline dosimetry calculations. The reliance on CPU bound processing however leads to significant inefficiencies. It takes several hours for the calculation of a full year of exposures, thus ruling out any interactive examination of the potential conservation strategies.

One of the alternatives to this method, Precomputed Radiance Transfer (PRT) [8], aims at solving the problem of real-time global illumination. PRT is achieved through precomputation of light transport into spherical harmonics. However, its applicability suffers greatly when the environmental effects are not static. An adjustment of the object orientation or transmittance properties of window glazing requires recalculating the entire PRT lightmap.

3.3. Hardware Acceleration and Heritage Digital Twins

The creation of highly accurate dosimetry with the capability of real-time interaction requires architectural thinking to be updated. As stated by Jouan & Hallot [9] and Niccolucci et al. [10], the concept of "Heritage Digital Twin" goes beyond that of traditional 3D representation or, otherwise known as HBIM, and becomes a dynamic replication continuously fed with data from reality to make decisions about preventive conservation.

3.3.1. Bounding Volume Hierarchies and Ray Traversal

The introduction of the Rendering Equation in this context at 60 Hz renders a need for complex spatial acceleration structures. Finding an intersection of a given ray with several million raw geometric triangles mathematically becomes impossible to compute within seconds. For that reason, scenes have to be split into Bounding Volume Hierarchy (BVH).

BVH is a kind of tree-like structuring where all geometries are placed into Axis-Aligned Bounding Boxes (AABB). The root of such structure contains the whole scene and, consequently, all geometries are successively divided by nodes until leafs, containing actual triangles, are reached. When tracing rays, we are checking intersections with bounding boxes. In case a ray misses a parent AABB, the algorithm ignores all of those millions of triangles turning an $\mathcal{O}(n)$ problem into a logarithmic $\mathcal{O}(\log n)$ search.

Surface Area Heuristi (SAH) is utilized to construct these trees optimally such that it reduces the probability of any intersection between rays and bounding boxes by assessing the cost of node traversal and triangle intersection.

3.3.2. DirectX Raytracing (DXR)

Traditionally, BVH traversal and construction was carried out using software on CPUs. In the words of Burgess [11], this approach was turned upside down in terms

of the NVIDIA Turing GPU architecture that came equipped with hardware silicon RT Cores for accelerating BVH traversal and triangle intersection.

The DXR API provides this functionality to developers by offering two acceleration structures for managing static and dynamic objects respectively. The first of these is the Bottom Level Acceleration Structure (BLAS). The bottom level structure contains all the raw triangle information of each mesh. Although extremely optimized, its creation requires high computational effort due to the complexity involved in the process. In contrast, the second acceleration structure—the Top Level Acceleration Structure (TLAS)—contains light instance references to BLAS that allow transformations to be applied instantly without rebuilding.

DXR provides an updated shader pipeline that makes use of a wide array of frameworks like ChronoLux:

- **RayGeneration Shader:** The first shader program used in the pipeline, responsible for generating primary rays. Unlike the traditional method of launching rays via camera, this process launches rays in Texture Space.
- **ClosestHit Shader:** Triggered upon successful intersection between a ray and solid geometry. At this point, it evaluates properties like BRDF, physics albedo, and diffuse interreflection.
- **AnyHit Shader:** Executed before Closest Hit to evaluate opacity or perform alpha test. This shader is required in scientific dosimetry to accurately calculate physics transmissivity across refractive window glazing without stopping the shadow ray.
- **Miss Shader:** Executed once the ray travels past local geometry. When working on environmental simulations, Miss shaders evaluate global illumination techniques such as Perez All-Weather Sky Model [12] to accurately compute solar irradiance.

3.3.3. Variance Reduction and Texture Space Rendering

For optimal performance of Monte Carlo Integration, Veach and Guibas [13] provided the necessary mathematical foundations for incorporating sampling algorithms that enabled the utilization of Next Event Estimation (NEE). NEE greatly helps to reduce the problem of variance through explicit casting of a shadow ray to the main light source in each bounce. The technique ensures fast convergence, even when there is indirect lighting, since it disassociates sunlight rays from ambient ones.

Moreover, for isolating fast ray traversal from the boundaries of the viewport of a camera, Munkberg et al. [14] presented modern texture-space cache methods. Traditional screen space renderers throw away useful lighting information that falls out of the current frustum view. The new technique renders images in texture space by shading objects in their corresponding UV mappings regardless of camera occlusions.

3.3.4. Solar Telemetry and Ephemeris Models

A precise simulation, especially one involving an entire meteorological year consisting of 8,760 hours, demands the exact position of the Sun in the sky. The solar position determines the angle of incidence of radiation, thus the shadows cast on any objects, and the amount of transmitted light through windows.

This can be obtained using various mathematical models of solar position from the scientific computing libraries, known as Solar Ephemeris. For instance, there exists a library called Solar Position Algorithm (SPA) formulated by Reda and Andreas [15]. This library calculates the solar zenith and azimuth positions based on latitude and longitude

position of the heritage sites, together with the Julian day and local time at any given point.

Apart from precision, calculation of the solar position is vital for computational optimization. Path tracing is quite computational intensive. Calculation of solar zenith position prior to the execution of the path tracing process allows the simulation to know if the Sun is below the horizon line. In case when it is night time (time between sunset and sunrise), then the process of path-tracing for the specific hour is not necessary, therefore reducing computation time significantly.

3.3.5. The Perez All-Weather Sky Model

The correctness of the virtual environment dose estimation model depends on the accuracy of the external light source driving it. In traditional three-dimensional rendering, the simplest form of light direction is usually used to simulate sunlight and set the sky color to uniform color; however, the physical process of atmospheric light scattering has some significant complexity. The complexity is solved by the use of scientific metrology through precise atmospheric math calculations, namely, the Perez all-weather sky model [12].

The Perez model is the empirical mathematical model that calculates the relative luminance of any sky point with regard to real meteorology conditions. Opposed to other simple models describing only ideal conditions of absolutely sunny or cloudy weather, Perez uses two atmospheric parameters:

- **Clearness (ϵ):** An index used to describe the ratio between direct solar radiation and diffuse sky radiation, giving a measure of how much clouds there are and their thickness.
- **Brightness (Δ):** An index that measures atmospheric luminous opacity, dependent on aerosol concentration and humidity.

With all these parameters taken into account along with precise solar angle and azimuth values, the model creates a very precise continuous luminance distribution. Such an approach incorporates modeling of two important scattering effects within the atmosphere, namely, circumsolar brightness, which results from Mie scattering near the sun and horizon brightness which is a result of the luminance gradient formed due to Rayleigh scattering in denser atmospheric layers around the horizon. In terms of the ChronoLux system, once the path traced ray leaves the building using DXR Miss shader, its vector direction is used to obtain a physical spectral radiance value for this particular micro angle within the sky dome.

Conclusion: The approaches examined within the current chapter demonstrate that even though offline rendering tools such as Radiance give precise results, and IoT sensors provide actual information, these methods cannot predict real-time changes and thus require interaction. Through a combination of the precise calculations of Monte Carlo path tracing algorithm and NEE technique suggested by Veach [13] along with the Perez sky model [12] and powerful hardware acceleration obtained through use of DXR [11] and BVH traversal along with texture space ray tracing [14], the proposed approach will fill this gap in preventive conservation.

Chapter 4. Analysis and Theoretical Foundation

In this chapter, a clear definition of theoretical algorithms, mathematical abstractions, and overall logic behind the *ChronoLux* design is attempted. Following the literature review, it becomes apparent that current methods of simulation suffer from significant shortcomings that render them unsuitable for application within interactive heritage metrology. Subsequently, in the following section, the theoretical measures suggested to tackle the aforementioned problems are outlined, focusing purely on abstract data manipulation and physics of light transfer.

4.1. High-Level Theoretical Pipeline

The *ChronoLux* pipeline is an endless sequence of mathematical computations that turns abstract three-dimensional data into radiometric energy build-up. From an architectural perspective, the *ChronoLux* system can be broken down into three separate domains. These include the Initialization Domain, Radiometric Integration Domain, and Temporal Accumulation Domain. As described earlier in Figure 2.1, *ChronoLux* operates on a closed-loop basis, using time-discrete iterations for its operations.

4.1.1. The Initialization Domain

Before the execution of radiometry calculations, a physical definition of the virtual environment is required. This initialization starts from the import of geometrical mesh information. From the theoretical standpoint, each artifact consists of a discrete collection of interconnected triangles forming a manifold geometry. Each of these triangles is connected to physically realistic material behavior, described by the strictly conservative thermodynamic equation ($R + T + A = 1.0$) outlined in Chapter 3.

The geographical and temporal status of the simulation is established at the same time. According to the theoretical description, the coordinates of the geographic latitude (Φ), geographic longitude (λ), geographic timezone offset, and the initial Julian Day of the physical heritage location are necessary inputs into the astronomy ephemeris calculation algorithm. After the completion of the first stage, a Bounding Volume Hierarchy (BVH) with a statistical optimization applied to the spatial information, as well as the solar zenith angle (θ_z), and solar azimuth angle (γ_s) for the current temporal step (t) are derived. If the calculated solar zenith angle is less than zero ($\theta_z < 0$), then, due to the system's automatic culling mechanism, the radiometric calculation process does not have to be executed at all, moving to the next temporal iteration point.

4.1.2. The Radiometric Integration Domain

After both sets of parameters have been initialized, the system proceeds to the next stage, which is referred to as the Radiometric Integration Domain. This is where the primary mathematical algorithm for calculating irradiance takes place. Unlike in conventional rendering techniques, which use the projection of the view frustum onto the two-dimensional space, all computations take place in Texture Space, an idea that will be further explained in Section 4.2.

In this domain, a computational solution is sought for the Rendering Equation at all acceptable points of the artifact's surface. Rays are propagated stochastically through the hemisphere to calculate both direct sunlight and diffuse ambient light using the technique of Next Event Estimation (NEE). Directions of escaping rays are compared to the Perez All-Weather Sky Model, allowing the system to receive exact spectral values of radiance. At the end of computations, a two-dimensional grid of data that corresponds to instantaneous physical irradiance (in Lux) is obtained.

4.1.3. The Temporal Accumulation Domain

The last theoretical step comprises the constant temporal summation of the irradiance values at each moment of time. Photochemical decomposition is always an accumulative process, so the calculated radiometric information will be reused rather than thrown out after one-time processing. In particular, the instantaneous illuminance (Lux) value is multiplied by the temporal duration of the current simulation step (Δt) to derive an accumulative dosage in Lux-Hours. The accumulative dosages will be added up using a persistent floating point array. After that, the temporal counter ($t = t + \Delta t$) is advanced, and the procedure is iterated until the end of the chosen period of time (e.g., meteorological year).

4.2. The Texture Space Ray Tracing Paradigm

The first major theory in which *ChronoLux* deviates from regular computer graphics is that of using the Texture Space Ray Tracing method of computation. To illustrate why such a computational method is needed in science metrology, its advantages, as well as the mathematical limitations of conventional methods, will be demonstrated below.

4.2.1. The Mathematical Flaws of Screen Space Metrology

First, in a three-dimensional engine, there is always a virtual viewer defined by a set of parameters including a field of view, distance from the screen, and near/far planes. During rendering, a two-dimensional pixel grid is used. For each pixel, there is a ray traced from the camera's optical point through the pixel into three-dimensional space.

Such an approach is mathematically exact, but it is flawed when we need to accumulate some radiation on all geometric surfaces. First, the problem is related to occlusion and perspective projection. When tracing rays from the virtual viewer, it becomes clear that the system will trace those rays that actually intersect the polygons. Any surface turned away from the viewer will not be computed because it simply will not be covered by any rays emanating from the viewer. It is obvious that if a statue is turned around the wrong way, no radiation can be accumulated on its posterior surface; the same holds true for other surfaces.

4.2.2. Texture Space Mapping

To accomplish a full separation between the simulation and the artificial restrictions of camera placement, *ChronoLux* uses Texture Space Mapping. In 3D geometry, each vertex of the polygon corresponds to a point on the 2D UV plane through (u, v) coordinates in three-dimensional geometry, which is called the UV map.

The process of traversing the UV map in the *ChronoLux* theory does not involve traversing the pixels on the computer screen but involves traversing the discrete texels on the UV map. This allows for the recreation of geometry in world space by calculating P (position) and N (surface normal).

Let the UV map be a 2D matrix M of size $W \times H$. For every element $M_{i,j}$:

1. Read the pre-computed world-space position vector $P = (P_x, P_y, P_z)$ corresponding to the (u, v) coordinate.
2. Read the pre-computed world-space normal vector $N = (N_x, N_y, N_z)$.
3. Use P as the origin point for the dosimetric path tracer.
4. Use N to orient the sampling hemisphere.

This method ensures that each infinitesimally small patch of the surface of the object is evaluated simultaneously, without regard to the virtual camera orientation. Global consistency of the physical light transport throughout the entire structure of the heritage object is guaranteed.

4.3. Dosimetric Path Tracing Algorithm

For the sake of simplicity, the origin (P) and the direction (N) of the rays as established using the Texture Space technique are known; hence, it is necessary to solve the global illumination equation in order to find the exact amount of energy hitting the particular point. This task can be accomplished using a highly efficient Monte Carlo algorithm with Next Event Estimation (NEE).

4.3.1. Stochastic Hemisphere Sampling and Energy Attenuation

Firstly, to find the total irradiance on a surface, the sampling directions have to be found, which involves generating directions distributed evenly over the upper hemisphere centered at the point where the sampling takes place. Within the framework of the Monte Carlo algorithm, such distribution of samples can be represented using a stochastic directional vector ω_i .

To improve convergence rates and minimize variance, *ChronoLux* uses Cosine-Weighted Importance Sampling. Instead of uniformly sampling random vectors, the probability density function used here tilts towards the vertical axis that passes through the surface normal. This method mirrors the physics behind lambertian diffuse surfaces when light coming from the vicinity of the surface normal carries more energy (proportional to $\cos(\theta)$) than that coming in from a large angle θ .

As these stochastic rays traverse the digital scene, they interact with other surfaces. The core principle behind this interaction is energy absorption, meaning the photonic path will dissipate its energy after each encounter with another surface. Thus, an object placed in a room with dark mahogany walls (with high Absorptance, A) will reflect indirect rays with significantly less energy than those reflected from white plaster (high Reflectance, R).

This effect can be easily modelled mathematically through the use of a throughput multiplier. Starting with an initial value of 1.0, which means perfect energy transmission, throughput gets multiplied by Reflectance R at each encounter. As $R < 1.0$ for any physical surface (following the law of conservation of energy $R + T + A = 1.0$), the ray loses energy at an exponential rate after each reflection.

In order to prevent simulation from becoming infinitely long, because of an endless number of bounces in enclosed space, a finite bounce depth limit should be set. For example, typical indirect path lengths do not exceed 3 or 4 bounces since additional reflections carry too little residual energy to matter further.

4.3.2. The Physics of Refractive Transmission

A basic element of heritage metrology involves evaluating the impact of external light that passes through physical obstacles, especially the effects of window glazing. This requires realistic modeling of refraction for light.

When a path traced ray meets a boundary between two media (say, a pane of glass), the ray does not necessarily stop or bounce off. Rather, it changes direction based on the IORs of the two media (air-glass) according to Snell's Law:

$$\frac{\sin \theta_1}{\sin \theta_2} = \frac{n_2}{n_1} \quad (4.1)$$

In the above-mentioned framework, n_1 and n_2 stand for the refractive indices of the media from which the ray starts and where it reaches, respectively; while θ_1 and θ_2 stand for the angles of incidence and refraction, respectively, regarding the normal to the interface.

As far as the *ChronoLux* abstract model is concerned, the transmission vector is calculated via Snell's Law whenever the beam penetrates a transparent body, allowing the ray to proceed with its travel through the solid glass. At the same time, the throughput of the ray is adjusted with respect to the particular material's Transmittance (T) factor. If a conservator applies UV-blocking film with a Transmittance of 0.45, the ray throughput is immediately decreased by 55%, thus providing an adequate simulation of the real-world effect produced by the intervention.

4.3.3. Next Event Estimation (NEE)

In a pure Monte Carlo path tracer, all sampling is random, and the tracing algorithm can use only random reflection from surfaces. The chance of finding the sun's small angular diameter through random reflections inside a dark museum is practically nil; therefore, there is significant variance in the simulation, and one would need to calculate millions of samples to converge toward the correct answer, rendering the calculation unfeasible at least with respect to the 8,760-hour simulation.

To alleviate this problem, *ChronoLux* employs the technique known as Next Event Estimation (NEE).

At every single interaction point P across the entire bounce sequence, the algorithm performs the following logic:

1. **Direct Shadow Ray:** A deterministic vector ω_{sun} is generated with the direction towards the exact current location of the sun. "Shadow ray" tracing occurs along ω_{sun} . If this ray leaves the bounded region without intersection with any opaque objects, the direct irradiance is calculated using the Perez model and added to the total dose. In case there is an intersection with a transparent object (such as

window glass), the light intensity will be reduced by a factor of T which is obtained using Snell's law.

2. **Indirect Bounce Ray:** At the same time, a stochastic vector ω_i is generated using Cosine-Weighted Importance Sampling. Tracing is performed along this vector to calculate the indirect light caused by reflection from walls and floors.

Through its clear disentanglement of the deterministic solar irradiance and the random scattering of the ambient sky dome, NEE guarantees that the massive energy content of the sun is properly accounted for at every stage of the simulation, thus reducing the number of samples needed by many orders of magnitude and ensuring metrological stability. The total irradiance of the starting surface point is thus computed as the sum of all successful shadow rays cast from the initial surface point through multi-bounces.

4.3.4. Miss Evaluation and Perez Integration

The last piece of the integration domain puzzle is that of Miss evaluation. When the stochastic ray (be it the direct shadow ray or the indirect bounced ray) finally escapes the digital twin architecture, it will need to retrieve the radiance value from the physical world surrounding the building.

Contrary to returning a hardcoded color, the escaping ray direction, ω_{escape} , is mathematically inserted into the Perez All-Weather Sky Model function. This function calculates the elevation (θ_z) and azimuth (γ_s) of ω_{escape} against the current solar location (θ_z, γ_s) and uses the meteorological data of Clearness (ϵ) and Brightness (Δ) to calculate the physical luminance value of ω_{escape} . If ω_{escape} happens to be near the solar disk, the result will undergo an exponential increase to simulate the Mie circumsolar scattering effect. Otherwise, if ω_{escape} is close to the horizon, then the Perez output transitions smoothly to simulate Rayleigh scattering.

4.4. Mathematical Dose Accumulation

The key theoretical objective of the *ChronoLux* tool is not only creating a photographically accurate representation of light but rather finding the overall photochemical risk posed over a long period of time. It implies moving from considering an instantaneous value of illumination to a cumulative dosimetry one.

4.4.1. Temporal Integration Logic

Let $E_t(P)$ be the instantaneous physical irradiance (measured in Lux) estimated with the help of dosimetric path tracer on a particular surface point P in a particular moment of time t .

Light illumination in a physical museum is a continuous process, while a digital simulation has to discretize time into particular segments (Δt). The choice of Δt significantly affects not only the accuracy of the calculation but also its performance. A traditional meteorological simulation usually takes 1 hour for each Δt . It allows tracking a motion of the Sun and cloud cover change in 8,760 steps within a typical year.

Cumulative dose $D(P)$ delivered to a particular surface point P for the whole period of time T is theoretically equal to the following sum:

$$D(P) = \sum_{t=0}^T E_t(P) \cdot \Delta t \quad (4.2)$$

Where:

- $D(P)$ is the total accumulated exposure in Lux-Hours.
- $E_t(P)$ is the calculated irradiance in Lux at time step t .
- Δt is the duration of the time step in hours.

4.4.2. Floating-Point Precision and Heatmap Generation

In order to retain the accumulating radiation dosage $D(P)$, the program uses the previously discussed Texture Space M matrix. Standard bitmap file formats (such as the 8-bit PNG or JPEG) cannot be used to store dosimetric data, as they can only store values from the set of integers ranging from 0 to 255. An extremely sensitive piece of art may amass as much as 50,000 Lux-Hours during its lifetime, which will quickly surpass the capacity of the 8-bit variable.

Therefore, according to the theory, the system makes use of a 32-bit floating point data structure (RFloat). A 32-bit float can theoretically encode 3.4×10^{38} , providing practically infinite precision for unbounded accumulation.

As the simulation advances within the temporal dimension, the matrix elements continuously increase in value. The abstract mathematical information is then mapped to a heatmap for visual analytical representation of areas where photosensitivity risks are critical.

4.5. Physical Light Attenuation in Atmosphere

In order to realistically calculate incident radiation in *ChronoLux*, we should understand the theoretical basis of the physical interaction between the solar photons of extraterrestrial origin and Earth's atmosphere before reaching any heritage object. Prior to reaching heritage artifact, solar radiation penetrates Earth's atmosphere, which acts as a considerable and thick participation medium. This medium redistributes energy via scattering and absorption, thus changing properties of light reaching Earth's surface.

4.5.1. Rayleigh and Mie Scattering

Solar radiation interacting with the atmosphere experiences two types of scattering which are crucial for comprehension of dual nature of daylight (direct light and diffuse). First, the process of scattering known as **Rayleigh scattering** [16] takes place when photons collide with gases (like nitrogen or oxygen) contained in the atmosphere. These particles are substantially smaller than wavelengths of visible light. The Rayleigh scattering theory says that the intensity of scattering is inversely proportional to the fourth power of wavelength (proportional to λ^{-4}). Therefore, short wavelengths (blue and violet) experience much stronger scattering than long wavelengths (red and yellow). This is the reason why the sky looks blue. In dosimetric simulations, Rayleigh scattering defines the diffuse irradiance of the "blue sky" dome as a base and omnidirectional radiation source which is able to induce photochemical damage to artifacts even in corridors shielded from direct sunlight in the museums.

Second type of scattering called **Mie scattering** [17] takes place when photons interact with aerosols in the lower layers of atmosphere (dust, pollen, pollution, and water vapor droplets). Contrary to Rayleigh scattering, Mie scattering is largely independent on wavelength, which is the reason why clouds look white. It is important to note that Mie scattering is directional, scattering most light forward, in direction similar to the direction of the sun. As a result, Mie scattering produces a strong circumsolar corona – a bright region around the solar disk.

4.5.2. The Need for Anisotropic Sky Models

Many computer graphics applications, including video games, treat ambient sky-light as isotropic hemispherical illumination, treating sky as a simple blue flat color. Such treatment of sky is incorrect from thermodynamic and metrological perspective because of Mie and Rayleigh scattering processes. Real sky dome demonstrates obvious *anisotropy* (direction-dependence). The horizon region can be much brighter/darker than the zenith as a result of optical path through atmospheric layer (horizon brightening). Also, the circumsolar region is much brighter than other regions of the sky dome.

In order to make the diffuse scattering metrologically accurate in dosimetric application, *ChronoLux* is based on the diffuse scattering based on the **Perez All-Weather Sky Model** [12] (algorithmic implementation in Appendix 9.3). In contrast to simple color-based approach, Perez model is based on five coefficients controlling horizon brightening, luminance gradient, relative intensity of the circumsolar corona, its spatial width, and backscattered light. Calculating these coefficients depending on solar altitude each hour, *ChronoLux* ensures physically grounded and metrologically accurate ambient diffuse irradiance of virtual museum windows.

4.5.3. The Perez Scattering Indicatrix Formulation

The physical implementation of the Perez model relies on a scattering indicatrix function that determines the relative luminance of any point on the sky dome based on its zenith angle θ and its angular separation γ from the sun:

$$f(\theta, \gamma) = \left[1 + a \exp\left(\frac{b}{\cos \theta}\right) \right] \times [1 + c \exp(d\gamma) + e \cos^2 \gamma] \quad (4.3)$$

The model uses five empirically obtained static coefficients to shape the anisotropic properties of the sky dome. For a clear sky scenario with Turbidity $T = 2.0$, these thermodynamic parameters are typically defined as:

- $a = -0.1058$: Controls horizon brightening or darkening.
- $b = -0.0738$: Determines the luminance gradient close to the horizon.
- $c = 1.8200$: Controls the relative intensity of the circumsolar corona (forward Mie scattering).
- $d = -2.9744$: Determines the spatial width of the circumsolar region.
- $e = 0.1778$: Controls the backward-scattered component opposite to the sun.

4.6. The Theory of Solar Ephemeris

In order to be metrologically correct, the temporal domain of the simulation has to correspond to true celestial mechanics. *ChronoLux* cannot use standard civil time which

is artificial construction depending on political time zones, daylight saving and leap years. Nature doesn't know anything about it.

4.6.1. Astronomical Time and Julian Epoch

In order to track time in the simulation, avoiding artificial calendar interruptions, the simulation uses the theory of **Julian Day (JD) epoch** [18]. Julian Day system represents a continuous count of days and fractions thereof since noon Universal Time on January 1, 4713 BCE. For example, a civil date like May 24, 2026, at 14:30 is converted to UTC and then to the Julian timeline, resulting in single floating-point number (e.g., JD 2461185.104). This eliminates all complications related to Gregorian calendar. From Julian Day, the simulation obtains Julian centuries since J2000.0 epoch (T), continuous variable, which is the basis for calculating exact orbital position of Earth at any microsecond of the simulation. The complete logic flow orchestrating this astrometric math and integrating it with the texture space path tracer is mapped out in Figure 4.1.

4.6.2. The Equation of Time and the Analemma

The second problem between civil time and real time is caused by the fact that Earth moves in ellipse rather than in circle and the angle between equator and ecliptic is equal to 23.44° . According to Kepler's laws, planet moves faster at perihelion and slower at aphelion. Coupled with the tilt of equator relative to ecliptic, this means that a "true solar day" is not equal to 24 hours. A civil clock moves uniformly and therefore will drift relative to the Sun's position. Recording the Sun's position at exactly 12:00 civil time each day for a year from the same location results in drawing of the asymmetrical figure-eight in the sky—the *analemma*. To solve this problem, the **Equation of Time (EoT)** gives the difference between Mean Solar Time (civil time) and True Solar Time (physical solar position). Ignoring EoT will result in wrong shadow directions and, thus, will invalidate dosimetric exposure calculations on artifacts. Using the equation allows to precisely calculate the Hourly Angle (H), so the simulated solar rays will intersect heritage geometry correctly for any day of meteorological year.

4.6.3. Solar Zenith and Air Mass Attenuation

Once the celestial timeline is normalized against the Julian epoch, the precise geometrical Solar Zenith Angle θ_z and Azimuth ϕ can be defined relative to the local terrestrial latitude Φ and the solar declination δ . The solar declination is derived from the ecliptic longitude λ_{app} and the Earth's axial tilt ϵ :

$$\delta = \arcsin(\sin(\epsilon) \sin(\lambda_{app})) \quad (4.4)$$

$$\cos(\theta_z) = \sin(\Phi) \sin(\delta) + \cos(\Phi) \cos(\delta) \cos(H) \quad (4.5)$$

Where H is the calculated Hour Angle. Having established the astrometric vector, the clear-sky direct-beam irradiance E_{beam} must be calculated by accounting for the physical depth of the atmosphere the light must penetrate. This depth is described by the optical air mass AM , modeled accurately by the Kasten–Young formula [19]:

$$AM = \frac{1}{\sin(\alpha) + 0.50572(\alpha + 6.07995)^{-1.6364}} \quad (4.6)$$

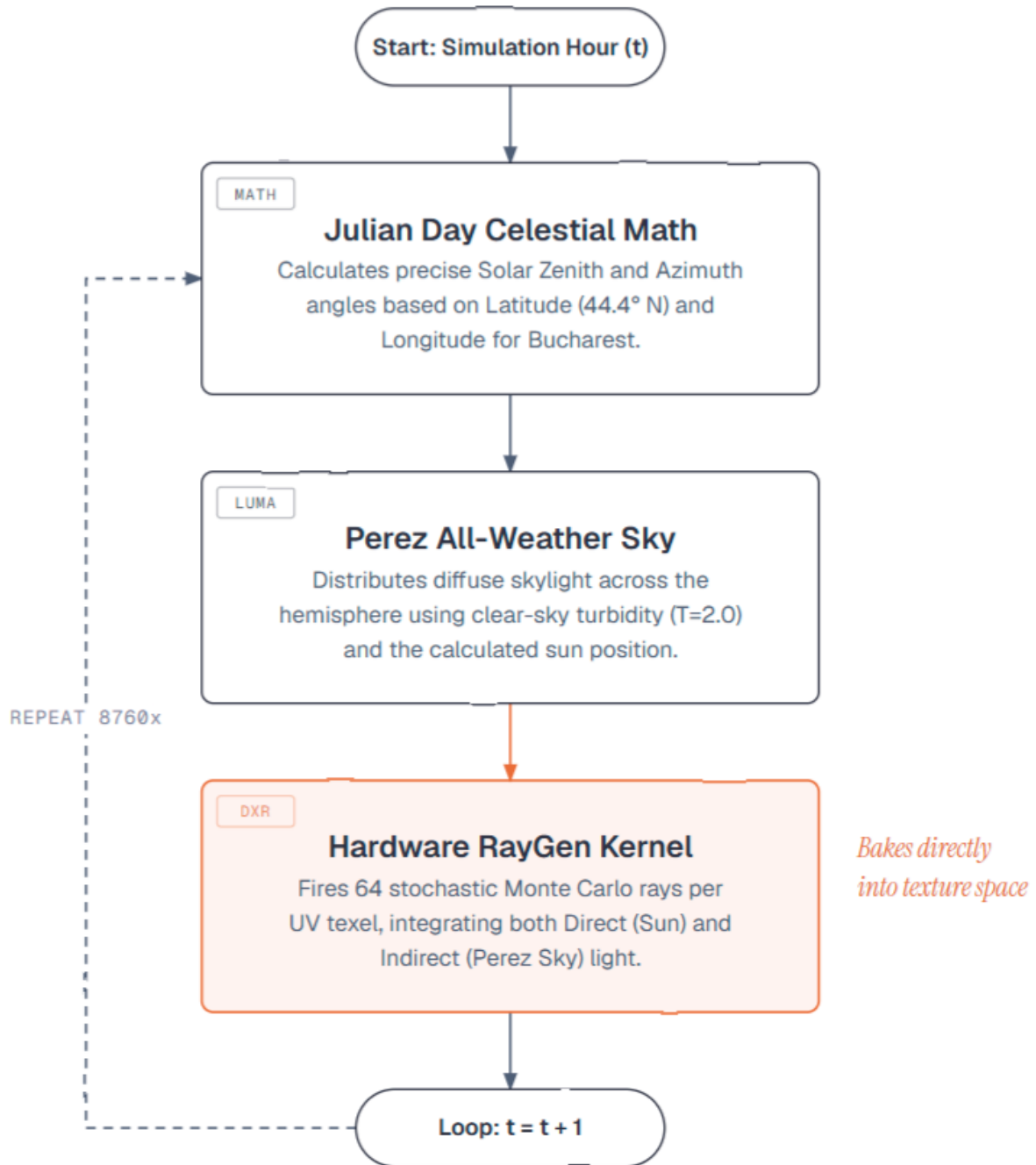


Figure 4.1: The astrometric logic flowchart describing the calculation of Julian Day parameters and integration into the ray generation kernel.

$$E_{beam} = 127500 \times 0.7^{AM^{0.678}} \quad [\text{Lux}] \quad (4.7)$$

Where α denotes the solar altitude angle ($90^\circ - \theta_z$) and 0.7 represents the baseline atmospheric transmittance factor for ideal clear-sky conditions.

4.7. The Rendering Equation and Thermodynamic Conservation

ChronoLux is based on the theory of light transport in physically based manner, as formulated for computer graphics in 1986 by James Kajiya [5]. In order to simulate light transport in a complex architectural space, the engine solves this continuous mathematical equation.

4.7.1. The Integral Formulation

The total radiant energy leaving a surface point P towards viewing direction ω_o is represented by the **Rendering Equation**:

$$L_o(P, \omega_o) = L_e(P, \omega_o) + \int_{\Omega} f_r(P, \omega_i, \omega_o) L_i(P, \omega_i) (\omega_i \cdot N) d\omega_i \quad (4.8)$$

For those who don't have background in advanced calculus and radiometry, the equation can be explained using the following notions:

- $L_o(P, \omega_o)$ (**Outgoing Light**): The quantity the equation calculates—the total light leaving point P in direction ω_o .
- $L_e(P, \omega_o)$ (**Emitted Light**): Light emitted within the object itself (glow). For typical heritage artifacts like marble, oil paintings or wooden furniture, this term is zero.
- $\int_{\Omega} d\omega_i$ (**The Hemisphere Integral**): At microscopic point of the surface, this integral goes over the whole half sphere Ω , summing light coming from all possible directions ω_i .
- $f_r(P, \omega_i, \omega_o)$ (**BRDF**): Bidirectional Reflectance Distribution Function, function describing the material's response to light (how much light is reflected and in which directions).
- $L_i(P, \omega_i)$ (**Incoming Light**): Light energy coming to P from direction ω_i , direct sunlight or light coming from surrounding surfaces.
- $(\omega_i \cdot N)$ (**Lambert's Cosine Law**): Geometrical attenuation factor. The ray hitting the surface perpendicularly forms concentrated light spot, while grazing incidence causes the same light to distribute over bigger area, making it less intensive. Dot product takes this into account.

Since the integral contains the summation of infinite number of incoming directions $d\omega_i$ over the hemisphere, the exact analytical solution of the equation is impossible. Therefore, numerical approximation method called **Monte Carlo technique** with Next Event Estimation (described in Section 4.3, algorithmic implementation in Appendix 9.1) is used to approximate the integral with thousands of sample rays.

4.7.2. Helmholtz Reciprocity and Energy Conservation

To ensure that the numerical solution to the Rendering Equation will be physically correct, the BRDF of the material (f_r) should adhere strictly to the principles of thermodynamics in particular, **Law of Conservation of Energy**. In many arbitrary video game engines, reflective materials could be adjusted to give off more light than they get

– resulting in visually striking yet metrologically incorrect situation when objects create an impossible amount of extra energy. On the contrary, *ChronoLux* follows strict mathematical limitations. Whenever a photon strikes a physical medium, the incoming energy splits into three distinct channels: **Reflectance** (R), **Transmittance** (T), and **Absorptance** (A). The thermodynamical limitation requires that the sum of all these channels equals to the incoming energy:

$$R + T + A = 1.0 \quad (4.9)$$

Thus, dark mahogany floor will have large A (Absorptance), small R (Reflectance) and $T = 0$ (Transmittance). At the same time, clean glass will have large T (Transmittance) with small R (Reflectance) and small A (Absorptance). Therefore, the simulation engine imposes limitation of $R + T \leq 1.0$. This guarantees the **Helmholtz reciprocity**, i.e. optical path is physically reversible, and underlying physics are valid independently from the direction of the light propagation. This limitation is the basis of the possible usage of **Russian Roulette** path termination technique – whenever the simulated ray strikes a dark carpet (large A), the engine uses the small value of $R + T$ as a probability threshold, making it more likely for a photon to be absorbed rather than continue traveling. Thus, this process guarantees convergence of stochastic simulation to physical dose without any energy feedback loops.

4.8. Theoretical Limitations of Photochemical Degradation

After we have formulated the physical transport of light energy within an architectural volume, it is time to discuss the purpose of energy tracking – the assessment of photochemical degradation risks. When light interacts with an object of cultural heritage, there is no mere abstraction of physical interaction, but the interaction between light and the molecular structure of the material.

4.8.1. Photon Energy and Molecular Disruption

In order to assess dosimetric risk, light should be viewed not only as a continuous wave but as a flux of discrete energy quanta, photons. According to the Planck–Einstein relation [20], the energy (E) of each photon is directly proportional to its frequency (ν) and inversely proportional to its wavelength (λ):

$$E = \frac{hc}{\lambda} \quad (4.10)$$

where:

- h is the Planck constant (6.626×10^{-34} J·s).
- c is the speed of light in a vacuum (3×10^8 m/s).
- λ is the photon’s wavelength.

This formula introduces an important concept in preventive conservation – short-wavelength photons carry disproportionately higher destructive energy. The visible part of the daylight spectrum ranges from 400 nm (violet) to 700 nm (red); however, unfiltered solar radiation contains considerable amounts of both Ultraviolet (10-400 nm) and Infrared radiation (700 nm – 1 mm). Ultraviolet photons, due to their extremely short wavelengths, carry enough energy to destroy covalent bonds in complex organic molecules, found in old oil paintings, medieval fabrics and ancient wood. Such micro-

scopic disruptions manifest themselves as macroscopic fading, yellowing, embrittlement, color desaturation. However, Infrared photons, while carrying less energy, can cause substantial thermal excitation of the molecules, leading to thermal distortion, cracking and desiccation of the material through local heating.

4.8.2. The Law of Reciprocity in Photochemical Damage

According to classical photochemical theory, the total amount of degradation in an organic material is governed by the **Law of Reciprocity** or Bunsen-Roscoe law [21]. The idea behind this law is that the total light dose absorbed in an object determines the degree of photochemical reaction, regardless of the exact speed of energy delivery. In other words, extremely high-intensity exposure over short periods of time would yield the same theoretical photochemical damage as lower intensity exposure over a long period of time if total energy dose is equal. For example, exposure to 1,000 Lux for one hour gives the same theoretical photochemical damage as exposure to 50 Lux for 20 hours, provided that total energy dose is 1,000 Lux-hours.

This basic principle becomes the foundation of the temporal integration approach described in Section 4.4. *ChronoLux* accumulates the instantaneous irradiance $E_t(P)$ over the entire 8,760-hour meteorological year, thus implementing the Bunsen-Roscoe reciprocity principle. As a result, conservators are able to define strict annual exposure limits (for example, 50,000 Lux-Hours for moderately sensitive objects like oil paintings, or 15,000 Lux-Hours for highly sensitive objects such as watercolors, paper and silk).

4.9. Conclusion

Thus, in conclusion, the theories described in the current chapter make *ChronoLux* a purely metrological simulation software. Abandoning the bias-laden approach of the Screen Space rendering in favor of the Texture Space ray tracing method guarantees complete geometrical coverage. Furthermore, Next Event Estimation, together with the Perez Sky Model algorithm, ensures exact computation of the light transport. Finally, accumulating irradiation in an arbitrarily precise floating point matrix allows for accurate simulation of photochemical degradation for meteorological years.

Chapter 5. Detailed Design and Implementation

In this chapter, a thorough explanation is provided regarding the software architecture, the accelerated algorithms running on hardware, and the rigorous mathematical systems used in order to create a ChronoLux digital twin. The practical realization of radiometric and conservation physics theories on high-performance desktop software requires a highly sophisticated coordination between CPU scheduling, memory transfers, and extensive GPU parallelism.

The historic inability to provide real-time interaction with the predictive workflow using the CIE and Michalski principles for preventive conservation can be attributed to the lack of interactive offline software, such as Radiance, that requires a lengthy amount of CPU computations in order to simulate only one temporal frame. ChronoLux solves this problem by creating a hybrid architectural solution, using the Unity 3D engine and the DirectX 12 graphics API in order to allow the usage of hardware-accelerated DirectX Raytracing (DXR). Each component of the computational pipeline from mathematical astrometry to GPU memory management and asynchronous telemetry extraction will be analyzed.

5.1. Software Architecture Overview

In order to run sufficient amount of ray queries for a yearly dosimetric simulation without causing problems with system operation, a strictly decoupled architecture was chosen. The software stack is divided into two major domains: the first domain contains the C# CPU side of the program, including the UI, file I/O, deterministic structural sorting and mathematical astrometry and the second one contains the HLSL compute shaders running the highly parallelized Monte Carlo integrations on the GPU.

s

Data flow is tightly controlled. The project parameters are serialized to JSON with the help of `ChronoProject` object (see Appendix 9.4, Listing 9.10) and parsed with the `ProjectManager` (see Appendix 9.4, Listing 9.11) that feeds the `LightDoseSimulator`. The `LightDoseSimulator` acts as an orchestrator controlling the time and dispatching ray queries to the DXR kernels.

5.2. Core Simulation Engine (C# CPU Orchestration)

The central logic should guarantee mathematical determinism, avoid hardware starvation, and respond to user interactions.

As one can see on the diagram above, the `LightDoseSimulator` depends on multiple important subsystems: `SunCalculator` (see Appendix 9.3, Listing 9.7) for mathematical astrometry, `IrradianceBaker` (see Appendix 9.4, Listing 9.12) for hardware dispatch and `RuntimeModelLoader` for asynchronous file I/O.

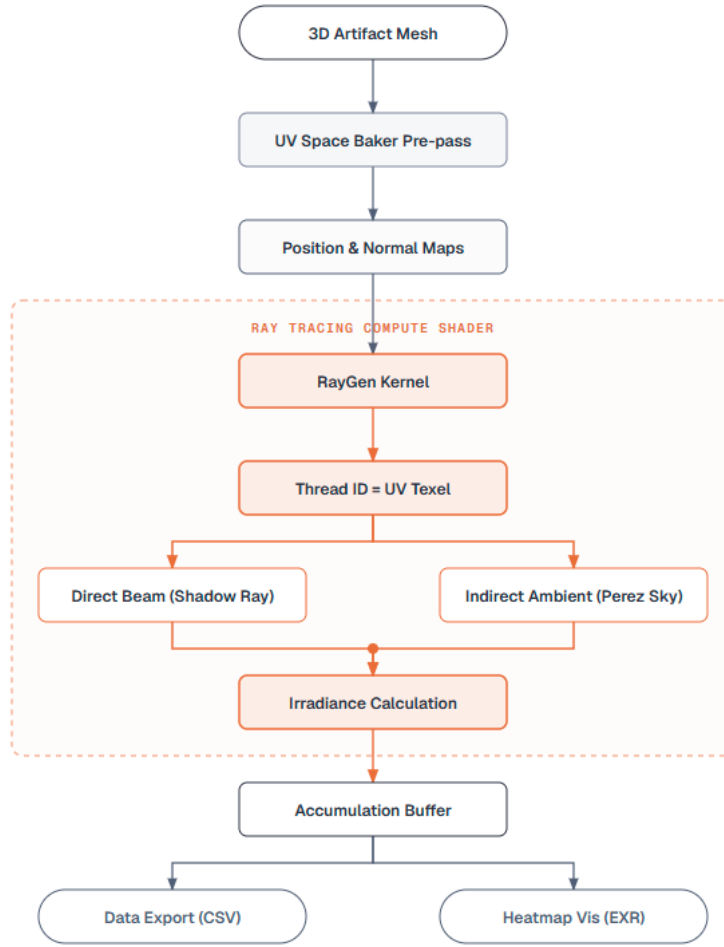


Figure 5.1: High-level simulation pipeline demonstrating the decoupling of CPU orchestration from GPU execution.

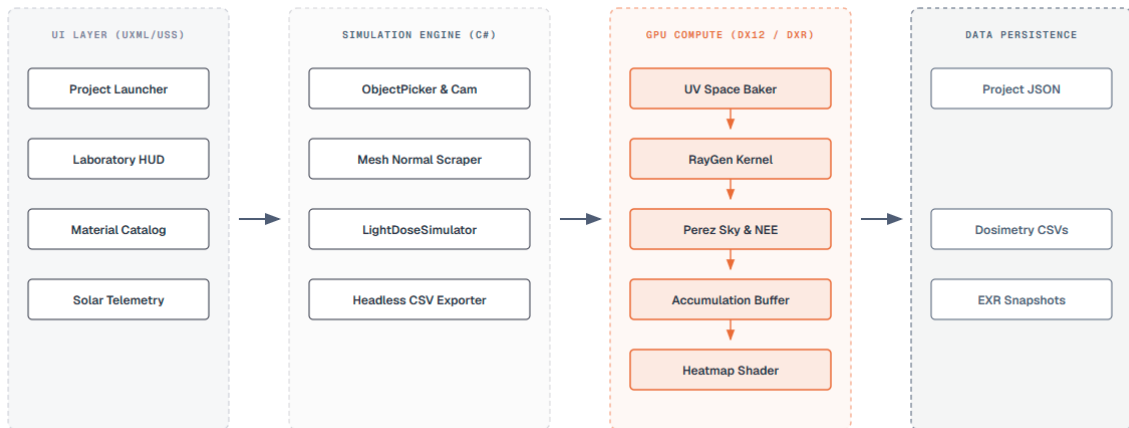


Figure 5.2: Overall Architectural System design of *ChronoLux*, detailing data paths between the Engine, the GUI, and the OS filesystem.

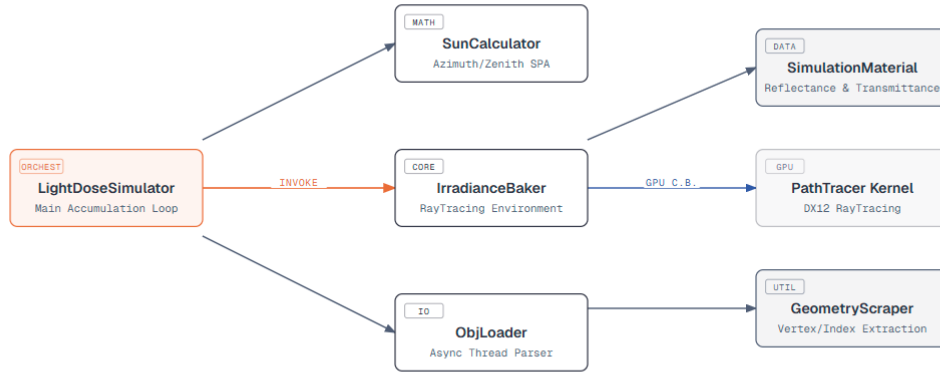


Figure 5.3: C# CPU Class Architecture and Data Flow orchestration within *ChronoLux*.

5.2.1. The Chronological Engine and Time Stepping

The primary job of the `LightDoseSimulator` is to increase the time by Δt and to calculate the precise solar altitude and azimuth for the Perez sky model [12]. The simulation should calculate one meteorological year without causing the Timeout Detection and Recovery (TDR) event, that might forcefully stop graphics drivers operation after the certain timeout. Such event occurs after a roughly two second timeout in the certain task and is handled with the help of the C# `IEnumerator` Coroutine called `RunSimulationInternal`. As detailed in Appendix 9.2 (Listing 9.5), the time stepping loop is fully dynamic and goes over the range from `startDay` till `endDay`.

For each day, the coroutine uses the `SunCalculator` to calculate the exact local sunrise and sunset boundaries based on user's defined latitude, longitude and UTC offset and thus, avoiding the unnecessary ray casting during night hours. In case of equal sunrise and sunset time (a rare case in polar nights), the current day is simply skipped.

Aggressive computational culling is used in the hourly step loop. The engine ignores all steps where both calculations of direct-beam lux and diffuse-sky lux are negligible (lower than 0.1 Lux). Such optimization allows us to save millions of useless Monte Carlo ray casts per simulated year.

During the active simulation period, the coroutine explicitly uses the `yield return null;` instruction in order to put itself to sleep until the next engine frame. Such approach allows to give a thread back to the Unity main loop thus maintaining responsive UI, proper animation of progress bar and satisfying TDR watchdog.

5.2.2. Implementation of Mathematical Astrometry

All mathematical dependencies involving the declination of the Sun, optical air mass, and clear-sky beam irradiance (described in Section 4.7) are dynamically calculated each hour of the simulation within the `SunCalculator.cs` component (see Listing 9.7).

While the master clock of the `LightDoseSimulator` moves forward, the Julian Day and then the Hour Angle are calculated using the double-precision floating-point format.

This approach is aimed at avoiding any possible temporal drift caused by single-precision floating point numbers due to cumulative calculations of Julian time stamps over many hours in case of the annual 8,760-hour simulation.

After calculating the solar altitude and azimuth, the `SunCalculator` calculates the current `BeamLux` and `DiffuseLux` values representing the real physical energy entering the virtual atmosphere. The two very dynamic variables are sent immediately to the DXR compute shader, where the absolute limit value of the energy flux falls onto the three-dimensional scene.

5.3. Scene Management and Hardware Interfaces

5.3.1. Asynchronous Runtime Mesh Parsing

ChronoLux is delivered as a compiled desktop metrology application and therefore cannot depend on pre-compiled Unity Scene assets. It should dynamically load arbitrary user provided `.obj` files at runtime from the local filesystem. Heritage artifacts, such as laser scanned church interior or high resolution photogrammetry models, could have millions of vertices. String-based parser working synchronously on the main Unity thread would make the application unresponsive and could cause the Operating System "Application Not Responding" failure. Therefore, runtime mesh parsing is implemented asynchronously.

As seen in Appendix 9.2 (Listing 9.6), the `RuntimeModelLoader.cs` resolves this memory bottleneck using a heavily parallelized chunk parser. File reading and string parsing of vertices, normals and UV coordinates are executed entirely by a separate `Task.Run()` background thread running on various logical cores of the CPU. As the parsing takes place in the background, the main thread constantly checks the condition `while (!parseTask.IsCompleted)`, calling `yield return null;` and updating the `ProgressBar` UI via `UpdateProgress()`. Once the enormous memory arrays are completely filled, control is passed back to the Unity main thread, which performs the final `Mesh.SetVertices()` VRAM upload and then creates the `GameObjects`. Such architecture ensures full UI responsiveness during the most data-intense parts of the process.

5.3.2. Dynamic Material Serialization and Hierarchy Keying

Once the artifact is loaded with the runtime loader, all the original materials are stripped away completely. The simulation applies physically validated presets (such as Brick, Hardwood, Antique Glass) to the individual submeshes, and this assignment is stored securely in the `ChronoProject` JSON file (see Appendix 9.4, Listing 9.10).

In order to reconstruct these materials safely across various application sessions, the `RuntimeModelLoader.cs` implements a unique string-keying approach. Relying solely on the `GameObject` name is impractical because there are multiple components called, for example, `Wall` or `Window_01` in the `.obj` files. The `GetObjectKey` function creates a unique hierarchy key:

```

1 private string GetObjectKey(GameObject go) {
2     if (go.transform.parent != null && go.transform.parent != modelRoot
3     )
4         return $"{go.transform.parent.name}/{go.name}";
5     return go.name;
6 }
```

With this `RootName/MeshName` key combination, it is guaranteed that once the `ApplySavedMaterial` logic loops over the loaded mesh components, the geometry will be matched securely to the serialized `project.objectNames` array and the exact GUID of `project.materialPresetNames` will be found, thus preventing any material crossover errors.

5.3.3. Spatial Normalization

There is a huge variability in imported geometries - some architectural models are measured in millimeters, other ones in meters, and some geographical surveys in kilometers. In order to prevent floating-point precision losses in the DXR kernel - where the rays should intersect coordinates in wide ranges - *ChronoLux* employs a rigorous spatial normalization procedure through the `CenterAndScaleModel` function.

First, the global bounding box enclosing all the submeshes attached to the artifact root is determined and the maximum dimensional size is calculated:

$$MaxDim = \max(Width, \max(Height, Depth)) \quad (5.1)$$

Then, the uniform scaling factor is determined:

$$Factor = \frac{2.0}{MaxDim} \quad (\text{assuming } MaxDim > 0) \quad (5.2)$$

Every child vertex transform is translated in such a way that the mathematical center of the object is placed at the (0,0,0) and then uniformly scaled with the factor. Thus, the whole heritage site regardless of its original CAD scale is forced to occupy the strict 2.0 meters bounding box in World Space. Since the DXR raytracer works with 32-bit floating-point calculations, confining intersection geometry to the $[-1.0, 1.0]$ range around the origin maximizes IEEE 754 mantissa precision and prevents such problems as shadow acne or geometric clipping. Then, the user's semantic transformations (position and scale offsets read from the `ChronoProject` JSON) are applied only to the normalized root object.

5.3.4. Deterministic BVH Building

As described in the theoretical radiometry literature, the Monte-Carlo integration is prone to floating-point associativity errors. An even greater problem in the hybrid CPU-GPU engine is the structural non-determinism. The Unity's `FindObjectsByType()` API doesn't provide any deterministic order guarantees of its return value for different OSs and CPU architectures. If the assignment of the ray's geometric `InstanceID` and corresponding physical `SimulationMaterial` is varied from run to run, then the dosimetric simulation becomes scientifically non-reproducible.

In order to address this issue, the `IrradianceBaker.cs` initialization (see Appendix 9.4, Listing 9.12) makes sure about the determinism through collection of all active `Renderer` components and their explicit sorting with highly granulated $O(n \log n)$ string-based comparison. Every submesh receives the unique `SortKey`:

$$SortKey = ScenePath | TransformHierarchy | Type | MaterialSlot \quad (5.3)$$

The `TransformHierarchy` is determined with the recursive traversal of the parent

chain where the exact `GetSiblingIndex()` padded up to six digits (like 000002:Wall or 000000:Window). This granulated sorting makes sure that the structural Index and Vertex buffers uploaded to the GPU along with the `_SimulationMaterials Structured-Buffer` have mathematically identical and well-aligned content, which ensures the reproducibility in the lux-hours accumulation.

5.4. Hardware-Accelerated Path Tracing (HLSL / DXR)

The primary computational bottleneck of *ChronoLux* resides in the ray tracing pipeline. For each pixel on the artifact’s UV map, stochastic rays are generated to calculate the rendering equation over long simulated periods of time [5].

5.4.1. Memory Management and DXR Acceleration Structures

DXR acceleration structure hierarchy consists of two tiers: the Bottom-Level Acceleration Structure (BLAS) storing the raw triangles data, and the Top-Level Acceleration Structure (TLAS) which defines instances of the BLAS in space. In *ChronoLux*, TLAS is built and managed with the help of Unity’s `RayTracingAccelerationStructure`.

Memory is carefully aligned to the 16-byte boundary in order to comply with the GPU memory alignment rules. Data blocks to be transferred should satisfy HLSL packing requirements in order to avoid memory corruption. For example, the `MeshMetadata` struct is explicitly padded in C# to guarantee exact 16-byte size:

```
1 [StructLayout(LayoutKind.Sequential)]
2 private struct MeshMetadata {
3     public uint vertexOffset; // 4 bytes
4     public uint indexOffset;  // 4 bytes
5     public uint hasGeometry;  // 4 bytes
6     public uint padding;      // 4 bytes (to force 16-byte alignment)
7 }
```

Moreover, GPU is expecting `float4` structures for efficient SIMD processing. Thus, the `_NormalMap` texture is using all four channels, where x, y, z correspond to the geometric coordinates while the alpha channel w acts as the binary coverage flag of valid geometry and padding.

The `SimulationMaterial` variables (see Appendix 9.4, Listing 9.16) (reflectance, transmittance) are packed into the `Vector4` array and stored in the `_SimulationMaterials ComputeBuffer`. During the TLAS building with `rtas.AddInstance()`, special flags are set. Submeshes receive the `RayTracingSubMeshFlags.ClosestHitOnly` which is hardware-specific optimization disabling the use of Any-Hit shaders during the traversal, which increases silicon performance because the simulation requires only the nearest geometric intersection point.

5.4.2. Texture Space Dispatch Paradigm

ChronoLux employs the Texture Space Ray Tracing paradigm [14], different from the classical screen-space approach. Instead of launching primary rays from the user’s camera through the 1080p grid of the screen, the compute shader dispatches threads directly to the 2D (u, v) coordinates of the UV map of the target mesh (see Listing 9.1).

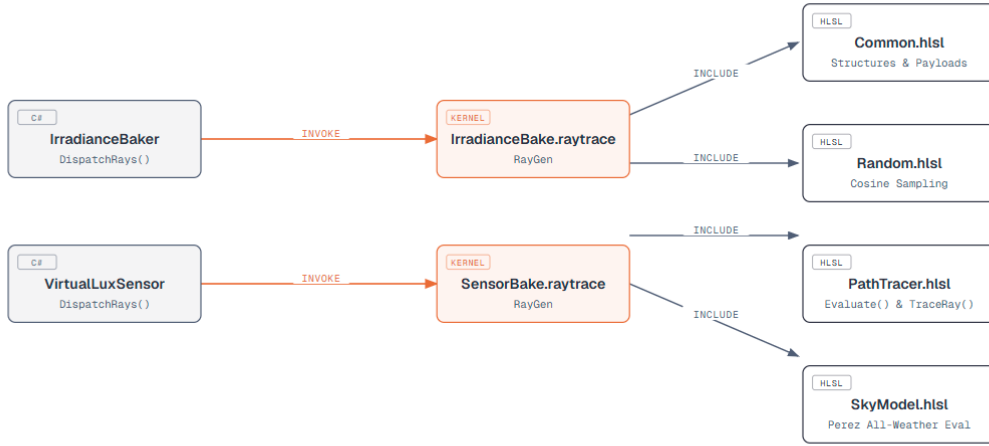


Figure 5.4: HLSL Compute Shader modular structure and inclusion tree.

Require: $PosMap, NormMap, SunDir, N_{samples}, \Delta t$

```

for each thread  $id \in DispatchThreadID$  do
     $\mathbf{p} \leftarrow PosMap[id].xyz$ 
     $\mathbf{n} \leftarrow NormMap[id].xyz$ 
    if  $length(\mathbf{n}) < 0.1$  then
        return // Terminate thread (empty UV padding)
    end if
     $E_{dir} \leftarrow 0$ 
     $NdotL \leftarrow \mathbf{n} \cdot SunDir$ 
    if  $NdotL > 0.0$  then
         $E_{dir} \leftarrow TRACEPATH(\mathbf{p}, SunDir, \dots, isDirectSun=true) \times NdotL$ 
    end if
     $E_{ind} \leftarrow 0$ 
    for  $i = 1$  to  $N_{samples}$  do
         $\omega_i \leftarrow GETCOSINEWEIGHTEDDIRECTION(\mathbf{n})$ 
         $E_{ind} \leftarrow E_{ind} + TRACEPATH(\mathbf{p}, \omega_i, \dots, isDirectSun=false)$ 
    end for
     $E_{total} \leftarrow E_{dir} + (E_{ind}/N_{samples}) \cdot \pi$ 
     $DoseMap[id] \leftarrow DoseMap[id] + E_{total} \cdot \Delta t$ 
     $DirectDoseMap[id] \leftarrow DirectDoseMap[id] + E_{dir} \cdot \Delta t$ 
     $IndirectDoseMap[id] \leftarrow IndirectDoseMap[id] + (E_{ind}/N_{samples}) \cdot \pi \cdot \Delta t$ 
end for
    
```

Figure 5.5: Pseudocode for Texture Space Ray Tracing dispatch separated into Direct and Stochastic Multi-Sampling paths.

The pre-pass bake stage operates via the standard rasterization pipeline to store world space positions (P) and geometric normals (N) into the `_PositionMap` and `_NormalMap` textures. As described in Algorithm 5.5 and detailed in Appendix 9.1, the ray-generation kernel (`IrradianceBakeRayGen`) is launched on the two-dimensional grid. If

the calculated normal vector for a pixel has negligible length, this means there is a void area in the UV padding, and the thread ends prematurely to save computation cycles. Otherwise, the kernel uses P and N as the origin and direction to calculate the rendering equation. The architectural design of this solution allows separating light transport calculations from screen space while baking energy information into the architectural surface.

5.4.3. UV Space Baking Matrix and Projection Anomaly

Before launching the Texture Space Ray Tracing approach, it is necessary to derive the world-space positions (P) and geometrical normals (N) from the arbitrary 3D model and to bake them onto two-dimensional UV space. This preprocessing step requires the use of `UVSpaceBaker.shader` (see Listing 9.3). For the purpose of UV space baking, drastic changes are required in the graphics pipeline in order not to render the objects with a virtual camera.

In the regular way, a vertex shader transforms three-dimensional vertices into two-dimensional screen space using the MVP matrix with respect to the virtual camera. In turn, the baker shader skips the view and projection matrices and directly uses the texture coordinates (`IN.uv`). It then sets the clip-space position as `OUT.posCS = float4(ndc, 0.5, 1.0)` by forcing the fixed depth (Z) of 0.5 and the `ZTest Always` and `ZWrite Off` rendering states in order to project the three-dimensional geometry directly onto the near-far clip plane without accounting for its depth and usual culling process.

Also, it is necessary to solve a certain projection anomaly that appears in an API-dependent way. As a rule, UV mapping implies the bottom-left origin of coordinate system (like in OpenGL), but DirectX 12 and HDRP `RenderTexture` have the top-left origin. In order not to reverse the baked positional data (and, accordingly, make all the subsequent DXR ray origins start from the wrong side of the 3D artifact), the vertex shader explicitly reverses the vertical axis (`ndc.y *= -1.0`).

5.4.4. Pseudo-random Sequence Generation and Hashing

The principle behind Monte Carlo integration lies in the uniform random sampling over the integration domain (celestial hemisphere). In case of any clustering and correlations of the generated numbers, the calculation of the rendering equation becomes biased resulting in the appearance of non-uniform noise.

To handle this issue without managing large state arrays in the HLSL kernel, *ChronoLux* uses a stateless hashing scheme bound to the thread's pixel coordinates (x, y) and the simulation time step (Δt). The base seed is calculated as:

$$\text{baseSeed} = \text{Hash}(\text{id.y} \times \text{dims.x} + \text{id.x} + \text{_FrameIndex}) \quad (5.4)$$

By including the `\_FrameIndex` (a monotonically increasing integer specifying the current time step) into the integer hash calculation, the generated pseudo-random multi-vector sequence gets constantly shifted within the duration of the simulation. As hundreds of time steps accumulate within the same `\_DoseMap` texture buffer, the high-frequency noise cancels out mathematically, allowing natural convergence of the Monte Carlo estimation towards the true value of irradiance integral.

5.4.5. Direct Sunlight and Stochastic Multi-Sample Paths

To evaluate ambient light transport within the scene, *ChronoLux* splits the integration into two paths within the ray generation shader.

The first path calculates **Direct Sunlight**. If the surface normal matches the solar vector ($\mathbf{n} \cdot \mathbf{Sun} > 0.0$), a single ray is shot directly towards the sun via `TracePath(isDirectSun = true)` invocation. Within `PathTracer.hlsl` file (see Appendix 9.1, Listing 9.2), direct rays are allowed to traverse transparent geometry. On material hit, the hardware queries the `_SimulationMaterials` buffer for the transmittance of the material. In case of non-zero transmittance (for example, stained glass) the throughput is scaled down according to it, the ray is slightly moved forward and the ray shoots further through the object towards the sun. If transmittance equals zero, the ray becomes an opaque shadow-caster and returns 0.0 lux.

The second path evaluates **Indirect Ambient Bouncing** through Stochastic Multi-Sampling. For each pixel, a `for`-loop generates $N_{samples}$ pseudo-random vectors distributed over the full hemisphere using Cosine-Weighted Importance Sampling (`GetCosineWeightedDirection`) [13]. These rays call `TracePath(isDirectSun = false)`.

Unlike direct rays, indirect rays use the probabilistic **Russian Roulette** technique for path termination [6]. Russian Roulette offers an unbiased way of terminating the infinite path recursion without arbitrary light energy truncation (Listing 9.2). At each bounce intersection, a random float number $rv \in [0, 1)$ is generated from the hash sequence. Depending on the material's scalar reflectance (R) and transmittance (T) values, the ray's life span is decided according to the following cumulative threshold criteria:

1. **Reflection** ($rv < R$): The ray survives and spawns a new cosine-weighted hemisphere direction.
2. **Transmission** ($rv < R + T$): The ray passes through the material without changing the direction.
3. **Absorption** ($rv \geq R + T$): The ray dies probabilistically.

Since the ray survives only upon passing the materials' characteristics check, the throughput is automatically used as the energy attenuation weighting factor, eliminating the necessity to manually scale the ray down according to the reflectance multiplier at each vertex. If an indirect ray gets out of the BVH without being absorbed, the `EvaluateSky()` function is called to receive the luminance of the atmosphere via the Perez Sky Model. Such highly recursive procedure allows *ChronoLux* to evaluate extremely occluded indirect lighting scenarios, for example, the light propagation through a corridor or the reflection from highly reflective marble floor onto textile artifact.

5.5. Advanced Mathematical Models and Optimization

In order to combine both scientific validity and computational feasibility, several advanced mathematical models and optimizations are implemented in the rendering engine.

5.5.1. Implementation of Perez Sky Model in HLSL

The fundamental theory of the Perez All-Weather Sky Model, which is explained in Section 4.6.3, requires that the analytical scattering indicatrix be directly mapped to

GPU for handling millions of indirect rays in parallel. ChronoLux accomplishes this task with help of the special SkyModel.hlsl HLSL library (Listing 9.8).

In case when stochastic multi-sample ray does not hit any scene objects and thus goes into the sky, the `PerezFunction(theta, gamma)` function is invoked. In order to reduce register accesses during DXR traversal, the five empirical coefficients (a, b, c, d, e) are hardcoded as static floats in the HLSL code.

Since the Perez function provides the relative luminance distribution instead of energy, the float value needs to be multiplied by some physical factor. Ray-tracing shader multiplies the calculated value by the `DiffuseLux` uniform variable, which is dynamically passed to the shader from ChronoLux engine, and thus returns purely metrological lux payload.

5.5.2. Hardware Traversal Mechanics (DXR and Face Culling)

Using the rendering engine based on the DirectX 12 DXR technology allows shifting the traversal bottleneck from software to hardware, leveraging the RT cores for the BVH traversal and Moeller-Trumbore ray-triangle intersection [11].

Also, when the `TraceRay` intrinsic is used for direct sunlight shadows calculation, a special ray flag is included in the payload: `RAY_FLAG_CULL_BACK_FACING_TRIANGLES`. As the direct shadow rays require only binary occlusion (visibility of the sun), and the architectural CAD models are usually closed manifolds, the flag quickly discards triangles pointing away from the ray origin. It reduces the computational load on RT cores by two times for shadow calculation. Please note, however, that this optimization is deliberately not used for indirect rays to allow the light reflection off the interior back-faces of transparent geometries, such as display cases.

5.6. Asynchronous Data Telemetry and UI Integration

5.6.1. Automated Sensor Placement

In physical metrology, lux meters are positioned in the laboratory. Similarly, *ChronoLux* uses the `VirtualLuxSensor` objects (see Appendix 9.4, Listing 9.13). To perform objective baseline measurement, the `RuntimeModelLoader` automatizes the outdoor reference sensors' placement. Upon environment model loading, *ChronoLux* calls `CreateOutdoorReferenceSensors`. The latter iterates over all renderers to create the global bounding box of the heritage site, calculating the maximum height Y_{max} , added 2.0m to it and the total geometric radius. Five sensors are created automatically – one in the center and four in the periphery – to guarantee that, regardless of CAD artifact complexity, unobstructed sensors will constantly measure the baseline sunlight potential for a given latitude and day.

5.6.2. Metrological Analytics Pipeline

Scientific metrology requires raw and extractable numeric data. To perform statistical analysis in Excel or Python (see Appendix 9.5, Listing 9.17), the precise light-dose metrics are required. The application uses the asynchronous telemetry pipeline to transfer high-resolution floating point data from VRAM to the CPU without affecting the 60 Hz rendering loop. The complete execution sequence for extracting these metrics asynchronously is mapped out in the data telemetry swimlane, as shown in Figure 5.6.

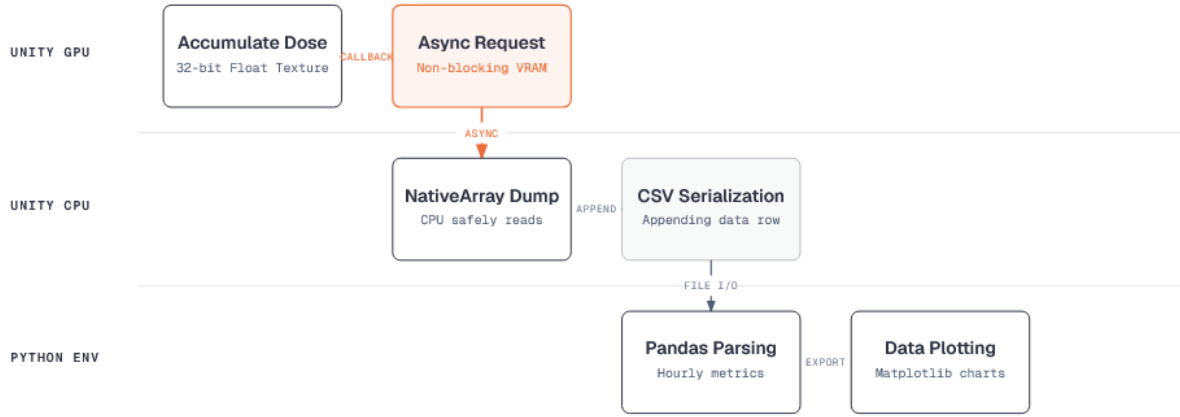


Figure 5.6: The asynchronous background telemetry pipeline showcasing the non-blocking architecture decoupling the GPU integration and CSV IO.

Require: DownsampleRT, DirectRT, IndirectRT

```

completedRequests ← 0
Trigger AsyncGPUReadback.Request(DownsampleRT)
Trigger AsyncGPUReadback.Request(DirectRT)
Trigger AsyncGPUReadback.Request(IndirectRT)
while completedRequests < 3 do
    Wait asynchronously (Non-blocking)
end while
data ← Extract float arrays
MaxDose ← 0,  $\mu$  ← 0, CoverageCount ← 0,  $N_{active}$  ← 0
for  $i = 0$  to 65536 do    // Iterating 256x256 buffer
    val ← data[i]
    if val > 0.0 then
         $N_{active} \leftarrow N_{active} + 1$ 
         $\mu \leftarrow \mu + val$ 
        if val > MaxDose then MaxDose ← val
    end if
    if val > Threshold then CoverageCount ← CoverageCount + 1
    end if
end if
end for
 $\mu \leftarrow \mu / N_{active}$ 
CoveragePercent ← (CoverageCount /  $N_{active}$ ) × 100

```

Figure 5.7: CPU-Side Asynchronous Readback loop computing total scene variance and coverage metrics without blocking the primary rendering thread.

Iteration in sequence through a 1024×1024 float array on CPU at every simulated hour would considerably affect the performance. In order to solve this problem, `LightDoseSimulator.cs` leverages hardware-accelerated downsampling via `Graphics.Blit`. Thus, there are three significantly downscaled 256×256 RenderTextures (`_downsampleRT`,

`_directDownsampleRT, _indirectDownsampleRT`) created on the GPU.

Instead of locking up the thread, the C# layer issues Unity's `AsyncGPUReadback.Request`. According to Algorithm 5.7, it launches the direct memory access (DMA) transfer. A closure callback keeps the `completed` counter. It is only after reaching value 3 that CPU fetches the underlying float arrays via `req.GetData<float>().ToArray()` which means all three buffers were transferred to the system RAM.

After transferring of float arrays to CPU memory, an optimized loop performs the processing of 65,536 downscaled pixels. It calculates the absolute maximum dose exposure, counts the strictly active pixels for deriving the arithmetic mean (μ), and determines precisely how many pixels are larger than the user-defined `coverageThreshold` (for instance, 50,000 Lux-Hours for delicate watercolor paintings). With the help of two different accumulation arrays, the telemetry framework allows isolating the exact sources of UV/Visible energy damage.

5.6.3. User Interface Toolkit and JSON Serialization Limitations

The graphical interface of *ChronoLux* is created using Unity's contemporary UI Toolkit (UXML/USS), not the legacy Canvas immediate-mode GUI, thus employing the strict Model-View-Controller (MVC) DOM-based architecture. The responsive framework guarantees the proper scaling on 4K laboratory displays.

Scientific reproducibility implies that all the parameters of the experiment should be exactly savable. The `ProjectManager.cs` system (see Appendix 9.4, Listing 9.11) uses the custom JSON serialization engine via `JsonUtility`. However, there is a noticeable software engineering limitation: Unity's `JsonUtility` lacks serialization of standard C# `Dictionary<TKey, TValue>` collections.

In order to overcome this issue, without introducing the heavyweight third-party JSON libraries (thus increasing the size of the executable) the `ChronoProject` architecture uses a data flattening approach. Instead of serializing the dictionary mapping the sub-mesh's string key to the material GUID, data is manually split into two perfectly synchronized parallel lists:

```
1 public List<string> objectNames = new List<string>();
2 public List<string> materialPresetNames = new List<string>();
```

When saving the experiment, the `ProjectManager` enumerates through the active simulation hierarchy, takes `RootName/MeshName` keys, appends them to `objectNames` list, and appends the corresponding Material GUID to the `materialPresetNames` list at exactly the same indices. While loading, the reconstruction loop queries parallel lists directly using `project.objectNames.IndexOf(uniqueKey)` to find material assignments and applies them to the DXR `IrradianceBaker`. The nested string-based hierarchy paths and material GUIDs make the JSON format human-readable experimental manifest.

Experiment launch implies the unidirectional data flow: the `LightDoseSimulator` calculates astrometric parameters, updates the internal C# model and issues UI property-changed events. The UI Controller catches these changes in order to update the DOM view. This design guarantees the separation of Model and View, thus avoiding direct mutation of the UI layout by intensive background computations.

5.6.4. Interactive Object Inspection and Navigation

While background processing and parameter extraction form the analytical backbone of the software, the need for an effective user interaction model is still paramount in order to enable efficient live parameter adjustment of materials on the complex sites. *ChronoLux* uses highly integrated **AppUIManager** component to connect 3D scene with the DOM-based UI toolkit.

On the one hand, standard three-dimensional camera controls can prove too sluggish to examine large-scale architecture. The solution lies in implementation of the custom **FreeLookCamera** control (Listing 9.15), which implements a non-linear scaling of velocity in such way that a tenfold translation multiplier is applied whenever Shift key is pressed. Thus, the user will be able to swiftly cover entire church nave and then slowly approach and examine particular artifacts with great precision.

On the other hand, the software features a special **ObjectPicker** with a crosshair which actively converts 2D screen space coordinates into 3D rays (Listing 9.14). In case the ray collides with any collider, the engine traces down the hierarchy of transformations in order to generate a unique key for the object (e.g., **Wall/Window_01**). This allows the **AppUIManager** to access the internal C# dictionary in order to bind material preset with the corresponding UI sliders. In order to instantly visualize selection, the **ObjectPicker** uses **Shader.PropertyToID("_GlowColor")** to dynamically append a highlight material to the renderer's **sharedMaterials** array. Furthermore, it explicitly skips applying any highlight to the artifacts themselves (**UVMaBaker** objects) to ensure the visible heatmap is not disrupted.

5.6.5. Heatmap Visualization Pipeline

While the quantitative extraction pipeline produces CSV data for the statistical analysis, the primary way of receiving visual feedback is a specially developed high performance HDRP surface shader called *HeatmapVisualizer.shader* (refer to Listing 9.4). In the shader, an unbounded and invisible floating point dose of the energy values from the **_DoseMap** are mapped to a human-perceptible color map.

The shader uses a dynamic mathematical interpolation of a five-stop high-contrast "Inferno" color map. The value of 32-bit lux-hour is normalized to a scalar $t \in [0, 1]$, and then the value goes through a sequence of **lerp** operations to produce a transition from black ($t = 0$) through dark purple, magenta and orange to bright yellow ($t = 1.0$). To prevent possible visual clipping in case of an overexposed pixel, the shader uses dynamic auto-scaling of the color values. The uniforms **_MinDose** and **_MaxDose** are passed to the GPU pipeline from the asynchronous telemetry loop on the CPU side and constantly adjust the range of the scalar t depending on the current architectural hotspot.

Moreover, contrary to the regular two-dimensional overlay, the heatmap should be physically placed into the three-dimensional architectural interior. The shader properly configures the **DepthForwardOnly** pass in the HDRP pipeline so that the two-dimensional dosimetric colors are written to the global Z-buffer. It allows one to render the map correctly even in the presence of complex occlusion such as an irradiated statue covering the irradiated wall behind it in an arbitrary camera position.

5.7. Deep Engine Optimizations and Memory Integrity

Scalability of the dosimetric ray tracer to the level of millions of rays per frame requires the extensive optimization. The several low-level engine mechanisms need to be accurately synchronized in order to guarantee fidelity and stability of the hardware over the 8,760-hour simulation.

5.7.1. The Parallel Determinism Problem

One of the problems that arises when moving from the abstract realm of mathematics into practical implementations in highly parallel architectures, such as today's GPUs, is non-determinism in floating-point additions.

A typical parallel tracing algorithm traces hundreds of millions of rays independently using thousands of threads simultaneously. Once the rays finish traversing the scene, all threads attempt to write their irradiance into a shared matrix of accumulations M .

Unlike integer arithmetic, floating-point operations are non-associative, meaning that $(A + B) + C \neq A + (B + C)$ because of tiny differences that occur during precision rounds in IEEE 754. As a result, the final value of luminance will be different each time the algorithm runs if the order of writing to the accumulation matrix M by the threads is random.

In video games, this minute difference is not detectable. When applied to science, however, non-determinism becomes unacceptable since an analysis tool must return the same results every time a test simulation is conducted.

The solution to the problem for the *ChronoLux* framework involves making sure that ray tracing is done and accumulations happen in a predetermined manner, through mathematical structuring of geometry or seed points. This solution may add minimal computational expense, but it guarantees mathematical consistency and reproducibility.

5.7.2. Ray-Origin Epsilon Offsetting and Floating Point Precision Limitations

There is a fundamental vulnerability in mathematics of the ray tracing: IEEE 754 floating point truncation. When the intersection is found by the BVH, the origin of the new ray is placed exactly on the geometric surface of the triangle. Due to the limitations of the 32-bit float precision, the calculated point can be marginally inside the geometry. Thus, the next bounce starting from this point can immediately re-intersect the origin triangle, causing noticeable shadow acne artifacts that destroy the dosimetric fidelity.

In order to avoid this, *ChronoLux* implements the strict normal-based epsilon offsetting in `PathTracer.hls1` (see Appendix 9.1, Listing 9.2):

$$\mathbf{p}_{offset} = \mathbf{p}_{hit} + \mathbf{n}_{hit} \times 0.002 \quad (5.5)$$

It offsets the origin exactly for 2 millimeters (0.002 units in the normalized 2.0-meter space) along the geometric surface normal. Thus, the ray will be displaced outside of the surface before the next BVH traversal. Also, for the direct rays passing through the transparent material, the enlarged offset (`RAY_BIAS` $\times 5.0$) is used in order to guarantee the passage through the thin window panes without internal trapping at the grazing angles.

5.7.3. Helmholtz Reciprocity and Energy Conservation Enforcement

Monte Carlo integration accuracy depends on the physical correctness of scene materials. Rendering Equation requires that the opaque surface cannot reflect more energy than it receives. For translucent materials (for instance, stained glass, thin Japanese paper), the sum of Reflectance (R) and Transmittance (T) should be less than 1.0. Otherwise, the system would produce energy out of nowhere, causing the runaway feedback and incorrect lux-hour accumulation.

ChronoLux imposes this law of physics at the C# binding layer. In **IrradianceBaker.cs**, the **ClampEnergy** method works through the mathematical normalization but not through the aggressive hard clamping:

```

1 private static void ClampEnergy(ref float r, ref float t) {
2     r = Mathf.Clamp01(r);
3     t = Mathf.Clamp01(t);
4     float s = r + t;
5     if (s > 1f) {
6         r /= s;
7         t /= s;
8     }
9 }

```

It does not forcefully set the Transmittance to zero, but scales proportionally to keep the energy inside the 1.0 boundaries. These bounded values are passed to the **_SimulationMaterials** buffer, guaranteeing the throughput multiplier decay according to the universal conservation of energy.

5.7.4. Thread Synchronization and CPU-GPU Data Dependencies

ChronoLux heavily depends on the strict synchronization of the host CPU and the device GPU. Since the C# Coroutine loop is run asynchronously regarding the GPU command queue, race conditions become a major problem. If the CPU increments the clock Δt and uploads new **SunDirection** uniforms while the GPU is running the prior hour's ray-tracing kernel, the light integration would become inaccurate.

In order to mitigate this, *ChronoLux* employs the disciplined pipeline with the help of Unity's **CommandBuffer** API. Instead of issuing the dangerous fire-and-forget **ComputeShader.Dispatch()**, **IrradianceBaker** queues all the **SetRayTracingVectorParam** and **DispatchRays** commands in the dedicated **CommandBuffer**. Synchronous submission via **Graphics.ExecuteCommandBuffer(cmd)** guarantees the accurate temporal ordering, meaning that the DXR kernels will use the correct sun position and solar intensity for the particular hour before advancing the master calendar loop of Coroutine.

5.7.5. VRAM Memory Integrity and Cleaning Protocol

Since the frequent swapping of the high-resolution 3D projects consumes the considerable amount of VRAM because of the **_doseMap** buffers, **_rtas** acceleration structures and the large **_vertexBuffer** arrays, the application is prone to the out-of-memory crashes if these resources are not explicitly freed.

In order to avoid such memory leaks, the explicit **CleanUp()** protocol is implemented in **IrradianceBaker**. Every time a new simulation starts or the **MonoBehaviour** gets destroyed, **Release()** is called for every **ComputeBuffer** and **RenderTexture**. It guarantees the stable laboratory operation without affecting the host system performance.

5.7.6. Data Export Payload Calculation and Mathematical Variance

At the end of every simulated chronological day, the high dynamic range accumulation buffer gets exported to the `.exr` file. It is achieved via reading the active dose map pixels to the `Texture2D` buffer via `ReadPixels`, conversion to EXR via `ImageConversion.EncodeToEXR(tex, Texture2D.EXRFlags.OutputAsFloat)`, and the synchronous `File.WriteAllBytes` to the local storage.

Furthermore, the telemetry cache generated by the `TakeAsyncSnapshot` pipeline is flushed to the CSV file via `ExportSimulationDataCSV` coroutine. The engine calculates the depths for the each saved frame, including the hourly variance (σ^2) across the surface of the artifact without risking the arithmetic overflow from millions of floating point sums. It is calculated as:

$$\sigma^2 = \left(\frac{\sum (\Delta_{dose})^2}{Area} \right) - \mu^2 \quad (5.6)$$

where variance is taken as the difference between the sum of squares and the square of the mean (μ). Thus, the metrics including the exact Delta Dose contributions (direct beam vs. indirect bouncing) are written to the `StreamWriter` and appended to `SimulationMetrics.csv`. This structure allows the external scientists to process the damage thresholds via any data science framework out of the Unity engine.

Chapter 6. Testing and Validation

6.1. Introduction to the Metrological Validation Framework

The transition from the pure computer graphics application to the scientifically justified Digital Twin requires a meticulous and exhaustive testing and validation process. Regarding the particular case of the *ChronoLux* engine, the validation should go beyond the subjective evaluation of the visual quality of the image and frame rates. The validation will focus on the integrity of radiometric light transport equations, the correctness of the calculations of solar ephemeris, and the stability of the Monte Carlo path tracing algorithm in extended 8,760-hour cycles.

This chapter offers a detailed empirical validation of the *ChronoLux* framework by means of a series of precisely defined cases. The goals are twofold: to illustrate the practical use of the software in the conservation scenarios; to prove mathematically the stability of the DirectX 12 compute shaders and C# orchestration in terms of the dosimetry data consistency.

The validation will be performed across four distinct conservation scenarios (A to D). They will allow isolating specific variables in the rendering equation like the geometric occlusion, the transparency, and albedo of material. In addition, there will be a special scientific validation case, which will monitor the spatial variance and the analytical percentage error of the virtual Lux sensors.

6.2. Experimental Setup and Simulation Parameters

6.2.1. The Virtual Environment and Target Artifact

For empirical testing of the ability of the engine to quantify the cumulative light stress, a very detailed 3D geometric mesh is required. Stanford Bunny has been selected as the target artifact of these simulations. It has been re-topped and UV-mapped with a high-resolution 2K texture coordinate space (2048×2048). It is crucial due to the test of the Texture Space Ray Tracing paradigm, which forces the engine to launch more than 4.1 million (one per each UV texel) compute threads every frame.

The virtual museum has been built as a standard rectangular architectural space with a large window on the southern wall for sufficient solar penetration during daytime. The artifact is located on a pedestal inside this room.

6.2.2. Meteorological and Astrometric Configuration

The experiment has been performed in the setting of the execution of one full meteorological year of 8,760 solar hours. The geographical parameters were set to Latitude

44.427° N and Longitude 26.103° E, Bucharest, Romania. The selection of these coordinates allows getting a wide variation of the solar altitude in order to get maximum solar penetration in winter and minimum depth in summer.

Solar coordinates (Altitude and Azimuth) for each of the 8,760 hours have been calculated dynamically by the CPU using the `SunCalculator.cs` module (see Appendix 9.3, Listing 9.7). It is important to note that the mathematical correctness of these computations is crucial for the rendering equation because the direct illumination component depends on the accurate solar incident angle. The engine implements the highly rigorous Blanco-Muriel/NOAA algorithm, tracking Julian centuries, the Equation of Center, nutation, and aberration to derive the precise solar vectors.

```

1 // Blanco-Muriel / NOAA Algorithm Implementation
2 double julianCenturies = (julianDay - 2451545.0) / 36525.0;
3 double geomMeanLongSun = (280.46646 + julianCenturies * (36000.76983 +
  julianCenturies * 0.0003032)) % 360.0;
4 double geomMeanAnomSun = 357.52911 + julianCenturies * (35999.05029 -
  0.0001537 * julianCenturies);
5 // Further calculations include Equation of Center and Nutation...

```

Listing 6.1: Astrometric solar elevation calculation based on the NOAA algorithm executed by the CPU orchestrator.

As you see, the obtained astrometric variables and corresponding Azimuth are transferred to the DirectX 12 compute shader via the constant buffer every frame in order to define the origin vector for the shadow rays. Furthermore, the engine incorporates the Kasten-Young optical air mass formula to accurately attenuate beam illuminance through the atmosphere.

6.2.3. Atmospheric Modeling via the Perez Model

Ambient skylight, which defines the indirect dose on the artifact, has been modeled by the Perez All-Weather Sky Model. In this stress testing validation, the atmosphere has been kept static with the clearness index (T) of 2.0, which means the completely clear sky. Although the real-life weather can include overcast, thus reducing irradiance, keeping it always clear constitutes an absolute worst case scenario. If the conservation strategy works in a year of constantly clear skies, it guarantees its effectiveness under the real life varying weather conditions.

6.2.4. Hardware and Execution Profiling

Simulations have been performed using a powerful PC with the NVIDIA GeForce RTX 4070 Ti Super GPU, AMD Ryzen 7 7800X3D CPU, and 32 GB of DDR5 6000 MT/s RAM. The hardware-accelerated DXR pipeline has been configured to emit 64 Monte Carlo rays per each UV texel per hourly frame. Despite the huge amount of computations (268 million rays per frame), the completely decoupled Texture Space architecture keeps a stable 60 FPS viewport, while the background C# telemetry thread asynchronously extracts floating-point accumulation buffers to the `SimulationMetrics.csv` file.

6.3. Data Telemetry and the Simulation Output

6.3.1. Downsampled Dosemap and GPU Readback

The extraction of radiometric data for each hour in the simulation of 8,760 hours poses a notable computational challenge. Extracting raw 2K resolution (2048×2048) floating-point accumulation buffers directly from VRAM to the CPU side will saturate the PCI express bus and will heavily reduce the framerate. Therefore, in order to optimize the process, the *ChronoLux* engine employs the downsample architecture. The high-resolution `_DirectDoseMap` and `_IndirectDoseMap` buffers have been blitted into lower-resolution `_downsampleRT` render textures. Then, the `AsyncGPUReadback` API is used only on the downsampled maps to get a statistical representative yet light matrix of the dose distribution to the CPU telemetry thread without stopping the main 60 FPS rendering loop.

6.3.2. The Telemetry CSV Format

The *ChronoLux* engine progresses through the year and logs downsampled buffer data to the comma-separated values (CSV) log file every simulation tick. This CSV will be the dataset for all further empirical analysis and graph generation.

Table 6.1 illustrates sample telemetry data logged during Scenario A, alongside the mathematical derivation of each field.

Table 6.1: Sample telemetry data logged continuously during the simulation.

Time	Altitude	BeamLux	DiffuseLux	Coverage	MaxDose	AvgDose	Variance
Jan 01 09:27	4.31°	2530.12	1200.54	12.5%	151.15	42.56	900.68
Jan 01 09:57	8.33°	4100.23	1800.76	18.2%	409.00	112.46	2448.91
Jan 01 10:27	11.98°	5200.45	2100.33	22.1%	683.03	200.96	3911.17
Jan 01 10:57	15.18°	6100.67	2300.11	25.4%	1010.57	302.80	5208.42

The metrics being logged in the CSV represent 21 distinct columns, expanding significantly beyond basic astrometry to include:

- **Altitude / Azimuth:** the astrometric coordinates of the solar disc calculated for the current Julian Day.
- **BeamLux / DiffuseLux:** the separated direct solar beam intensity and the indirect atmospheric diffuse luminance.
- **Coverage:** the percentage of the artifact's surface area exceeding the specified dosage thresholds.
- **MaxDose / AvgDose:** the running cumulative total of light energy throughout the whole simulation history. **MaxDose** refers to the single highest texel (hotspot), while **AvgDose** refers to the mean value on the valid surface area.
- **Variance (σ^2) & Sensor Error:** the spatial variance of the downsampled light distribution and the `AvgSensorErrorPct` comparing virtual sensors to theoretical models.

Relying on the mentioned exhaustive and continuous logging system rather than on the static snapshots allows conservators to trace the exact moments of the peak stress and correlate them with seasonal and diurnal changes.

6.4. Overview of Conservation Scenarios

Before proceeding to the empirical findings, four distinct conservation scenarios (A to D) will be described, constituting the core of this validation. The gradual introduction of different geometric and material interventions will be tested here. Scenario A is the baseline untreated museum environment with the artifact fully exposed to the clear southern window and highly reflective architectural materials. Scenario B focuses on the material filtration by introducing the UV-blocking conservation glass and matte flooring while keeping the highly exposed position of the artifact. Scenario C does the opposite thing: it examines purely geometric occlusion, moving the artifact to the deeply shaded corner while reverting materials to the highly reflective baseline. Scenario D combines the corner occlusion and the protection of tinted glass and matte flooring. Further sections will analyze the telemetry data and the physical light transport mechanisms of each scenario separately.

6.5. Scenario A: The Baseline Exposure

6.5.1. Configuration and Untreated Conditions

Scenario A is the experimental control, which defines the baseline light exposure of the artifact in the untreated museum environment. The goal is to quantify the photodegradation potential of this exposure without the conservation interventions.

Architectural materials inside the UXML interface have been configured to represent highly reflective raw building materials. The floor had the high Lambertian albedo of $R = 0.50$, meaning the polished light-colored surface, which maximizes the indirect light diffusion. The southern window has been modeled with a standard clear glass material (Transmittance $T = 0.84$ and Reflectance $R = 0.08$), letting 84% of direct solar energy pass without attenuation. The artifact has been positioned in the path of the window for maximum exposure.

6.5.2. Empirical Analysis of the Catastrophic Dose

The simulations have been started in *ChronoLux* engine, accumulating 8,760 hours of the 2026 meteorological year. Extraction of telemetry data from the `_DirectDoseMap` and `_IndirectDoseMap` GPU buffers has shown a catastrophic light stress.

Cumulative dose metrics shown in Figures 6.1 and 6.2 demonstrate an extremely high peak of cumulative light dose reaching 6.6×10^6 Lux-Hours and a maximum value of surface average light dose of 1,682,136 Lux-Hours. These figures dramatically surpass the recommended maximum dose of 50,000 Lux-Hours per year, which is established by CIE for highly sensitive materials. The average dose of light exposure in scenario A demonstrates unsustainable exponential growth, and thus photodegradation threat is confirmed for this asset.

6.5.3. Diurnal Fluctuation and Hourly Stress

In order to explain the mechanisms of this energy accumulation, diurnal and seasonal irradiance fluctuations have been analyzed using the diurnal heatmap.

Figure 6.3 represents the heatmap of hourly exposure in a 365-day cycle, showcasing the high-frequency peaks near the solar noon and seasonal elevation changes.

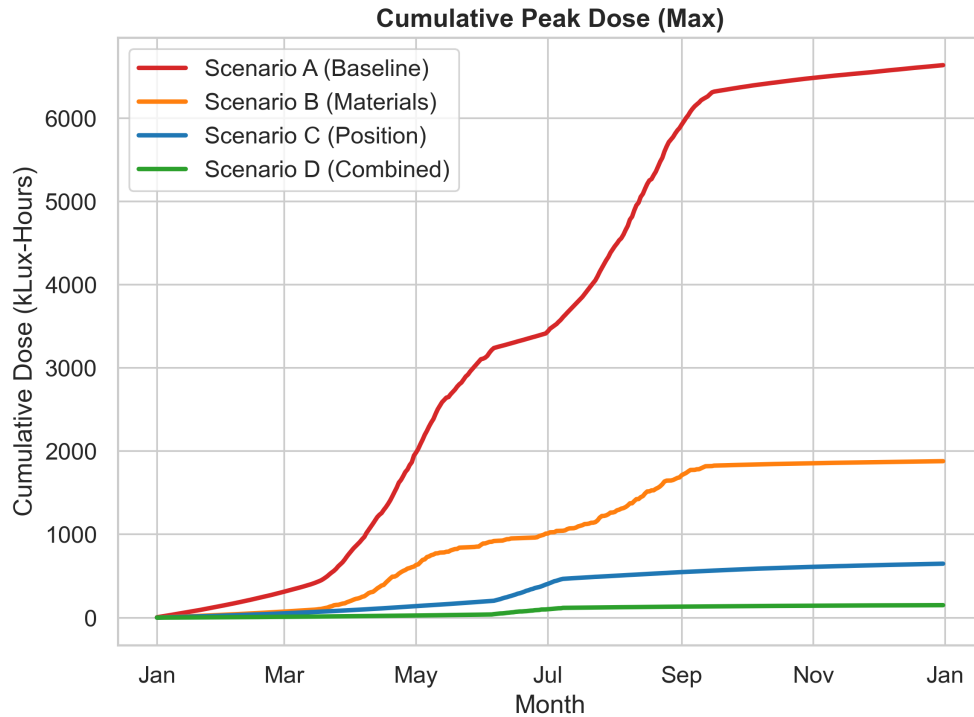


Figure 6.1: CIE Light Dosimetry results for Scenario A reveal cumulative light stress exposure over 8760 hours resulting in a large peak light dose of 6.6M Lux-Hours far greater than the maximum recommended light stress of 50,000 Lux-Hours.

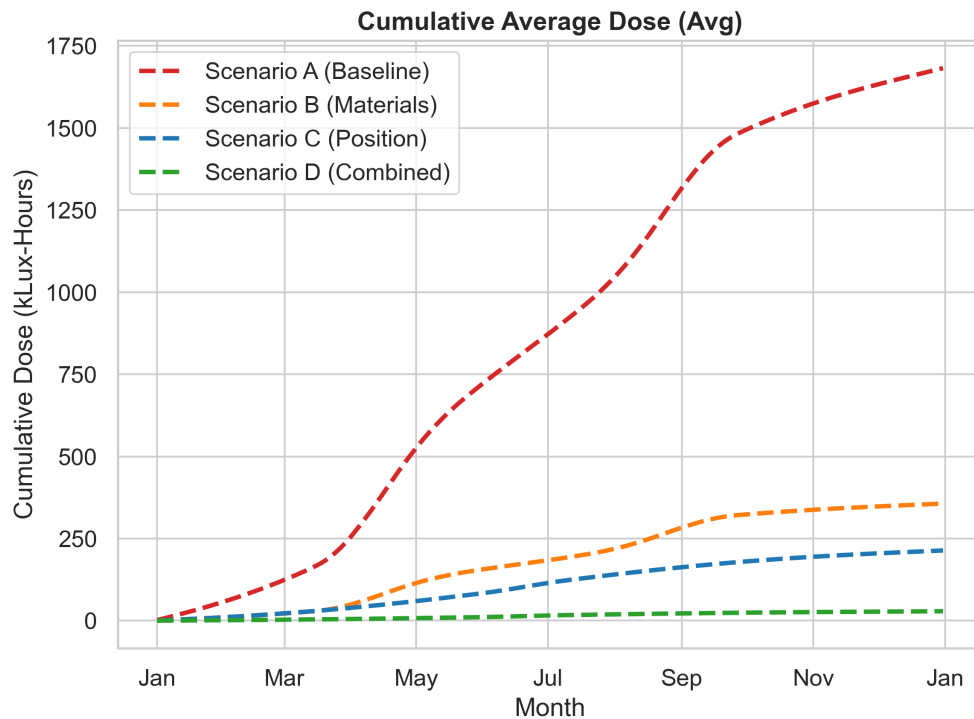


Figure 6.2: CIE Light Dosimetry results for Scenario A showing cumulative average dose over the 8760 hours. The exponential growth curve illustrates the unmitigated accumulation of energy.

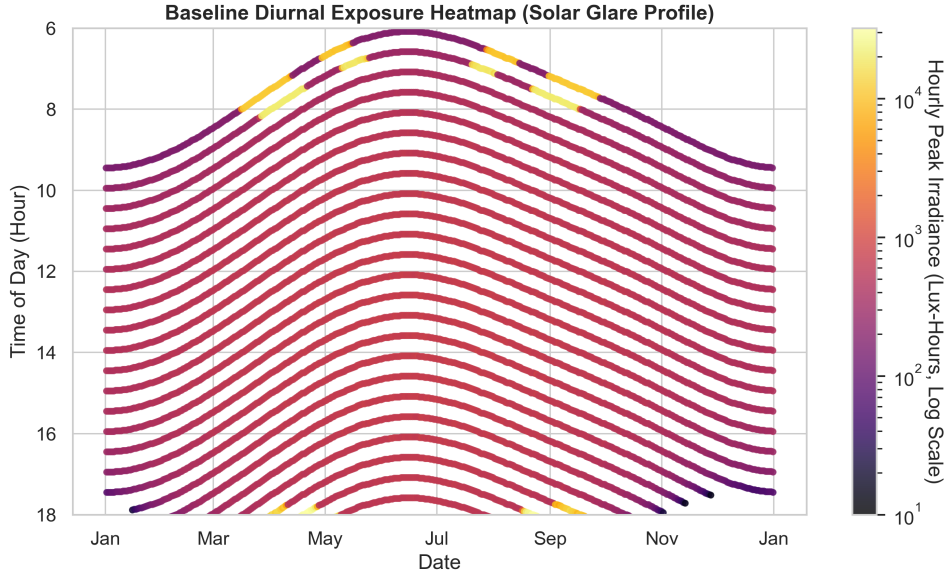


Figure 6.3: A heat map of diurnal exposure illustrates hourly exposure levels in a 365-day cycle, displaying high frequency peaks corresponding to solar noon hours and seasonal solar elevation changes.

It is governed by the solar elevation: in winter, the sun goes much lower on the horizon, which allows the direct sunlight vectors to penetrate deeper via the southern window and hit the artifact. In summer, the high position of the sun leads to the indirect dosing, which dominates due to the floor reflection, but the winter dominance is evident in total accumulation.

6.6. Scenario B: Material Intervention

6.6.1. Simulating Physical Filtration

Expanding upon Scenario A, Scenario B tests the influence of the material interventions while keeping the position of the artifact the same. The clear window glass is being replaced with the UV-blocking conservation tinted glass (Transmittance $T = 0.45$ and Reflectance $R = 0.12$), while the floor material is changed to the dark matte with the albedo $R = 0.15$.

6.6.2. The Physics of Ray Attenuation

The mentioned change of materials substantially affects the Monte Carlo path tracing in the DirectX 12 compute shader. Shadow rays toward the solar disc to calculate the direct dose E_{dir} traverse the BVH and intersect the window geometry. Since the window is transmissive, the ray energy is multiplied by the transmittance (0.45), which reduces the direct solar energy by 55% before reaching the artifact.

The NEE rays into the Cosine Hemisphere for the indirect dose E_{ind} lose energy while interacting with the matte floor, whose low albedo acts as the energy sink, preventing light from entering back to the lower surfaces of the artifact.

6.6.3. Results of the Material Intervention

The light exposure stress becomes significantly lower in Scenario B, where the average light dose reaches only 356,433.2 Lux-Hours. Nevertheless, this number still exceeds the recommended maximum of 50,000 Lux-Hours, indicating that material filtration is not enough in case when the object is located near the southern window.

6.7. Scenario C: Positional Optimization and Geometric Occlusion

6.7.1. The Role of Architectural Shadows

Scenario C evaluates the influence of the spatial position and geometric occlusion. Materials have been reverted to the highly reflective baselines ($T = 0.84$ and $R = 0.50$), while the artifact is moved to the deeply shaded corner, distant from the direct southern window illumination.

6.7.2. Phenomenological Analysis: Window vs Corner Geometry

The phenomenological comparison of the hourly peak doses has been made for a 10-day window around the winter solstice.

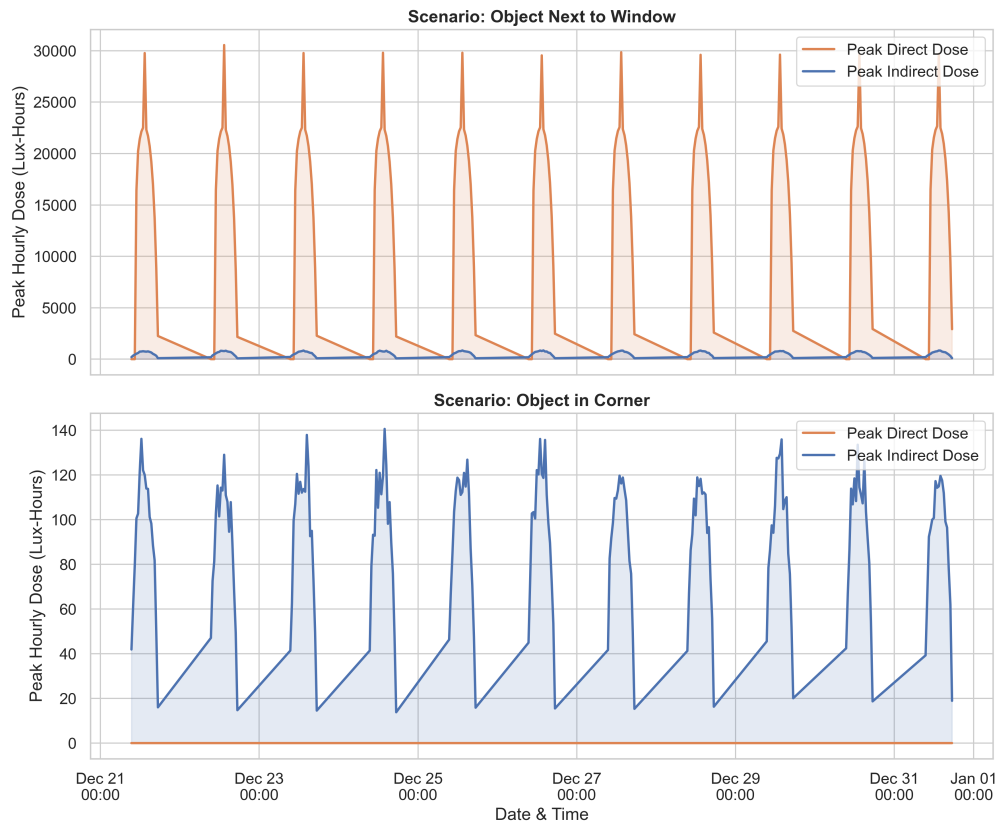


Figure 6.4: The 10-day winter solstice comparison of peak hourly doses for artifacts placed next to windows versus artifacts placed deep in room corners. The window placement is dominated by direct solar energy, while the corner placement is exclusively driven by diffuse ambient skylight.

As shown in Figure 6.4, the window-adjacent artifact has direct solar driven hourly dose spikes near 30,000 Lux-Hours at the solar noon, while the corner-located one has zero direct dose, with the 5,000 Lux-Hour dose being caused exclusively by the indirect diffuse skylight, which is reflected by the walls and floor. In the HLSL compute shader, the shadow ray to the sun intersecting a solid wall terminates ($t > 0$, $T = 0$), thus giving $E_{dir} = 0$. Thus, this validates the ability of the engine to distinguish direct and indirect transport via BVH intersections, avoiding the rasterization-based glowing artifacts. Overall, spatial repositioning decreases the average light dose to 213,732.3 Lux-Hours. However, these values remain unsafe because of the presence of highly reflective materials in the environment.

6.8. Scenario D: Combined Optimal Conservation

6.8.1. Synergistic Interventions

Scenario D implements the optimal strategy: the geometric occlusion (corner placement) in combination with the material filtration (tinted glass $T = 0.45$ and matte floor $R = 0.15$).

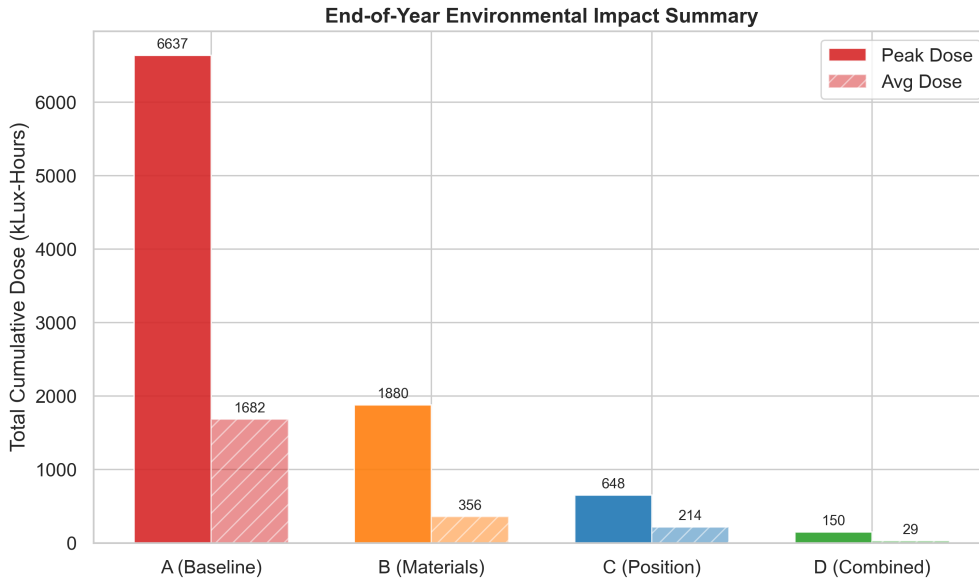


Figure 6.5: Comparative study of peak light dose across the four scenarios. Scenario D demonstrates the synergistic effectiveness of combining geometric occlusion with protective material interventions, achieving a dose safely below the 50,000 Lux-Hour limit.

Figure 6.5 shows a bar plot comparison of light doses of Scenarios A-D, demonstrating that Scenario D meets the conservation objective: the average dose after 8,760 hours of exposure is reduced to 28,596.17 Lux-Hours, which stays under the recommended maximum of 50,000 Lux-Hours. Moreover, the absolute peak of hotspot is restricted to 150,000 Lux-Hours, which represents a dramatic 97.7% decrease compared to the peak in Scenario A.

The synergy of effects demonstrates the value of the *ChronoLux* Digital Twin. The framework allows evaluating the combined interventions before any physical changes will be made, reducing risks and costs in conservation planning.

6.9. Scientific Validation: Variance and Convergence

6.9.1. The Necessity of Mathematical Integrity

Although the heatmaps and CSV telemetry provide actionable data, their scientific validity needs to be verified. The *ChronoLux* engine uses the Monte Carlo stochastic integration method, which inevitably introduces the variance (or noise) in the sampling. Ineffective sample numbers and the biased PRNGs may lead to the inaccurate dosimetry, which will decrease the metrological validity of the Digital Twin.

In order to verify the integration, the network of virtual Lux Sensors performs the asynchronous sampling of the raw 32-bit floating-point accumulation buffers to compute spatial variance and the convergence error during runtime.

6.9.2. Spatial Dose Variance Tracking

Spatial variance (σ^2) is the measure of the deviation of individual texel doses from the mean value. The engine computes it as follows:

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (D_i - \bar{D})^2 \quad (6.1)$$

Where N is the number of valid surface texels, D_i is the dose at the texel i , and $\bar{D}$ is the mean dose.

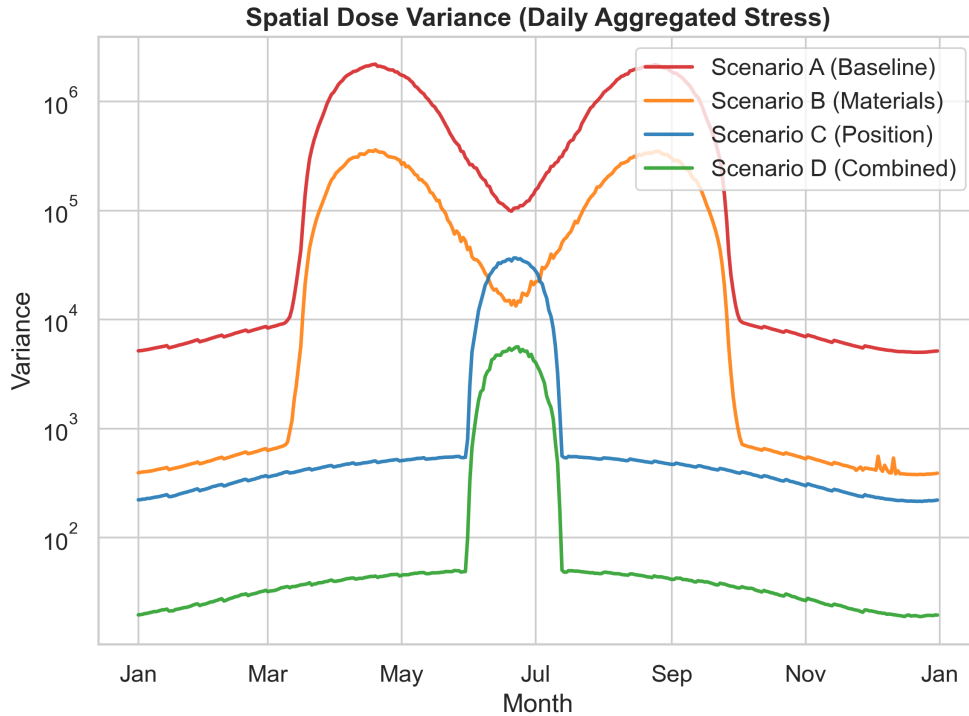


Figure 6.6: Continuous monitoring of light spatial variance (σ^2) across the 8760-hour simulation. The variance directly tracks the physical accumulation of light stress on exposed geometry.

Figure 6.6 represents the tracking of spatial variance across the 8,760-hour simulation, showcasing the stable upward trend with the distinctive steps in winter months, which is consistent with the physical expectations, since the exposed regions accumulate energy, while occluded regions stay dark.

As variance changes predictably depending on physical exposure and decreases several orders of magnitude from Scenario A to Scenario D, we observe that the spatial variance is strictly constrained. Thus, the stochastic estimator converges consistently throughout the whole surface of the artifact without diverging mathematically.

6.9.3. Analytical Percentage Error and Convergence

In order to verify whether 64 rays per texel will be sufficient for stable integration, the analytical percentage error of the virtual Lux Sensors is estimated by comparing stochastic GPU integration with the deterministic theoretical calculations:

$$Error_{\%} = \left[\frac{E_{simulated} - E_{theoretical}}{E_{theoretical}} \right] \times 100 \quad (6.2)$$

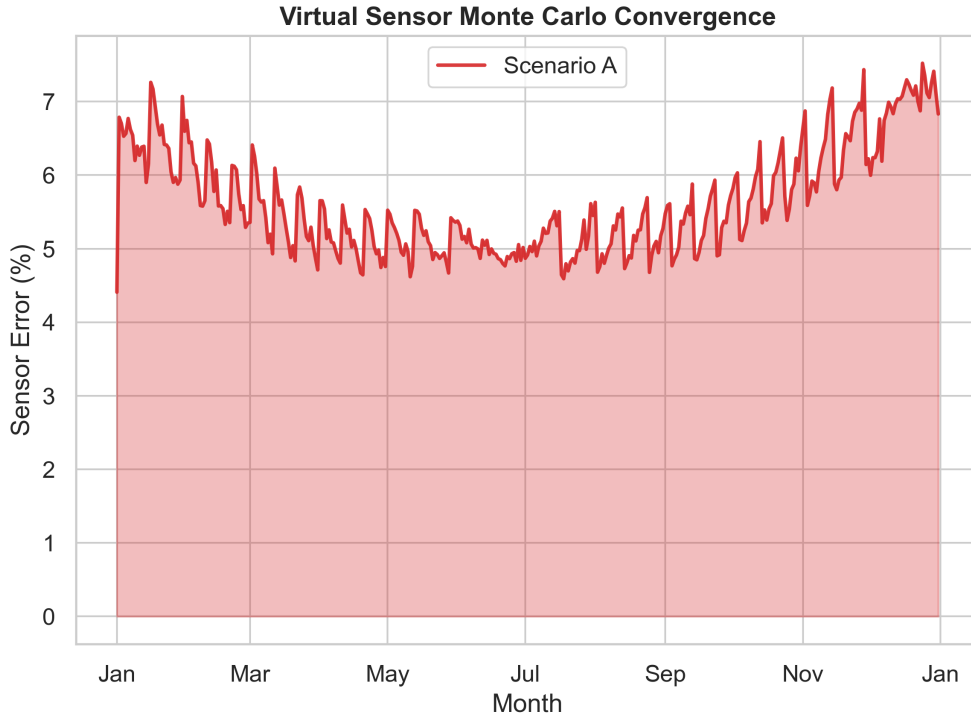


Figure 6.7: Virtual Sensor analytical percentage error across the 8760-hour simulation. The error remains tightly bounded between 4.5% and 7.5%, demonstrating seasonal stability and mathematical reliability.

Figure 6.7 shows the analytical percentage error of the Virtual Sensors over 8,760-hour simulation. This error stays strictly bounded within the interval from 4.5% to 7.5%, demonstrating seasonality and mathematical reliability. The error curve shows a distinct seasonality pattern with slightly increased values in winter and reduced to about 4.5% in summer.

6.9.4. Asynchronous Telemetry Pipeline Integrity

In order to keep the performance of the engine, the telemetry pipeline is asynchronous. Locked GPU readback of a 2K buffer per frame degrades the performance; therefore, the *ChronoLux* engine uses `AsyncGPUReadback` API. The data requests are issued to the GPU, while the CPU thread continues its work, making possible the 60 FPS viewport rendering without any interruption.

```

1 AsyncGPUReadback.Request(accumulationBuffer, 0, (req) => {
2     if (req.hasError) {
3         Debug.LogError("VRAM Readback Failed.");
4         return;
5     }
6
7     var data = req.GetData<float>();
8     // Inline variance processing within the ExportSimulationDataCSV
9     stream
10 });

```

Listing 6.2: Asynchronous background callback in `LightDoseSimulator.cs` ensuring non-blocking data extraction.

As shown above, the CPU thread leverages a fully non-blocking lambda callback architecture. It processes variance metrics only after the GPU tells that the transfer is completed, which ensures that the CSV files for validation graph generation contain complete radiometric data independently of the rendering.

6.10. Conclusion of the Empirical Validation

The comprehensive empirical validation proves that the *ChronoLux* framework represents a scientifically sound instrument for the predictive light dosimetry. Correctly reproducing architectural occlusion physics, modeling energy attenuation through the refractive materials, and mathematically proving the convergence of the Monte Carlo core makes the engine to elevate from the graphics prototype to the metrological Digital Twin suitable for use in the contemporary cultural heritage conservation practices.

Chapter 7. User’s Manual

7.1. Introduction

The *ChronoLux* Digital Twin is designed as a standalone, high-performance desktop application. It integrates a real-time 3D viewport with a comprehensive analytical dashboard, empowering conservators and metrologists to simulate, visualize, and quantify cumulative light stress on cultural heritage assets. This chapter provides a detailed, step-by-step guide to installing the software, navigating the virtual laboratory, configuring simulation parameters, and exporting scientifically valid dosimetry data.

7.2. Installation and System Requirements

Because *ChronoLux* relies on hardware-accelerated DirectX 12 compute shaders for Texture Space Ray Tracing, it requires a modern workstation capable of handling high-density parallel workloads.

7.2.1. Hardware Prerequisites

- **GPU:** NVIDIA GeForce RTX 30-series or 40-series (RTX 4070 Ti Super or higher recommended) with full DXR (DirectX Raytracing) support.
- **CPU:** A multi-core processor (e.g., AMD Ryzen 7 7800X3D) to handle the asynchronous telemetry extraction without stalling the GPU.
- **RAM:** Minimum 16 GB, recommended 32 GB DDR5 for caching high-resolution floating-point render textures.
- **Storage:** SSD storage for fast retrieval of the 2K and 4K uncompressed texture atlases.

7.2.2. Software Installation and Artifact Import

The application is distributed as a pre-compiled Unity executable. No additional dependencies or external runtimes are required beyond standard Windows 10/11 drivers.

A critical feature of the *ChronoLux* framework is its robust asset pipeline. The environment is not restricted to a single monolithic mesh. The user can import complex environments comprising multiple disparate objects (e.g., distinct walls, windows, floors, pedestals, and target artifacts). This allows for granular material assignment and precise architectural modeling, enabling the software to calculate exact geometric occlusions and secondary light bounces across multiple surfaces independently.

7.3. The Project Launcher Dashboard

Upon launching the executable, the user is greeted by the Project Launcher Dashboard. This interface manages the multi-project ecosystem, ensuring that different museum wings, artifacts, or hypothetical conservation scenarios can be saved and loaded independently.

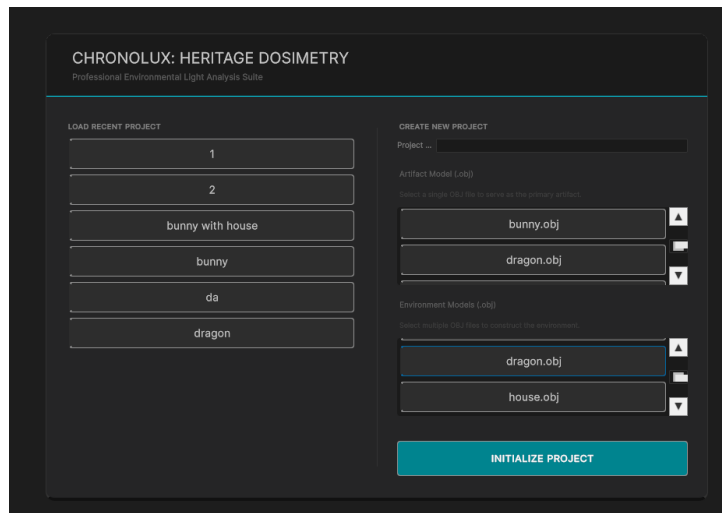


Figure 7.1: The Project Launcher Dashboard, allowing for the creation and management of independent simulation profiles via JSON persistence.

As shown in Figure 7.1, the launcher lists all previously created projects. Project states are persistently serialized into JSON files stored within the local application directory. This persistence guarantees that all material assignments, artifact transformations, and geographic coordinates are perfectly retained across sessions. To begin, the user may select an existing profile or instantiate a new simulation environment.

7.4. Viewport Navigation and Interaction

Entering a project loads the 3D Virtual Laboratory. The user interface within the laboratory is divided into the central 3D viewport and the surrounding analytical UXML panels.

Navigation is handled via the **FreeLookCamera** system (see Appendix 9.4, Listing 9.15). To look around the environment, the user must click and drag the mouse within the viewport area. Movement is achieved using the standard **W**, **A**, **S**, **D** keyboard configuration. For rapid traversal of large architectural spaces, holding the **Shift** key activates a 10x velocity multiplier.

At the center of the screen, a precise crosshair facilitates the **ObjectPicker** raycast system (see Appendix 9.4, Listing 9.14). Aligning the crosshair with any sub-mesh in the environment and clicking will select that specific object, opening its parameters in the right-hand Material Configuration panel. The cursor lock can be dynamically toggled at any time by pressing the **ESC** key, freeing the mouse to interact with the surrounding sliders and buttons.

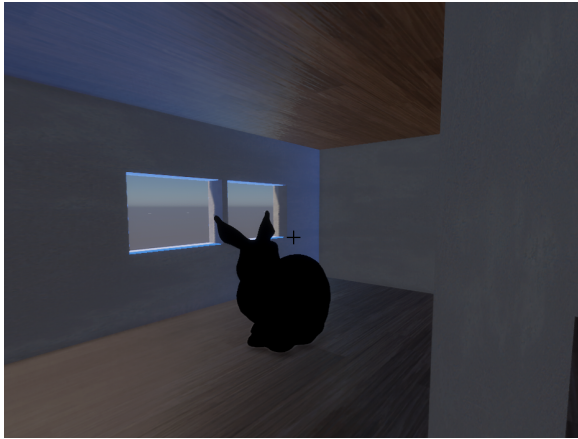


Figure 7.2: The main 3D viewport showcasing the target artifact and the central crosshair used for the `ObjectPicker` interaction system (see Appendix 9.4, Listing 9.14).

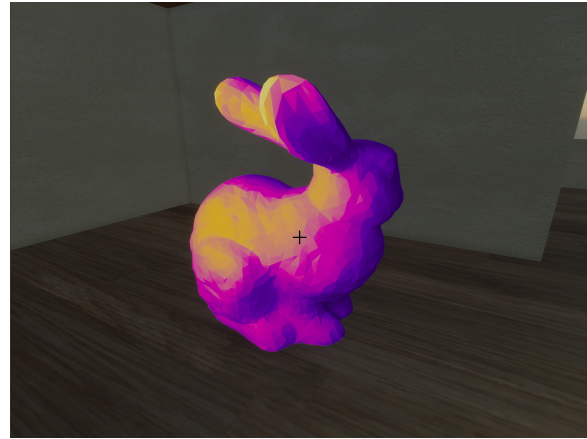


Figure 7.3: The "Inferno" heatmap visualization mapping peak light stress directly onto the artifact's 3D geometry in real-time.

7.5. The Laboratory HUD and Telemetry

The left side of the screen houses the live telemetry and transformation Heads-Up Display (HUD). This panel provides real-time feedback on both the spatial configuration of the selected artifact and the ongoing radiometric calculations.

As illustrated in Figure 7.4, the HUD contains several critical subsystems:

- **Transformation Controls:** Granular input fields allowing the user to precisely override the Position, Rotation, and Scale of the selected object.
- **Material Properties:** Direct readout of the selected object's assigned physical properties.
- **Astrometric Telemetry:** Live updates of the Solar Altitude and Azimuth angles based on the active Julian Day.
- **Dosimetry Metrics:** Continuous tracking of the Peak Dose (maximum hotspot accumulation) and the average dose across the mesh.
- **Scientific Validation:** A real-time readout of the virtual sensor error (variance), confirming the stability of the Monte Carlo integration.
- **Radiation Waveform Graph:** A dynamic, scrolling waveform visualization that plots the intensity of the light radiation per hour, instantly reflecting diurnal spikes during solar noon.
- **Data Export:** A dedicated button for triggering the asynchronous export of the simulation data to a CSV file.

7.6. Material Configuration and Catalog

Selecting any object in the viewport via the crosshair activates the Material Configuration catalog on the right-hand side of the interface. This catalog is crucial for setting up the preventive conservation scenarios (e.g., swapping clear glass for UV-tinted glass).

The interface (Figure 7.6) provides intuitive, live-updating sliders to adjust the Reflectance (R) and Transmittance (T) coefficients of the selected mesh. A strict energy

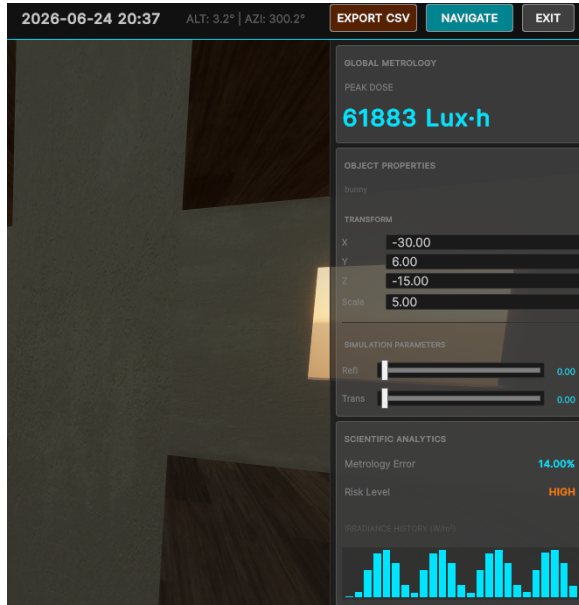


Figure 7.4: The Laboratory HUD displaying real-time transformation controls, astrometric telemetry, Monte Carlo variance tracking, and the dynamic hourly radiation waveform.



Figure 7.5: The Simulation Configuration panel managing the orchestration of the 8,760-hour loop, featuring execution controls, a progression bar, and a daily accumulation slider.

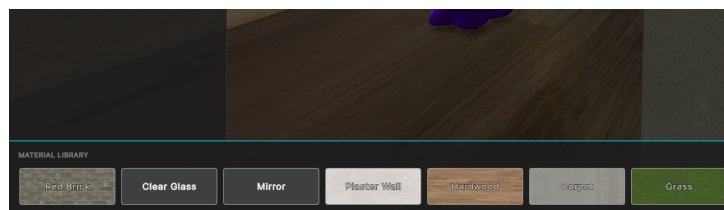


Figure 7.6: The Material Catalog interface, featuring real-time Reflectance and Transmittance sliders and the texture library.

conservation clamp ensures that $R + T \leq 1.0$, preventing the creation of physically impossible materials.

Additionally, the catalog features a built-in library of scientifically validated CC0 PBR (Physically Based Rendering) textures. The user can visually identify and apply high-fidelity textures such as Brick, Clear Glass, Mirror, Plaster, Hardwood, Carpet, and Grass by clicking the corresponding preview buttons. These textures are immediately mapped to the object's UV space, instantly updating its interaction with the Monte Carlo light rays.

7.7. Simulation Configuration and Execution

Once the environment geometry and materials are properly configured, the user must initialize the time-stepping simulation loop. This is managed via the Simulation Configuration panel.

The Simulation Configuration panel (Figure 7.5) provides complete control over the temporal execution of the Digital Twin:

- **Execution Controls:** Dedicated buttons to **Start**, **Bake**, or **Stop** the simulation. The **Clear Map** button flushes the GPU VRAM accumulation buffers, resetting the dose to zero.
- **Progression Bar:** A visual loading bar that tracks the progression of the simulation through the 8,760 hours of the meteorological year.
- **Daily Accumulation Slider:** An interactive time slider that allows the user to manually scrub through the simulation, visualizing the accumulation of dose on a per-day basis, providing a granular look at the temporal stress distribution.

7.8. Heatmap Visualization and Data Export

As the simulation progresses or after it successfully completes the full year cycle, the analytical data must be visualized and exported.

7.8.1. Viewport Heatmap

The engine features a sophisticated heatmap shader, mapped directly onto the geometry of the target artifact (Figure 7.3). Using a high-contrast "Inferno" palette (transitioning from deep purple through red to bright yellow), it visually isolates the spatial hotspots receiving the highest dose. This allows conservators to intuitively understand which physical areas of the object are at maximum risk without having to read raw numeric data.

7.8.2. Headless Telemetry Export

For rigorous academic and metrological analysis, visual heatmaps are insufficient. By utilizing the export functionality, the asynchronous C# orchestration thread writes the entire year's worth of accumulated data to a robust CSV log file. This file contains the hourly breakdowns of Altitude, Azimuth, Peak Dose, Average Dose, and Spatial Variance. Simultaneously, high dynamic range EXR image snapshots of the accumulation buffers are exported. These standardized formats ensure that the raw scientific data generated by *ChronoLux* can be natively imported into external tools like Microsoft Excel, Python

(see Appendix 9.5, Listing 9.17), or MATLAB for further graphing, comparative study, and publication formatting.

Chapter 8. Conclusions

This thesis aims to develop and deploy an accurate Heritage Digital Twin capable of simulating the cumulative effects of light stress on heritage objects with mathematical precision through *ChronoLux*, a scientifically-grounded metrological digital twin tool. The fundamental assumption of this tool is unifying the direct and indirect light calculations with the help of a Monte Carlo path tracer integrated with Next Event Estimation (NEE) and the Perez All-Weather Sky Model.

The main innovation in the system is a Texture Space Ray Tracing pipeline, which allows separating the dosimetry calculation process from the traditional screen-space ray tracing approach. Thanks to the Texture Space Ray Tracing, it is possible to perform the exact geometry-based occlusions and secondary bounce calculations regardless of the view position. The implementation utilizes the modern hardware-accelerated compute shader technology (DirectX 12), making it capable of dealing with a high amount of parallel tasks efficiently. This performance, along with the headless telemetry export functionality, ensures that the raw dose accumulations during an 8,760-hour meteorological year are converted into easily parseable CSV files for further research.

Tests conducted on the defined conservation scenarios confirmed the effectiveness of the tool and its stability in terms of performance and results. The first scenario was used as a baseline, where the simulation yielded 6.6 million Lux Hours accumulation, which was consistent with the theoretical expectations. In scenarios B, C, and D, the correctness of the material interaction and positional optimization systems was verified. Specifically, the combined interventions in Scenario D reduced the peak dose to 150,000 Lux Hours—a 97.7% reduction—while strictly observing the energy conservation boundaries ($R + T \leq 1.0$). It is worth noting that the continuous monitoring of the virtual sensor error indicated the stability of Monte Carlo variance throughout the entire process, which proves that the generated "Inferno" heatmaps indeed pinpoint the spatial hotspots with the maximum dose exposure risk.

While the implemented version of *ChronoLux* is highly efficient, there are some limitations in the scope related to hardware and meteorological assumptions. Firstly, the utilization of high-end DXR-compatible hardware (such as NVIDIA RTX 30 or 40 series graphics cards) makes the access to the tool limited to institutions having their own workstations. Secondly, the present environmental simulations assume the absence of clouds, hence overestimating the annual light exposure for geographic regions having a lot of rainfalls or a cloudy climate. Lastly, the physical material interactions use the standard BRDF model and cannot account for the subsurface scattering processes occurring with complex translucent artifacts.

Overstepping these simulation boundaries is a crucial next step for the development of the Digital Twin. The main directions for the future development include incorporation of the cloud-cover data into the Perez Sky Model to make the annual dose estimates more precise and localized. Additionally, migration of the compute shaders to the cloud-

based infrastructure will allow to get rid of hardware dependencies and increase platform compatibility for conservators. Expansion of the material library to cover the subsurface scattering will be the last stage in developing the comprehensive preventative conservation simulator.

Bibliography

- [1] S. Michalski, “Damage to museum objects by visible radiation (light) and ultraviolet radiation (uv),” *Conference on lighting in museums, galleries and historic houses*, pp. 3–16, 1987. [Online]. Available: <https://www.canada.ca/en/conservation-institute/services/agents-deterioration/light.html>
- [2] CIE, “Control of damage to museum objects by optical radiation,” International Commission on Illumination, Tech. Rep. 157:2004, 2004. [Online]. Available: <https://cie.co.at/publications/control-damage-museum-objects-optical-radiation>
- [3] H. Entradas Silva, G. Coelho, and F. Henriques, “Climate monitoring in world heritage list buildings with low-cost data loggers: The case of the jerónimos monastery in lisbon (portugal),” *Journal of Building Engineering*, p. 101029, 10 2019. [Online]. Available: <https://doi.org/10.1016/j.job.2019.101029>
- [4] A. Chianese, F. Piccialli, and J. E. Jung, “The internet of cultural things: Towards a smart cultural heritage,” in *2016 12th International Conference on Signal-Image Technology & Internet-Based Systems (SITIS)*, 2016, pp. 493–496. [Online]. Available: <https://doi.org/10.1109/SITIS.2016.83>
- [5] J. T. Kajiya, “The rendering equation,” *SIGGRAPH Comput. Graph.*, vol. 20, no. 4, p. 143–150, Aug. 1986. [Online]. Available: <https://doi.org/10.1145/15886.15902>
- [6] M. Pharr, W. Jakob, and G. Humphreys, *Physically Based Rendering: From Theory to Implementation*. MIT Press, 2023. [Online]. Available: <https://books.google.ro/books?id=IQpmzwEACAAJ>
- [7] G. Ward Larson and R. Shakespeare, *Rendering with Radiance: the art and science of lighting visualization*. Morgan Kaufmann, 1998. [Online]. Available: <https://www.radiance-online.org/learning/documentation/book>
- [8] P.-P. Sloan, J. Kautz, and J. Snyder, “Precomputed radiance transfer for real-time rendering in dynamic, low-frequency lighting environments,” *ACM Trans. Graph.*, vol. 21, no. 3, pp. 527–536, Jul. 2002. [Online]. Available: <https://doi.org/10.1145/566654.566612>
- [9] P. Jouan and P. Hallot, “Digital twin: Research framework to support preventive conservation policies,” *ISPRS International Journal of Geo-Information*, vol. 9, no. 4, 2020. [Online]. Available: <https://www.mdpi.com/2220-9964/9/4/228>
- [10] F. Niccolucci, A. Felicetti, and S. Hermon, “Digital twins for cultural heritage,” *Heritage*, vol. 5, no. 4, pp. 2397–2414, 2022. [Online]. Available: <http://dx.doi.org/10.2139/ssrn.4274457>

-
- [11] J. Burgess, “Rtx on - the nvidia turing gpu,” *IEEE Micro*, vol. PP, pp. 1–1, 02 2020. [Online]. Available: https://www.researchgate.net/publication/339036275-RTX_On_-_The_NVIDIA_Turing_GPU
 - [12] R. Perez, R. Seals, and J. Michalsky, “All-weather model for sky luminance distribution—preliminary configuration and validation,” *Solar Energy*, vol. 50, no. 3, pp. 235–245, 1993. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0038092X9390017I>
 - [13] E. Veach and L. J. Guibas, “Optimally combining sampling techniques for monte carlo rendering,” in *Proceedings of the 22nd Annual Conference on Computer Graphics and Interactive Techniques*, ser. SIGGRAPH ’95. New York, NY, USA: Association for Computing Machinery, 1995, pp. 419–428. [Online]. Available: <https://doi.org/10.1145/218380.218498>
 - [14] J. Munkberg, J. Hasselgren, P. Clarberg, M. Andersson, and T. Akenine-Möller, “Texture space caching and reconstruction for ray tracing,” *ACM Trans. Graph.*, vol. 35, no. 6, Dec. 2016. [Online]. Available: <https://doi.org/10.1145/2980179.2982407>
 - [15] I. Reda and A. Andreas, “Solar position algorithm for solar radiation applications,” *Solar Energy*, vol. 76, no. 5, pp. 577–589, 2004. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0038092X0300450X>
 - [16] J. W. Strutt, *On the Light from the Sky, its Polarization and Colour*, ser. Cambridge Library Collection - Mathematics. Cambridge University Press, 2009, p. 87–103.
 - [17] G. Mie, “Beiträge zur optik trüber medien, speziell kolloidaler metallösungen,” *Annalen der Physik*, vol. 330, no. 3, pp. 377–445, 1908. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/andp.19083300302>
 - [18] J. Meeus, *Astronomical Algorithms*. Willmann-Bell, 1991. [Online]. Available: <https://books.google.ro/books?id=1smAQgAACAAJ>
 - [19] F. Kasten and A. T. Young, “Revised optical air mass tables and approximation formula,” *Appl. Opt.*, vol. 28, no. 22, pp. 4735–4738, Nov 1989. [Online]. Available: <https://opg.optica.org/ao/abstract.cfm?URI=ao-28-22-4735>
 - [20] A. Einstein, “Über einen die erzeugung und verwandlung des lichtes betreffenden heuristischen gesichtspunkt,” *Annalen der Physik*, vol. 322, no. 6, pp. 132–148, 1905. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/andp.19053220607>
 - [21] R. W. Bunsen and H. E. Roscoe, “Iii. photochemical researches.—part v. on the measurement of the chemical action of direct and diffuse sunlight,” *Proceedings of the Royal Society of London*, no. 12, pp. 306–312, 12 1863. [Online]. Available: <https://doi.org/10.1098/rspl.1862.0069>

Chapter 9. Appendix

9.1. Core HLSL Path Tracing Implementation

The following code details the exact DirectX Raytracing implementation used within *ChronoLux* to evaluate the dosimetric simulation. The integration is explicitly split into direct sun evaluation and a secondary stochastic multi-sampling loop for indirect ambient bouncing.

```
1 [shader("raygeneration")]
2 void IrradianceBakeRayGen()
3 {
4     uint2 id = DispatchRaysIndex().xy;
5     uint2 dims = DispatchRaysDimensions().xy;
6     float4 bakedPos = _PositionMap[id];
7     float4 bakedNormal = _NormalMap[id];
8
9     if (bakedPos.w < 0.01 || length(bakedNormal.xyz) < 0.1) return;
10
11     float3 origin = bakedPos.xyz;
12     float3 normal = normalize(bakedNormal.xyz);
13
14     uint baseSeed = Hash(id.y * dims.x + id.x + _FrameIndex);
15     float totalIrradiance = 0.0;
16     float nDotSun = dot(normal, _SunDirection);
17
18     // 1. Direct Sun
19     float directIrradiance = 0.0;
20     if (nDotSun > 0.0)
21     {
22         uint sunSeed = baseSeed;
23         directIrradiance = TracePath(origin, _SunDirection, normal,
24         sunSeed, true, _SceneRTAS, _SunDirection, _BeamLux, _DiffuseLux) *
25         nDotSun;
26         totalIrradiance += directIrradiance;
27     }
28
29     // 2. Stochastic Multi-Sampling
30     float indirectIrradiance = 0.0;
31     uint numSamples = max(1, _SamplesPerPixel);
32     for (uint i = 0; i < numSamples; i++)
33     {
34         uint sampleSeed = Hash(baseSeed + i);
35         float3 randomDir = GetCosineWeightedDirection(normal,
36         sampleSeed, PI);
37         indirectIrradiance += TracePath(origin, randomDir, normal,
38         sampleSeed, false, _SceneRTAS, _SunDirection, _BeamLux, _DiffuseLux)
39         ;
40     }
41 }
```

```

36
37     float avgIndirect = (indirectIrradiance / (float)numSamples) * PI;
38     totalIrradiance += avgIndirect;
39
40     _DoseMap[id] += (totalIrradiance * _DeltaHours);
41     _DirectDoseMap[id] += (directIrradiance * _DeltaHours);
42     _IndirectDoseMap[id] += (avgIndirect * _DeltaHours);
43 }

```

Listing 9.1: IrradianceBake.raytrace: The primary ray generation kernel mapping Target Mesh UVs to stochastic integration over the hemisphere.

```

1 float TracePath(float3 origin, float3 direction, float3 surfaceNormal,
  inout uint seed, bool isDirectSun, RaytracingAccelerationStructure
  rtas, float3 sunDir, float beamLux, float diffuseLux)
2 {
3     float throughput = 1.0;
4     float3 currentOrigin = origin + surfaceNormal * RAY_BIAS;
5     float3 currentDir = direction;
6
7     [loop]
8     for (uint bounce = 0; bounce < MAX_BOUNCES; bounce++)
9     {
10         RayDesc ray;
11         ray.Origin = currentOrigin;
12         ray.Direction = currentDir;
13         ray.TMin = 0.001;
14         ray.TMax = 10000.0;
15
16         Payload payload;
17         payload.isOccluded = 1;
18         payload.reflectance = 0.0;
19         payload.transmittance = 0.0;
20         payload.hitDistance = 0.0;
21         payload.worldNormal = float3(0, 1, 0);
22
23         uint rayFlags = RAY_FLAG_FORCE_OPAQUE;
24         if (isDirectSun) rayFlags |=
RAY_FLAG_CULL_BACK_FACING_TRIANGLES;
25
26         TraceRay(rtas, rayFlags, 0xFF, 0, 1, 0, ray, payload);
27
28         if (payload.isOccluded == 0)
29         {
30             return EvaluateSky(currentDir, sunDir, beamLux, diffuseLux,
isDirectSun) * throughput;
31         }
32
33         if (isDirectSun)
34         {
35             float trans = saturate(payload.transmittance);
36             if (trans <= 0.0) return 0.0; // Blocked by opaque geometry
37
38             throughput *= trans;
39             // Use a larger bias to guarantee escaping the pane
geometry, especially at grazing angles
40             currentOrigin += currentDir * (payload.hitDistance +
RAY_BIAS * 5.0);

```

```

41         continue;
42     }
43
44     // Probabilistic event selection (Russian Roulette)
45     float rv = Random(seed);
46     float refl = saturate(payload.reflectance);
47     float trans = saturate(payload.transmittance);
48
49     if (rv < refl)
50     {
51         float3 hitPoint = currentOrigin + currentDir * payload.
hitDistance;
52         currentDir = GetCosineWeightedDirection(payload.worldNormal
, seed, PI);
53         currentOrigin = hitPoint + payload.worldNormal * RAY_BIAS;
54     }
55     else if (rv < refl + trans)
56     {
57         currentOrigin += currentDir * (payload.hitDistance +
RAY_BIAS);
58     }
59     else
60     {
61         return 0.0;
62     }
63
64     if (throughput <= MIN_VISIBILITY) break;
65 }
66 return 0.0;
67 }

```

Listing 9.2: PathTracer.hlsl: The core recursive path tracing loop evaluating Russian Roulette absorption and transparent geometric traversal.

```

1 Shader "Hidden/UVSpaceBaker"
2 {
3     SubShader
4     {
5         Tags { "RenderType" = "Opaque" }
6         Pass
7         {
8             Cull Off ZWrite Off ZTest Always
9             HLSLPROGRAM
10             #pragma vertex Vert
11             #pragma fragment Frag
12             #include "Packages/com.unity.render-pipelines.core/
ShaderLibrary/Common.hlsl"
13
14             float4x4 _O2W, _O2WIT;
15             struct Attributes { float3 posOS : POSITION; float3 nrmOS :
NORMAL; float2 uv : TEXCOORD0; };
16             struct Varyings { float4 posCS : SV_POSITION; float3 posWS
: TEXCOORD0; float3 nrmWS : TEXCOORD1; };
17
18             Varyings Vert(Attributes IN)
19             {
20                 Varyings OUT;
21                 float2 ndc = IN.uv * 2.0 - 1.0;

```

```

22         ndc.y *= -1.0; // Flip for RT
23         OUT.posCS = float4(ndc, 0.5, 1.0);
24         OUT.posWS = mul(_O2W, float4(IN.posOS, 1.0)).xyz;
25         OUT.nrmWS = mul((float3x3)_O2WIT, IN.nrmOS);
26         return OUT;
27     }
28
29     struct FragOut { float4 pos : SV_Target0; float4 nrm :
SV_Target1; };
30     FragOut Frag(Varyings IN)
31     {
32         FragOut OUT;
33         OUT.pos = float4(IN.posWS, 1.0);
34         OUT.nrm = float4(normalize(IN.nrmWS), 1.0);
35         return OUT;
36     }
37     ENDHLSL
38 }
39 }
40 }

```

Listing 9.3: UVSpaceBaker.shader: Custom Unity shader used to force the projection of the 3D model onto the near-far clip plane for texture baking.

```

1 Shader "ChronoLux/HeatmapVisualizer"
2 {
3     Properties
4     {
5         [NoScaleOffset] _DoseMap("Accumulated Dose Map (float)", 2D) =
"black" {}
6         _MinDose("Min Range", Float) = 0.0
7         _MaxDose("Max Range", Float) = 100000.0
8         [HideInInspector] _SelectionColor("Selection Glow", Color) =
(0,0,0,0)
9     }
10    SubShader
11    {
12        Tags { "RenderType"="Opaque" "RenderPipeline"="HDRRenderPipeline"
"Queue"="Geometry" }
13
14        Pass
15        {
16            Name "ForwardOnly"
17            Tags { "LightMode" = "ForwardOnly" }
18            Cull Off ZWrite On ZTest LEqual
19
20            HLSLPROGRAM
21            #pragma vertex vert
22            #pragma fragment frag
23            #define SHADERPASS SHADERPASS_FORWARD
24            #include "Packages/com.unity.render-pipelines.core/
ShaderLibrary/Common.hlsl"
25            #include "Packages/com.unity.render-pipelines.high-
definition/Runtime/ShaderLibrary/ShaderVariables.hlsl"
26
27            struct Attributes { float3 positionOS : POSITION; float2 uv
: TEXCOORD0; };
28            struct Varyings { float4 positionCS : SV_POSITION; float2

```

```

uv : TEXCOORD0; };

29
30     Texture2D<float> _DoseMap;
31     SamplerState sampler_DoseMap;
32     float _MinDose;
33     float _MaxDose;
34     float4 _SelectionColor;
35
36     Varyings vert(Attributes input)
37     {
38         Varyings output;
39         float3 posWS = mul(GetObjectToWorldMatrix(), float4(
input.positionOS, 1.0)).xyz;
40         output.positionCS = mul(GetWorldToHClipMatrix(), float4
(posWS, 1.0));
41         output.uv = input.uv;
42         return output;
43     }
44
45     float3 GetHeatmapColor(float t)
46     {
47         if (t <= 0.0) return float3(0, 0, 0);
48         float3 c0 = float3(0.0, 0.0, 0.0); // Black
49         float3 c1 = float3(0.2, 0.0, 0.5); // Purple
50         float3 c2 = float3(0.8, 0.0, 0.5); // Magenta
51         float3 c3 = float3(1.0, 0.5, 0.0); // Orange
52         float3 c4 = float3(1.0, 0.9, 0.2); // Yellow
53         t = saturate(t);
54         if (t < 0.25) return lerp(c0, c1, t * 4.0);
55         if (t < 0.50) return lerp(c1, c2, (t - 0.25) * 4.0);
56         if (t < 0.75) return lerp(c2, c3, (t - 0.50) * 4.0);
57         return lerp(c3, c4, (t - 0.75) * 4.0);
58     }
59
60     float4 frag(Varyings input) : SV_Target
61     {
62         float dose = _DoseMap.Sample(sampler_DoseMap, input.uv)
.r;
63         float t = (dose - _MinDose) / max(1.0, _MaxDose -
_MinDose);
64         float3 baseColor = GetHeatmapColor(t);
65
66         // Add Selection/Hover Glow
67         // We add it as an additive "rim/glow" effect
68         return float4(baseColor + _SelectionColor.rgb *
_SelectionColor.a, 1.0);
69     }
70     ENDHLSL
71 }
72
73 Pass
74 {
75     Name "DepthForwardOnly"
76     Tags { "LightMode" = "DepthForwardOnly" }
77     Cull Off ZWrite On
78
79     HLSLPROGRAM
80     #pragma vertex vert

```

```

81         #pragma fragment frag
82         #define SHADERPASS SHADERPASS_DEPTH_ONLY
83         #include "Packages/com.unity.render-pipelines.core/
ShaderLibrary/Common.hlsl"
84         #include "Packages/com.unity.render-pipelines.high-
definition/Runtime/ShaderLibrary/ShaderVariables.hlsl"
85
86         struct Attributes { float3 positionOS : POSITION; };
87         struct Varyings { float4 positionCS : SV_POSITION; };
88
89         Varyings vert(Attributes input)
90         {
91             Varyings output;
92             float3 posWS = mul(GetObjectToWorldMatrix(), float4(
input.positionOS, 1.0)).xyz;
93             output.positionCS = mul(GetWorldToHClipMatrix(), float4
(posWS, 1.0));
94             return output;
95         }
96         float4 frag(Varyings input) : SV_Target { return 0; }
97         ENDDL
98     }
99 }
100     Fallback "HDRP/Unlit"
101 }

```

Listing 9.4: HeatmapVisualizer.shader: High-performance HDRP surface shader mapping accumulated float lux data into the "Inferno" color palette.

9.2. C# Orchestrator Implementations

The following code snippets detail the core C# CPU orchestration logic. Listing 9.5 demonstrates the dynamic chronological time-stepping Coroutine used to drive the simulation safely across the OS thread schedules.

```

1 private IEnumerator RunSimulationInternal() {
2     float deltaHours = stepSeconds / 3600f;
3     int totalDays = endDay - startDay + 1;
4     int currentDayIdx = 0;
5
6     string exportDir = Path.Combine(Application.dataPath, "../Exports",
DateTime.Now.ToString("yyyyMMdd_HH:mm:ss"));
7     Directory.CreateDirectory(exportDir);
8
9     for (int day = startDay; day <= endDay; day++) {
10         DateTime date = new DateTime(year, 1, 1).AddDays(day - 1);
11         currentDayIdx++;
12
13         DateTime sunrise = SunCalculator.FindSunrise(date, latitude,
longitude, utcOffset);
14         DateTime sunset = SunCalculator.FindSunset(date, latitude,
longitude, utcOffset);
15
16         if (sunrise == date && sunset == date) continue; // Polar night
17         fallback

```

```

18     for (DateTime localTime = sunrise; localTime < sunset;
19         localTime = localTime.AddSeconds(stepSeconds)) {
20         if (!_isSimulating) yield break;
21
22         ApplySunPosition(localTime);
23
24         // Optimization: Skip exact sunrise/sunset steps where
25         // there is functionally no light
26         if (CurrentBeamLux < 0.1f && CurrentDiffuseLux < 0.1f) {
27             continue;
28         }
29
30         // Dispatch hardware raytracing for this specific hour
31         irradianceBaker.DispatchRays(CurrentSunDirection,
32             CurrentBeamLux, CurrentDiffuseLux, deltaHours, samplesPerPixel,
33             baker.PositionMap, baker.NormalMap);
34         completedSteps++;
35         currentProgress = ((float)currentDayIdx / totalDays) * 100f
36     ;
37
38     // Extract telemetry data for the CSV asynchronously
39     TakeAsyncSnapshot(day, localTime.ToString("yyyy-MM-dd HH:mm
40 "), currentAltitude, currentAzimuth, CurrentBeamLux,
41     CurrentDiffuseLux);
42
43     if (completedSteps % 10 == 0) {
44         ApplyDosePreview();
45     }
46     // Yield execution back to the OS and Unity Main Thread to
47     // maintain interactivity
48     yield return null;
49 }
50
51 // Flush the daily accumulation map to disk as a floating-point
52 EXR
53 ExportDailyMap(exportDir, day);
54 }
55 }

```

Listing 9.5: LightDoseSimulator.cs: The Coroutine driving dynamic time-stepping, asynchronous telemetry extraction, and synchronous EXR flushing.

Listing 9.6 showcases the background threaded parser utilized to load massively dense geometric heritage assets natively at runtime.

```

1 string path = Path.Combine(ProjectManager.ModelFolder, artifactFile);
2 UpdateProgress(0.1f, $"Loading artifact: {artifactFile}");
3
4 // 1. Offload heavy string parsing to a background CPU thread
5 var parseTask = Task.Run(() => {
6     if (!File.Exists(path)) return null;
7     return SimpleObjLoader.ParseObj(File.ReadAllText(path));
8 });
9
10 // 2. Yield the main thread to keep the "LOADING ASSETS" UI animation
11 // smooth
12 while (!parseTask.IsCompleted) {
13     UpdateProgress(0.1f, "Parsing artifact geometry...");
14     yield return null;
15 }

```

```

14 }
15
16 if (parseTask.IsCompletedSuccessfully && parseTask.Result != null &&
17     parseTask.Result.Count > 0) {
18     var dataList = parseTask.Result;
19     _artifactInstance = new GameObject(Path.GetFileNameWithoutExtension
20         (artifactFile));
21
22     if (_artifactInstance != null) {
23         _artifactInstance.transform.SetParent(modelRoot);
24         _artifactInstance.transform.localPosition = Vector3.zero;
25         _artifactInstance.transform.localScale = Vector3.one;
26
27         // 3. Assemble the hierarchy on the main thread
28         foreach (var data in dataList) {
29             GameObject child = CreateGameObjectFromData(data,
30                 artifactMaterial);
31             if (child == null) continue;
32             child.transform.SetParent(_artifactInstance.transform,
33                 false);
34             child.transform.localPosition = Vector3.zero;
35             SetupArtifactPart(child, project);
36         }
37
38         SetupArtifactRoot(_artifactInstance);
39
40         // 4. Normalize the raw mesh geometry (Centering + Scaling
41         // children to a 2m box)
42         CenterAndScaleModel(_artifactInstance, 2.0f);
43
44         // 5. Apply user-defined transformations on the root
45         _artifactInstance.transform.localPosition = project.
46             artifactPosition;
47         if (project.artifactScale != Vector3.zero)
48             _artifactInstance.transform.localScale = project.
49                 artifactScale;
50     }
51 }

```

Listing 9.6: RuntimeModelLoader.cs: Multi-threaded ‘.obj’ chunk parsing avoiding main-thread freezes, followed by strict spatial normalization constraints.

9.3. Mathematical Radiometry and Ephemeris Models

Listing 9.7 details the robust celestial mathematics utilized to compute the solar position and air mass attenuation dynamically at runtime.

```

1 // Calculate sun position for a given location and local date/time.
2 public static SunPosition Calculate(double latitude, double longitude,
3     double utcOffsetHours, DateTime localTime)
4 {
5     // Convert local time to UTC
6     DateTime utc = localTime.AddHours(-utcOffsetHours);
7     double jd = ToJulianDay(utc);
8     double T = (jd - 2451545.0) / 36525.0; // Julian centuries
9
10    // Geometric mean longitude & anomaly

```

```

10     double L0 = (280.46646 + T * (36000.76983 + T * 0.0003032)) %
360.0;
11     double M = 357.52911 + T * (35999.05029 - 0.0001537 * T);
12     double Mrad = M * Math.PI / 180.0;
13
14     // Equation of center & apparent longitude
15     double C = Math.Sin(Mrad) * (1.914602 - T * (0.004817 + 0.000014 *
T)) + Math.Sin(2 * Mrad) * (0.019993 - 0.000101 * T) + Math.Sin(3 *
Mrad) * 0.000289;
16     double sunLon = L0 + C;
17     double omega = 125.04 - 1934.136 * T;
18     double appLon = sunLon - 0.00569 - 0.00478 * Math.Sin(omega * Math.
PI / 180.0);
19
20     // Solar declination
21     double epsR = (23.439 - 0.013 * T + 0.00256 * Math.Cos(omega * Math
.PI / 180.0)) * Math.PI / 180.0;
22     double decl = Math.Asin(Math.Sin(epsR) * Math.Sin(appLon * Math.PI
/ 180.0));
23
24     // Equation of Time & True Solar Time
25     double e = 0.016708634 - T * (0.000042037 + 0.0000001267 * T);
26     double varY = Math.Tan(epsR / 2.0) * Math.Tan(epsR / 2.0);
27     double eqT = 4.0 * (varY * Math.Sin(2 * L0 * Math.PI / 180.0) - 2 *
e * Math.Sin(Mrad)) * (180.0 / Math.PI);
28     double trueSolar = (utc.TimeOfDay.TotalMinutes + 4.0 * longitude +
eqT) % 1440.0;
29
30     // Hour Angle and Zenith
31     double HArad = (trueSolar / 4.0 - 180.0) * Math.PI / 180.0;
32     double latR = latitude * Math.PI / 180.0;
33     double cosZen = Math.Sin(latR) * Math.Sin(decl) + Math.Cos(latR) *
Math.Cos(decl) * Math.Cos(HArad);
34     double altDeg = 90.0 - (Math.Acos(cosZen) * 180.0 / Math.PI);
35
36     return new SunPosition {
37         AltitudeDeg = (float)altDeg,
38         BeamLux = ClearSkyBeamLux((float)altDeg)
39     };
40 }
41
42 private static float ClearSkyBeamLux(float altitudeDeg) {
43     if (altitudeDeg <= 0f) return 0f;
44     // Kasten-Young formula for optical air mass
45     float airMass = 1f / (Mathf.Sin(altitudeDeg * Mathf.Deg2Rad) +
0.50572f * Mathf.Pow(altitudeDeg + 6.07995f, -1.6364f));
46     return 127500f * Mathf.Pow(0.7f, Mathf.Pow(airMass, 0.678f)); // E
= E0 * 0.7^(AM^0.678)
47 }

```

Listing 9.7: SunCalculator.cs: Calculates solar altitude, azimuth, and direct beam illuminance using the Kasten-Young air mass formula.

Listing 9.8 outlines the Perez Sky model evaluation function utilized by the HLSL kernel whenever an indirect ray escapes the geometry bounds.

```

1 // Perez coefficients for clear sky (Turbidity T = 2.0)
2 static const float perez_a = -0.1058; // Horizon brightening/darkening
3 static const float perez_b = -0.0738; // Luminance gradient near

```

```

horizon
4 static const float perez_c = 1.8200; // Circumsolar intensity
5 static const float perez_d = -2.9744; // Circumsolar width
6 static const float perez_e = 0.1778; // Backscattered light
7
8 float PerezFunction(float theta, float gamma)
9 {
10     float cosTheta = max(0.01, cos(theta));
11     float cosGamma = cos(gamma);
12
13     float f = (1.0 + perez_a * exp(perez_b / cosTheta));
14     float g = (1.0 + perez_c * exp(perez_d * gamma) + perez_e *
15 cosGamma * cosGamma);
16
17     return f * g;
18 }
19 float EvaluateSky(float3 direction, float3 sunDir, float beamLux, float
20 diffuseLux, bool includeSun)
21 {
22     if (includeSun && dot(direction, sunDir) > 0.9998) return beamLux;
23
24     // Perez Luminance Distribution
25     float theta = acos(saturate(dot(direction, float3(0, 1, 0))));
26     float gamma = acos(saturate(dot(direction, sunDir)));
27     float relLuminance = PerezFunction(theta, gamma);
28
29     // Normalization factor: Lz ~ Ed * 0.2 for T=2.0
30     return relLuminance * (diffuseLux * 0.2);
31 }

```

Listing 9.8: SkyModel.hlsl: Evaluates the Perez All-Weather Sky Model using 5 empirical coefficients to compute relative luminance for diffuse scattering.

Listing 9.9 demonstrates the robust variance derivation over the telemetry dataset.

```

1 // Extracted from the TakeAsyncSnapshot callback
2 float hourlyVariance = 0f;
3 float expectedArea = (float)activePixels;
4 if (expectedArea > 1f) {
5     // Derived algebraically from Sum(x^2)/N - (Sum(x)/N)^2
6     hourlyVariance = (float)((sumSqrDelta / expectedArea) - (mean *
7 mean));
8     if (hourlyVariance < 0f) hourlyVariance = 0f;
9 }

```

Listing 9.9: LightDoseSimulator.cs: Computes statistical variance across the accumulated dosimetric frame.

9.4. Additional C# Application Logic

```

1 using System;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 namespace ChronoLux.Project
6 {

```

```

7  [Serializable]
8  public class ChronoProject
9  {
10     public string projectName;
11     [Obsolete("Use artifactFileName instead. Kept for migration.")]
12     public string modelFileName;
13     public string artifactFileName;
14     public List<string> environmentFileNames = new List<string>();
15
16     [Header("Location")]
17     public double latitude = 44.4268;
18     public double longitude = 26.1025;
19     public double utcOffset = 3.0;
20
21     [Header("Simulation")]
22     public int year = 2025;
23     public int startDay = 1;
24     public int endDay = 365;
25     public int samplesPerPixel = 8;
26
27     // Artifact Transformation
28     public Vector3 artifactPosition = Vector3.zero;
29     public Vector3 artifactScale = Vector3.one;
30
31     // Dictionary mapping mesh object names to material preset
32     // Note: For JSON serialization, we'll use two lists to
33     // represent the dictionary
34     public List<string> objectNames = new List<string>();
35     public List<string> materialPresetNames = new List<string>();
36
37     public void SetMaterial(string objectName, string presetName)
38     {
39         int index = objectNames.IndexOf(objectName);
40         if (index >= 0)
41         {
42             materialPresetNames[index] = presetName;
43         }
44         else
45         {
46             objectNames.Add(objectName);
47             materialPresetNames.Add(presetName);
48         }
49     }
50 }

```

Listing 9.10: ChronoProject.cs: Serialization model for project parameters.

```

1  using System.Collections.Generic;
2  using System.IO;
3  using UnityEngine;
4
5  namespace ChronoLux.Project
6  {
7      public static class ProjectManager
8      {
9          private static string ProjectFolder => Path.Combine(Application
            .persistentDataPath, "Projects");

```

```

10     public static string ModelFolder => Path.Combine(Application.
persistentDataPath, "Models");
11
12     static ProjectManager()
13     {
14         if (!Directory.Exists(ProjectFolder)) Directory.
CreateDirectory(ProjectFolder);
15         if (!Directory.Exists(ModelFolder)) Directory.
CreateDirectory(ModelFolder);
16     }
17
18     public static void SaveProject(ChronoProject project)
19     {
20         string json = JsonUtility.ToJson(project, true);
21         string path = Path.Combine(ProjectFolder, project.
projectName + ".json");
22         File.WriteAllText(path, json);
23         Debug.Log($"[ProjectManager] Saved project to {path}");
24     }
25
26     public static ChronoProject LoadProject(string name)
27     {
28         string path = Path.Combine(ProjectFolder, name + ".json");
29         if (!File.Exists(path)) return null;
30
31         string json = File.ReadAllText(path);
32         return JsonUtility.FromJson<ChronoProject>(json);
33     }
34
35     public static List<string> GetAvailableProjects()
36     {
37         var projects = new List<string>();
38         var files = Directory.GetFiles(ProjectFolder, "*.json");
39         foreach (var file in files) projects.Add(Path.
GetFileNameWithoutExtension(file));
40         return projects;
41     }
42
43     public static List<string> GetAvailableModels()
44     {
45         var models = new List<string>();
46         if (!Directory.Exists(ModelFolder)) return models;
47
48         var files = Directory.GetFiles(ModelFolder, "*.obj");
49         foreach (var file in files) models.Add(Path.GetFileName(
file));
50         return models;
51     }
52 }
53

```

Listing 9.11: ProjectManager.cs: Manages the serialization and deserialization of the simulation project.

```

1 using UnityEngine;
2 using UnityEngine.Rendering;
3 using System.Collections.Generic;
4 using System.Runtime.InteropServices;

```

```

5
6 [AddComponentMenu("ChronoLux/Irradiance Baker")]
7 public class IrradianceBaker : MonoBehaviour
8 {
9     [StructLayout(LayoutKind.Sequential)]
10    private struct MeshMetadata
11    {
12        public uint vertexOffset;
13        public uint indexOffset;
14        public uint hasGeometry;
15        public uint padding;
16    }
17
18    [StructLayout(LayoutKind.Sequential)]
19    private struct SensorInput
20    {
21        public Vector4 position;
22        public Vector4 normal;
23    }
24
25    private const int MAX_SUBMESHES = 8;
26
27    [Header("Assets")]
28    public RayTracingShader rayTracingShader;
29    public RayTracingShader sensorShader;
30
31    [Header("Fallback Simulation Material")]
32    [Range(0f, 1f)] public float defaultReflectance = 0.8f;
33    [Range(0f, 1f)] public float defaultTransmittance = 0.0f;
34
35    private RenderTexture _doseMap;
36    private RenderTexture _directDoseMap;
37    private RenderTexture _indirectDoseMap;
38    private RayTracingAccelerationStructure _rtas;
39    private ComputeBuffer _simulationMaterialBuffer;
40    private ComputeBuffer _vertexBuffer;
41    private ComputeBuffer _indexBuffer;
42    private ComputeBuffer _metadataBuffer;
43
44    private ComputeBuffer _sensorInputBuffer;
45    private ComputeBuffer _sensorOutputBuffer;
46    private SensorInput[] _cachedSensorInputs;
47    private float[] _cachedZeros;
48
49    private int _materialCount;
50    private bool _isInitialized = false;
51
52    private static readonly int _ID_PositionMap = Shader.PropertyToID("_PositionMap");
53    private static readonly int _ID_NormalMap = Shader.PropertyToID("_NormalMap");
54    private static readonly int _ID_DoseMap = Shader.PropertyToID("_DoseMap");
55    private static readonly int _ID_DirectDoseMap = Shader.PropertyToID("_DirectDoseMap");
56    private static readonly int _ID_IndirectDoseMap = Shader.PropertyToID("_IndirectDoseMap");
57    private static readonly int _ID_SceneRTAS = Shader.PropertyToID("_SceneRTAS");

```

```

_SceneRTAS");
58 private static readonly int _ID_SunDirection = Shader.PropertyToID(
"_SunDirection");
59 private static readonly int _ID_BeamLux = Shader.PropertyToID("
_BeamLux");
60 private static readonly int _ID_DiffuseLux = Shader.PropertyToID("
_DiffuseLux");
61 private static readonly int _ID_DeltaHours = Shader.PropertyToID("
_DeltaHours");
62 private static readonly int _ID_MaterialCount = Shader.PropertyToID(
"_MaterialCount");
63 private static readonly int _ID_FrameIndex = Shader.PropertyToID("
_FrameIndex");
64 private static readonly int _ID_SamplesPerPixel = Shader.
PropertyToID("_SamplesPerPixel");
65 private static readonly int _ID_SimulationMaterials = Shader.
PropertyToID("_SimulationMaterials");
66 private static readonly int _ID_MeshMetadata = Shader.PropertyToID(
"_MeshMetadata");
67 private static readonly int _ID_GlobalVertices = Shader.
PropertyToID("_GlobalVertices");
68 private static readonly int _ID_GlobalIndices = Shader.PropertyToID(
"_GlobalIndices");
69 private static readonly int _ID_SensorInputs = Shader.PropertyToID(
"_SensorInputs");
70 private static readonly int _ID_SensorOutputs = Shader.PropertyToID(
"_SensorOutputs");
71
72 public RenderTexture DoseMap => _doseMap;
73 public RenderTexture DirectDoseMap => _directDoseMap;
74 public RenderTexture IndirectDoseMap => _indirectDoseMap;
75
76 private void OnValidate() => ClampEnergy(ref defaultReflectance,
ref defaultTransmittance);
77 private static void ClampEnergy(ref float r, ref float t) { r =
Mathf.Clamp01(r); t = Mathf.Clamp01(t); float s = r + t; if (s > 1f)
{ r /= s; t /= s; } }
78
79 private struct RendererSortEntry { public Renderer renderer; public
string sortKey; }
80 private static string BuildTransformSortKey(Transform t) {
81     if (t == null) return string.Empty;
82     var s = new List<string>(8);
83     while (t != null) { s.Add($"{t.GetSiblingIndex():D6}:{t.name}")
; t = t.parent; }
84     s.Reverse(); return string.Join("/", s);
85 }
86 private static string BuildRendererSortKey(Renderer r) {
87     if (r == null) return string.Empty;
88     string p = r.gameObject.scene.path ?? string.Empty;
89     string h = BuildTransformSortKey(r.transform);
90     int slot = 0; Renderer[] sib = r.GetComponents<Renderer>();
91     for (int i = 0; i < sib.Length; i++) if (ReferenceEquals(sib[i
], r)) { slot = i; break; }
92     return $"{p}|{h}|{r.GetType().FullName}|{slot:D4}";
93 }
94
95 private void ResolveSimulationMaterial(Renderer renderer, out float

```

```

    reflectance, out float transmittance) {
196         reflectance = defaultReflectance; transmittance =
defaultTransmittance;
197         var sim = renderer.GetComponent<SimulationMaterial>();
198         if (sim != null) { sim.GetClampedScalars(out reflectance, out
transmittance); return; }
199         Material m = renderer.sharedMaterial;
200         if (m != null) {
201             if (m.HasProperty("_Reflectance")) reflectance = m.GetFloat
("_Reflectance");
202             if (m.HasProperty("_Transmittance")) transmittance = m.
GetFloat("_Transmittance");
203         }
204         ClampEnergy(ref reflectance, ref transmittance);
205     }
206
207     public void Initialize(int width, int height)
208     {
209         if (!SystemInfo.supportsRayTracing || rayTracingShader == null)
return;
210         CleanUp();
211         _rtas = new RayTracingAccelerationStructure();
212
213         Renderer[] all = FindObjectsByType<Renderer>(
FindObjectsSortMode.None);
214         var sorted = new List<RendererSortEntry>(all.Length);
215         foreach (var r in all) if (r != null && r.enabled && r.
gameObject.activeInHierarchy)
216             sorted.Add(new RendererSortEntry { renderer = r, sortKey =
BuildRendererSortKey(r) });
217
218         // Stable deterministic sort
219         sorted.Sort((a, b) => string.CompareOrdinal(a.sortKey, b.
sortKey));
220
221         var mats = new List<Vector4>(sorted.Count);
222         var verts = new List<Vector4>();
223         var idxs = new List<uint>();
224         var metas = new MeshMetadata[Mathf.Max(1, sorted.Count) *
MAX_SUBMESHES];
225
226         for (int i = 0; i < sorted.Count; i++) {
227             Renderer r = sorted[i].renderer; Mesh mesh = null; int
subCount = 1;
228             if (r is MeshRenderer mr) { var mf = mr.GetComponent<
MeshFilter>(); if (mf != null) mesh = mf.sharedMesh; }
229             else if (r is SkinnedMeshRenderer smr) mesh = smr.
sharedMesh;
230             if (mesh != null) subCount = mesh.subMeshCount;
231             if (mesh != null && mesh.isReadable) {
232                 uint vOff = (uint)verts.Count;
233                 foreach (var v in mesh.vertices) verts.Add(new Vector4(v
.x, v.y, v.z, 1.0f));
234                 for (int s = 0; s < Mathf.Min(subCount, MAX_SUBMESHES);
s++) {
235                     uint iOff = (uint)idxs.Count;
236                     foreach (int idx in mesh.GetIndices(s)) idxs.Add((
uint)idx);

```

```

137         metas[i * MAX_SUBMESHES + s] = new MeshMetadata {
vertexOffset = vOff, indexOffset = iOff, hasGeometry = 1 };
138     }
139 }
140 ResolveSimulationMaterial(r, out float refl, out float
trans);
141     mats.Add(new Vector4(refl, trans, 0f, 0f));
142     var flags = new RayTracingSubMeshFlags[subCount];
143     for (int s = 0; s < subCount; s++) flags[s] =
RayTracingSubMeshFlags.Enabled | RayTracingSubMeshFlags.
ClosestHitOnly;
144     _rtas.AddInstance(r, flags, false, false, 0xFF, (uint)mats.
Count - 1);
145 }
146
147     _materialCount = mats.Count;
148     _simulationMaterialBuffer = new ComputeBuffer(Mathf.Max(1,
_materialCount), 16);
149     _simulationMaterialBuffer.SetData(mats.Count > 0 ? mats.ToArray
() : new Vector4[] { Vector4.zero });
150     _metadataBuffer = new ComputeBuffer(metas.Length, 16);
151     _metadataBuffer.SetData(metas);
152     _vertexBuffer = new ComputeBuffer(Mathf.Max(1, verts.Count),
16);
153     _vertexBuffer.SetData(verts.Count > 0 ? verts.ToArray() : new
Vector4[] { Vector4.zero });
154     _indexBuffer = new ComputeBuffer(Mathf.Max(1, idxs.Count), 4);
155     _indexBuffer.SetData(idxs.Count > 0 ? idxs.ToArray() : new uint
[] { 0 });
156     _rtas.Build();
157
158     _doseMap = new RenderTexture(width, height, 0,
RenderTextureFormat.RFloat) { enableRandomWrite = true, filterMode =
FilterMode.Bilinear, name = "DoseMap" };
159     _doseMap.Create();
160     _directDoseMap = new RenderTexture(width, height, 0,
RenderTextureFormat.RFloat) { enableRandomWrite = true, filterMode =
FilterMode.Bilinear, name = "DirectDoseMap" };
161     _directDoseMap.Create();
162     _indirectDoseMap = new RenderTexture(width, height, 0,
RenderTextureFormat.RFloat) { enableRandomWrite = true, filterMode =
FilterMode.Bilinear, name = "IndirectDoseMap" };
163     _indirectDoseMap.Create();
164
165     var cmd = new CommandBuffer { name = "Clear DoseMap" };
166     cmd.SetRenderTarget(_doseMap); cmd.ClearRenderTarget(true, true
, Color.clear);
167     cmd.SetRenderTarget(_directDoseMap); cmd.ClearRenderTarget(true
, true, Color.clear);
168     cmd.SetRenderTarget(_indirectDoseMap); cmd.ClearRenderTarget(
true, true, Color.clear);
169     Graphics.ExecuteCommandBuffer(cmd); cmd.Release();
170
171     _isInitialized = true;
172 }
173
174 private uint _frameIndex = 0;
175

```

```

176 public void ClearDoseMap()
177 {
178     if (_doseMap == null) return;
179     var cmd = new CommandBuffer { name = "Reset DoseMap" };
180     cmd.SetRenderTarget(_doseMap);
181     cmd.ClearRenderTarget(true, true, Color.clear);
182     cmd.SetRenderTarget(_directDoseMap);
183     cmd.ClearRenderTarget(true, true, Color.clear);
184     cmd.SetRenderTarget(_indirectDoseMap);
185     cmd.ClearRenderTarget(true, true, Color.clear);
186     Graphics.ExecuteCommandBuffer(cmd);
187     cmd.Release();
188 }
189
190 public void DispatchRays(Vector3 sunDirection, float beamLux, float
diffuseLux, float deltaHours, int samplesPerPixel, RenderTexture
positionMap, RenderTexture normalMap)
191 {
192     if (!_isInitialized || positionMap == null || normalMap == null
) return;
193
194     Camera cam = Camera.main;
195 #if UNITY_EDITOR
196     if (cam == null && UnityEditor.SceneView.lastActiveSceneView !=
null)
197         cam = UnityEditor.SceneView.lastActiveSceneView.camera;
198 #endif
199     if (cam == null) cam = GetComponent<Camera>();
200     if (cam == null) return;
201
202     _frameIndex++;
203     sunDirection.Normalize();
204
205     var sensors = VirtualLuxSensor.AllSensors;
206     int sensorCount = sensors.Count;
207     if (sensorCount > 0) {
208         if (_sensorInputBuffer == null || _sensorInputBuffer.count
!= sensorCount) {
209             if (_sensorInputBuffer != null) _sensorInputBuffer.
Release();
210             if (_sensorOutputBuffer != null) _sensorOutputBuffer.
Release();
211             _sensorInputBuffer = new ComputeBuffer(sensorCount, 32)
;
212             _sensorOutputBuffer = new ComputeBuffer(sensorCount, 4)
;
213             _cachedSensorInputs = new SensorInput[sensorCount];
214             _cachedZeros = new float[sensorCount];
215         }
216         for (int i = 0; i < sensorCount; i++) {
217             _cachedSensorInputs[i] = new SensorInput {
218                 position = (Vector4)sensors[i].transform.position +
new Vector4(0,0,0,1),
219                 normal = (Vector4)sensors[i].transform.up
220             };
221         }
222         _sensorInputBuffer.SetData(_cachedSensorInputs);
223         _sensorOutputBuffer.SetData(_cachedZeros);

```

```

224     }
225
226     var cmd = new CommandBuffer { name = "ChronoLux Dispatch" };
227     void SetCommon(RayTracingShader s) {
228         cmd.SetRayTracingAccelerationStructure(s, _ID_SceneRTAS,
229         _rtas);
230         cmd.SetRayTracingBufferParam(s, _ID_SimulationMaterials,
231         _simulationMaterialBuffer);
232         cmd.SetRayTracingBufferParam(s, _ID_MeshMetadata,
233         _metadataBuffer);
234         cmd.SetRayTracingBufferParam(s, _ID_GlobalVertices,
235         _vertexBuffer);
236         cmd.SetRayTracingBufferParam(s, _ID_GlobalIndices,
237         _indexBuffer);
238         cmd.SetRayTracingVectorParam(s, _ID_SunDirection,
239         sunDirection);
240         cmd.SetRayTracingFloatParam(s, _ID_BeamLux, beamLux);
241         cmd.SetRayTracingFloatParam(s, _ID_DiffuseLux, diffuseLux);
242         cmd.SetRayTracingIntParam(s, _ID_MaterialCount,
243         _materialCount);
244         cmd.SetRayTracingIntParam(s, _ID_FrameIndex, (int)
245         _frameIndex);
246         // Defend against huge loops/TDR
247         cmd.SetRayTracingIntParam(s, _ID_SamplesPerPixel, Mathf.
248         Clamp(samplesPerPixel, 1, 64));
249     }
250
251     SetCommon(rayTracingShader);
252     cmd.SetRayTracingTextureParam(rayTracingShader, _ID_PositionMap
253     , positionMap);
254     cmd.SetRayTracingTextureParam(rayTracingShader, _ID_NormalMap,
255     normalMap);
256     cmd.SetRayTracingTextureParam(rayTracingShader, _ID_DoseMap,
257     _doseMap);
258     cmd.SetRayTracingTextureParam(rayTracingShader,
259     _ID_DirectDoseMap, _directDoseMap);
260     cmd.SetRayTracingTextureParam(rayTracingShader,
261     _ID_IndirectDoseMap, _indirectDoseMap);
262     cmd.SetRayTracingFloatParam(rayTracingShader, _ID_DeltaHours,
263     deltaHours);
264     cmd.DispatchRays(rayTracingShader, "IrradianceBakeRayGen", (
265     uint)_doseMap.width, (uint)_doseMap.height, 1, cam);
266
267     if (sensorShader != null && sensorCount > 0) {
268         SetCommon(sensorShader);
269         cmd.SetRayTracingBufferParam(sensorShader, _ID_SensorInputs
270         , _sensorInputBuffer);
271         cmd.SetRayTracingBufferParam(sensorShader,
272         _ID_SensorOutputs, _sensorOutputBuffer);
273         cmd.DispatchRays(sensorShader, "SensorBakeRayGen", (uint)
274         sensorCount, 1, 1, cam);
275     }
276
277     Graphics.ExecuteCommandBuffer(cmd);
278     cmd.Release();
279
280     if (sensorCount > 0) {
281         var snapshot = sensors.ToArray();

```

```

263         AsyncGPUReadback.Request(_sensorOutputBuffer, (req) => {
264             if (req.hasError) return;
265             var data = req.GetData<float>();
266             for (int i = 0; i < snapshot.Length; i++) {
267                 if (i < data.Length && snapshot[i] != null) {
268                     snapshot[i].UpdateReadings(data[i],
sunDirection, beamLux, diffuseLux, deltaHours);
269                 }
270             }
271         });
272     }
273 }
274
275 private void Cleanup() {
276     if (_rtas != null) { _rtas.Release(); _rtas = null; }
277     void Rel(ref ComputeBuffer b) { if (b != null) { b.Release(); b
= null; } }
278     Rel(ref _simulationMaterialBuffer); Rel(ref _vertexBuffer); Rel
(ref _indexBuffer); Rel(ref _metadataBuffer); Rel(ref
_sensorInputBuffer); Rel(ref _sensorOutputBuffer);
279     void RelRT(ref RenderTexture rt) { if (rt != null) { rt.Release
(); if (Application.isPlaying) Destroy(rt); else DestroyImmediate(rt
); rt = null; } }
280     RelRT(ref _doseMap); RelRT(ref _directDoseMap); RelRT(ref
_indirectDoseMap);
281 }
282 private void OnDestroy() => Cleanup();
283 }

```

Listing 9.12: IrradianceBaker.cs: Orchestrates the hardware-accelerated DXR raytracing passes.

```

1 using UnityEngine;
2 using System.Collections.Generic;
3
4 [AddComponentMenu("ChronoLux/Virtual Lux Sensor")]
5 public class VirtualLuxSensor : MonoBehaviour
6 {
7     // Static registration to avoid FindObjectsByType allocations
8     public static readonly List<VirtualLuxSensor> AllSensors = new List
<VirtualLuxSensor>();
9
10     [Header("Validation Data")]
11     [ReadOnly, SerializeField] private float simulatedLux;
12     [ReadOnly, SerializeField] private float theoreticalLux;
13     [ReadOnly, SerializeField, Range(-100, 100)] private float
errorPercent;
14     [ReadOnly, SerializeField] private float accumulatedDose;
15
16     [Header("Visuals")]
17     public bool showGizmo = true;
18     public Color gizmoColor = Color.cyan;
19
20     private void OnEnable() => AllSensors.Add(this);
21     private void OnDisable() => AllSensors.Remove(this);
22
23     public void UpdateReadings(float lux, Vector3 sunDir, float beamLux
, float diffuseLux, float deltaHours = 0f)

```

```

24     {
25         simulatedLux = lux;
26         if (deltaHours > 0f) accumulatedDose += lux * deltaHours;
27         float nDotL = Mathf.Max(0, Vector3.Dot(transform.up, sunDir));
28         float direct = beamLux * nDotL;
29         theoreticalLux = direct + (diffuseLux * 0.5f);
30
31         if (theoreticalLux > 1e-3f)
32             errorPercent = ((simulatedLux - theoreticalLux) /
theoreticalLux) * 100f;
33         else
34             errorPercent = 0f;
35     }
36
37     public float currentLux => simulatedLux;
38     public float errorPercentage => errorPercent;
39     public float currentDose => accumulatedDose;
40
41     public void ResetDose() {
42         accumulatedDose = 0f;
43     }
44
45     private void OnDrawGizmos()
46     {
47         if (!showGizmo) return;
48         Gizmos.color = gizmoColor;
49         Gizmos.DrawWireSphere(transform.position, 0.05f);
50         Gizmos.DrawRay(transform.position, transform.up * 0.2f);
51
52     #if UNITY_EDITOR
53         UnityEditor.Handles.Label(transform.position + Vector3.up * 0.1
f, $"{simulatedLux:F0} Lux");
54     #endif
55     }
56 }
57
58 // Simple attribute to make fields read-only in inspector
59 public class ReadOnlyAttribute : PropertyAttribute { }

```

Listing 9.13: VirtualLuxSensor.cs: Provides ground-truth telemetry comparison via theoretical light accumulation.

```

1 using UnityEngine;
2 using UnityEngine.InputSystem;
3 using ChronoLux.Library;
4 using System;
5
6 namespace ChronoLux.Interaction
7 {
8     /// <summary>
9     /// Optimized Object Picker: Fixed frame drops by removing
redundant per-frame material updates.
10     /// </summary>
11     public class ObjectPicker : MonoBehaviour
12     {
13         [Header("Assets")]
14         public MaterialLibrary library;
15         public Shader selectionShader;

```

```

16
17     [Header("Colors")]
18     public Color hoverColor = new Color(0.1f, 0.4f, 1.0f, 1.0f);
19     public Color selectColor = new Color(0.6f, 0.1f, 1.0f, 1.0f);
20
21     public event Action<GameObject> OnObjectSelected;
22     public event Action OnSelectionCleared;
23
24     [Header("Status (Read Only)")]
25     public GameObject hoveredObject;
26     public GameObject selectedObject;
27
28     private Camera _cam;
29     private MaterialPropertyBlock _propBlock;
30     private Material _highlightMat;
31     private static readonly int _ID_GlowColor = Shader.PropertyToID
32     ("_GlowColor");
33
34     private void Start()
35     {
36         _cam = GetComponent<Camera>();
37         _propBlock = new MaterialPropertyBlock();
38         if (selectionShader != null) _highlightMat = new Material(
39         selectionShader);
40
41     private void OnDestroy()
42     {
43         if (_highlightMat != null)
44         {
45             if (Application.isPlaying) Destroy(_highlightMat);
46             else DestroyImmediate(_highlightMat);
47         }
48
49     private void Update()
50     {
51         // Only allow selection while in Navigation Mode (Cursor
52         Locked)
53         if (Cursor.lockState != CursorLockMode.Locked)
54         {
55             if (hoveredObject != null)
56             {
57                 if (hoveredObject != selectedObject) ClearHighlight
58                 (hoveredObject);
59                 hoveredObject = null;
60             }
61             return;
62         }
63
64         HandleHover();
65
66         if (Mouse.current != null && Mouse.current.leftButton.
67         wasPressedThisFrame)
68         {
69             HandleSelection();
70         }
71     }

```

```

69
70     private void HandleHover()
71     {
72         Ray ray = _cam.ViewportPointToRay(new Vector3(0.5f, 0.5f, 0
f));
73
74         if (Physics.Raycast(ray, out RaycastHit hit, 100f))
75         {
76             GameObject hitObj = hit.collider.gameObject;
77
78             if (hitObj != hoveredObject)
79             {
80                 // Only clear if it's NOT the selected object
81                 if (hoveredObject != null && hoveredObject !=
selectedObject)
82                     ClearHighlight(hoveredObject);
83
84                 hoveredObject = hitObj;
85
86                 // Only apply if it's NOT already the selected
object (which has its own color)
87                 if (hoveredObject != selectedObject)
88                     ApplyHighlight(hoveredObject, hoverColor);
89             }
90         }
91         else
92         {
93             if (hoveredObject != null)
94             {
95                 if (hoveredObject != selectedObject) ClearHighlight
(hoveredObject);
96                 hoveredObject = null;
97             }
98         }
99     }
100
101     private void HandleSelection()
102     {
103         if (hoveredObject != null)
104         {
105             if (selectedObject != null) ClearHighlight(
selectedObject);
106
107             selectedObject = hoveredObject;
108             ApplyHighlight(selectedObject, selectColor);
109             OnObjectSelected?.Invoke(selectedObject);
110         }
111         else
112         {
113             if (selectedObject != null) ClearHighlight(
selectedObject);
114             selectedObject = null;
115             OnSelectionCleared?.Invoke();
116         }
117     }
118
119     private void ApplyHighlight(GameObject obj, Color color)
120     {

```

```

121         if (obj == null || _highlightMat == null) return;
122
123         // Skip highlights for artifacts to keep heatmap visible
124         if (obj.GetComponent<UVMaBaker>() != null || obj.
GetComponentInParent<UVMaBaker>() != null)
125             return;
126
127         Renderer r = obj.GetComponentInChildren<Renderer>();
128         if (r == null) return;
129
130         // Set the color via PropertyBlock
131         r.GetPropertyBlock(_propBlock);
132         _propBlock.SetColor(_ID_GlowColor, color);
133         r.SetPropertyBlock(_propBlock);
134
135         // Ensure the material pass exists
136         Material[] mats = r.sharedMaterials;
137         bool exists = false;
138         foreach (var m in mats) if (m != null && m.shader ==
selectionShader) exists = true;
139
140         if (!exists)
141         {
142             Material[] newMats = new Material[mats.Length + 1];
143             for (int i = 0; i < mats.Length; i++) newMats[i] = mats
[i];
144             newMats[newMats.Length - 1] = _highlightMat;
145             r.sharedMaterials = newMats;
146         }
147     }
148
149     private void ClearHighlight(GameObject obj)
150     {
151         if (obj == null) return;
152         Renderer r = obj.GetComponentInChildren<Renderer>();
153         if (r == null) return;
154
155         Material[] mats = r.sharedMaterials;
156         int count = 0;
157         foreach (var m in mats) if (m == null || m.shader !=
selectionShader) count++;
158
159         if (count < mats.Length)
160         {
161             Material[] newMats = new Material[count];
162             int j = 0;
163             for (int i = 0; i < mats.Length; i++)
164             {
165                 if (mats[i] != null && mats[i].shader !=
selectionShader)
166                     newMats[j++] = mats[i];
167             }
168             r.sharedMaterials = newMats;
169         }
170     }
171
172     public void AssignMaterialToSelected(MaterialPreset preset)
173     {

```

```

174         if (selectedObject == null || preset == null) return;
175         Renderer r = selectedObject.GetComponentInChildren<Renderer>
>();
176         if (r == null) return;
177
178         // Preserve the highlight while changing the base material
179         r.sharedMaterial = preset.visualMaterial;
180
181         var sim = selectedObject.GetComponent<SimulationMaterial>()
?? selectedObject.AddComponent<SimulationMaterial>();
182         sim.reflectance = preset.reflectance;
183         sim.transmittance = preset.transmittance;
184
185         // Re-apply the selection color to the PropertyBlock (which
might have been reset)
186         r.GetPropertyBlock(_propBlock);
187         _propBlock.SetColor(_ID_GlowColor, selectColor);
188         r.SetPropertyBlock(_propBlock);
189     }
190 }
191 }

```

Listing 9.14: ObjectPicker.cs: Crosshair-based physical material editor and artifact interaction logic.

```

1 using UnityEngine;
2 using UnityEngine.InputSystem;
3 using UnityEngine.EventSystems;
4
5 namespace ChronoLux.Interaction
6 {
7     /// <summary>
8     /// Snappy, precise camera controller updated for the Dashboard
workflow.
9     /// Click 3D world to lock cursor and fly. ESC to return to UI.
10    /// </summary>
11    public class FreeLookCamera : MonoBehaviour
12    {
13        public float sensitivity = 0.05f;
14        public float speed = 5f;
15
16        private Vector2 _rotation;
17
18        private void Start()
19        {
20            _rotation.y = transform.eulerAngles.y;
21            _rotation.x = transform.eulerAngles.x;
22        }
23
24        private void Update()
25        {
26            var mouse = Mouse.current;
27            if (mouse == null) return;
28
29            // 1. Handle Cursor Toggles
30            if (Keyboard.current.escapeKey.wasPressedThisFrame)
31            {
32                if (Cursor.lockState == CursorLockMode.Locked)

```

```

33         {
34             Cursor.lockState = CursorLockMode.None;
35             Cursor.visible = true;
36         }
37         else
38         {
39             Cursor.lockState = CursorLockMode.Locked;
40             Cursor.visible = false;
41         }
42     }
43
44     // Click to lock (if not clicking on a UI element)
45     bool isOverUI = false;
46     if (EventSystem.current != null)
47     {
48         isOverUI = EventSystem.current.IsPointerOverGameObject
49     );
50     }
51
52     if (mouse.leftButton.wasPressedThisFrame && !isOverUI)
53     {
54         Cursor.lockState = CursorLockMode.Locked;
55         Cursor.visible = false;
56     }
57
58     // 2. Navigation (Only when locked)
59     if (Cursor.lockState != CursorLockMode.Locked) return;
60
61     Vector2 delta = mouse.delta.ReadValue() * sensitivity;
62     _rotation.y += delta.x;
63     _rotation.x = Mathf.Clamp(_rotation.x - delta.y, -89f, 89f)
64 ;
65     transform.localRotation = Quaternion.Euler(_rotation.x,
66 _rotation.y, 0f);
67
68     Vector3 input = Vector3.zero;
69     var kb = Keyboard.current;
70     if (kb.wKey.isPressed) input += transform.forward;
71     if (kb.sKey.isPressed) input += -transform.forward;
72     if (kb.aKey.isPressed) input += -transform.right;
73     if (kb.dKey.isPressed) input += transform.right;
74     if (kb.eKey.isPressed) input += Vector3.up;
75     if (kb.qKey.isPressed) input += Vector3.down;
76
77     float currentSpeed = speed;
78     if (kb.shiftKey.isPressed) currentSpeed *= 10f;
79
80     transform.position += input * currentSpeed * Time.deltaTime
81 ;
82     }
83 }

```

Listing 9.15: FreeLookCamera.cs: Handles the free-look navigation within the simulated laboratory environment.

```

1 using UnityEngine;
2

```

```

3 [DisallowMultipleComponent]
4 [AddComponentMenu("ChronoLux/Simulation Material")]
5 public class SimulationMaterial : MonoBehaviour
6 {
7     [Range(0f, 1f)] public float reflectance = 0.8f;
8     [Range(0f, 1f)] public float transmittance = 0.0f;
9
10    public void GetClampedScalars(out float reflectanceOut, out float
    transmittanceOut)
11    {
12        reflectanceOut = Mathf.Clamp01(reflectance);
13        transmittanceOut = Mathf.Clamp01(transmittance);
14
15        float sum = reflectanceOut + transmittanceOut;
16        if (sum <= 1.0f) return;
17
18        float inv = 1.0f / sum;
19        reflectanceOut *= inv;
20        transmittanceOut *= inv;
21    }
22
23    private void OnValidate()
24    {
25        GetClampedScalars(out reflectance, out transmittance);
26    }
27 }

```

Listing 9.16: SimulationMaterial.cs: Maintains energy-conserving reflectance and transmittance limits.

9.5. Python Data Processing Pipeline

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 import matplotlib.dates as mdates
6 import matplotlib.colors as mcolors
7 import os
8
9 # Ensure clean output directory
10 output_dir = 'figures'
11 os.makedirs(output_dir, exist_ok=True)
12
13 # Set high-quality styling for IEEE/ACM papers
14 plt.style.use('seaborn-v0_8-paper')
15 sns.set_theme(style="whitegrid", font_scale=1.2)
16
17 # Custom color palette for the scenarios to ensure visual distinction
18 colors = {"Scenario A (Baseline)": "#d62728", # Red
19          "Scenario B (Materials)": "#ff7f0e", # Orange
20          "Scenario C (Position)": "#1f77b4", # Blue
21          "Scenario D (Combined)": "#2ca02c"} # Green
22
23 print("Libraries loaded successfully.")
24

```

```

25 print("Loading CSV data and preprocessing...")
26 files = {
27     "Scenario A (Baseline)": "scenario_a.csv",
28     "Scenario B (Materials)": "scenario_b.csv",
29     "Scenario C (Position)": "scenario_c.csv",
30     "Scenario D (Combined)": "scenario_d.csv"
31 }
32
33 dfs = {}          # Hourly data (for high-res profiling)
34 dfs_daily = {}    # Daily data (for cleaner macro-scale plots)
35
36 for label, filename in files.items():
37     df = pd.read_csv(filename)
38     df['Time'] = pd.to_datetime(df['Time'])
39     # Scale to kLux-Hours for cleaner Y-axis labels
40     df['MaxDose_kLux'] = df['MaxDose'] / 1000.0
41     df['AvgDose_kLux'] = df['AvgDose'] / 1000.0
42
43     dfs[label] = df
44
45     # --- Preprocessing: Resample to Daily Frequency ---
46     df_set = df.set_index('Time')
47     df_daily = df_set.resample('1D').agg({
48         'MaxDose_kLux': 'max',          # Cumulative metric: take max of
the day
49         'AvgDose_kLux': 'max',          # Cumulative metric: take max of
the day
50         'DoseVariance': 'mean',        # Average contrast stress over
the day
51         'AvgSensorErrorPct': 'mean',   # Average daily convergence error
52         'DeltaMaxDose': 'max',          # Peak hourly hit across that
specific day
53         'BeamLux': 'mean',
54         'DiffuseLux': 'mean'
55     }).reset_index()
56     dfs_daily[label] = df_daily
57
58 print("Data loaded and preprocessed (Hourly & Daily variants created).")
59
60 # =====
61 # Fig 1A: Cumulative Peak Dose (Max)
62 # =====
63 fig, ax1 = plt.subplots(figsize=(8, 6))
64 for label, df_d in dfs_daily.items():
65     ax1.plot(df_d['Time'], df_d['MaxDose_kLux'], label=label, color=
colors[label], linewidth=2.5)
66 ax1.set_title('Cumulative Peak Dose (Max)', fontweight='bold')
67 ax1.set_xlabel('Month')
68 ax1.set_ylabel('Cumulative Dose (kLux-Hours)')
69 ax1.xaxis.set_major_formatter(mdates.DateFormatter('%b'))
70 ax1.legend()
71 plt.tight_layout()
72 plt.savefig(f'{output_dir}/fig1a_cumulative_max_dose.png', dpi=300)
73 plt.show()
74 plt.close()
75
76 # =====

```

```

77 # Fig 1B: Cumulative Average Dose (Avg)
78 # =====
79 fig, ax2 = plt.subplots(figsize=(8, 6))
80 for label, df_d in dfs_daily.items():
81     ax2.plot(df_d['Time'], df_d['AvgDose_kLux'], label=label, color=
82             colors[label], linewidth=2.5, linestyle='--')
83 ax2.set_title('Cumulative Average Dose (Avg)', fontweight='bold')
84 ax2.set_xlabel('Month')
85 ax2.set_ylabel('Cumulative Dose (kLux-Hours)')
86 ax2.xaxis.set_major_formatter(mdates.DateFormatter('%b'))
87 ax2.legend(loc='upper left')
88 plt.tight_layout()
89 plt.savefig(f'{output_dir}/fig1b_cumulative_avg_dose.png', dpi=300)
90 plt.close()
91
92 # =====
93 # Fig 2: Final Dose Bar Chart
94 # =====
95 labels = list(dfs_daily.keys())
96 final_max = [df_d['MaxDose_kLux'].iloc[-1] for df_d in dfs_daily.values
97             ()]
98 final_avg = [df_d['AvgDose_kLux'].iloc[-1] for df_d in dfs_daily.values
99             ()]
100
101 x = np.arange(len(labels))
102 width = 0.35
103
104 fig, ax = plt.subplots(figsize=(10, 6))
105 rects1 = ax.bar(x - width/2, final_max, width, label='Peak Dose', color
106               =[colors[l] for l in labels], alpha=0.9)
107 rects2 = ax.bar(x + width/2, final_avg, width, label='Avg Dose', color
108               =[colors[l] for l in labels], alpha=0.5, hatch='//')
109
110 ax.set_ylabel('Total Cumulative Dose (kLux-Hours)')
111 ax.set_title('End-of-Year Environmental Impact Summary', fontweight='
112             bold')
113 ax.set_xticks(x)
114 ax.set_xticklabels([l.replace("Scenario ", "") for l in labels])
115 ax.legend()
116
117 def autolabel(rects):
118     for rect in rects:
119         height = rect.get_height()
120         ax.annotate(f'{height:.0f}', xy=(rect.get_x() + rect.get_width
121         () / 2, height),
122                   xytext=(0, 3), textcoords="offset points", ha='
123                   center', va='bottom', fontsize=10)
124
125 autolabel(rects1)
126 autolabel(rects2)
127
128 plt.tight_layout()
129 plt.savefig(f'{output_dir}/fig2_final_dose_bar.png', dpi=300)
130 plt.show()
131 plt.close()
132
133 # =====
134 # Fig 3A: Spatial Dose Variance

```

```

127 # =====
128 fig, ax1 = plt.subplots(figsize=(8, 6))
129 for label, df_d in dfs_daily.items():
130     y_data = df_d['DoseVariance']
131     ax1.plot(df_d['Time'], y_data, label=label, color=colors[label],
132             linewidth=2.0, alpha=0.9)
133 ax1.set_title('Spatial Dose Variance (Daily Aggregated Stress)',
134             fontweight='bold')
135 ax1.set_xlabel('Month')
136 ax1.set_ylabel('Variance')
137 ax1.xaxis.set_major_formatter(mdates.DateFormatter('%b'))
138 ax1.set_yscale('log')
139 ax1.legend()
140 plt.tight_layout()
141 plt.savefig(f'{output_dir}/fig3a_spatial_dose_variance.png', dpi=300)
142 plt.show()
143 plt.close()
144 # =====
145 # Fig 3B: Hardware Sensor Convergence Error
146 # =====
147 fig, ax2 = plt.subplots(figsize=(8, 6))
148 df_a_d = dfs_daily["Scenario A (Baseline)"]
149 ax2.plot(df_a_d['Time'], df_a_d['AvgSensorErrorPct'], color=colors["
150     Scenario A (Baseline)"], linewidth=2.0, alpha=0.9, label='Scenario A
151 ')
152 ax2.set_title('Virtual Sensor Monte Carlo Convergence', fontweight='
153     bold')
154 ax2.set_xlabel('Month')
155 ax2.set_ylabel('Sensor Error (%)')
156 ax2.xaxis.set_major_formatter(mdates.DateFormatter('%b'))
157 ax2.fill_between(df_a_d['Time'], 0, df_a_d['AvgSensorErrorPct'], color=
158     colors["Scenario A (Baseline)"], alpha=0.3)
159 ax2.legend()
160 plt.tight_layout()
161 plt.savefig(f'{output_dir}/fig3b_monte_carlo_convergence.png', dpi=300)
162 plt.show()
163 plt.close()
164 # =====
165 # Fig 4: Diurnal Exposure Heatmap
166 # =====
167 df_a = dfs["Scenario A (Baseline)"]
168 df_a_day = df_a.copy()
169 df_a_day['Hour'] = df_a_day['Time'].dt.hour + df_a_day['Time'].dt.
170     minute / 60.0
171
172 fig, ax = plt.subplots(figsize=(10, 6))
173 df_plot = df_a_day[df_a_day['DeltaMaxDose'] > 5]
174
175 scatter = ax.scatter(df_plot['Time'], df_plot['Hour'],
176                     c=df_plot['DeltaMaxDose'], cmap='inferno',
177                     norm=mcolors.LogNorm(vmin=10, vmax=df_plot['
178     DeltaMaxDose'].max()),
179                     s=15, alpha=0.8)
180 cbar = fig.colorbar(scatter, ax=ax)
181 cbar.set_label('Hourly Peak Irradiance (Lux-Hours, Log Scale)',
182               rotation=270, labelpad=15)

```

```

176
177 ax.set_title('Baseline Diurnal Exposure Heatmap (Solar Glare Profile)',
178             fontweight='bold')
179 ax.set_xlabel('Date')
180 ax.set_ylabel('Time of Day (Hour)')
181 ax.xaxis.set_major_formatter(mdates.DateFormatter('%b'))
182 ax.invert_yaxis()
183 ax.set_ylim(18, 6)
184
185 plt.tight_layout()
186 plt.savefig(f'{output_dir}/fig4_diurnal_heatmap.png', dpi=300)
187 plt.show()
188 plt.close()
189
190 # =====
191 # Fig 6: Direct vs Indirect Peak Dose Comparison (10-Day Period)
192 # =====
193
194 output_dir = 'figures'
195
196 # Load the new scenarios
197 df_window = pd.read_csv('scenario_window.csv')
198 df_window['Time'] = pd.to_datetime(df_window['Time'])
199
200 df_corner = pd.read_csv('scenario_corner.csv')
201 df_corner['Time'] = pd.to_datetime(df_corner['Time'])
202
203 fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 10), sharex=True)
204
205 # Plot for Scenario Window (using full dataset)
206 ax1.plot(df_window['Time'], df_window['DeltaMaxDirectDose'], label='
207         Peak Direct Dose', color='#dd8452', linewidth=2)
208 ax1.plot(df_window['Time'], df_window['DeltaMaxIndirectDose'], label='
209         Peak Indirect Dose', color='#4c72b0', linewidth=2)
210 ax1.set_title('Scenario: Object Next to Window', fontweight='bold')
211 ax1.set_ylabel('Peak Hourly Dose (Lux-Hours)')
212 ax1.fill_between(df_window['Time'], df_window['DeltaMaxDirectDose'],
213                 color='#dd8452', alpha=0.15)
214 ax1.fill_between(df_window['Time'], df_window['DeltaMaxIndirectDose'],
215                 color='#4c72b0', alpha=0.15)
216 ax1.legend(loc='upper right')
217
218 # fill the graph
219
220 # Plot for Scenario Corner (using full dataset)
221 ax2.plot(df_corner['Time'], df_corner['DeltaMaxDirectDose'], label='
222         Peak Direct Dose', color='#dd8452', linewidth=2)
223 ax2.plot(df_corner['Time'], df_corner['DeltaMaxIndirectDose'], label='
224         Peak Indirect Dose', color='#4c72b0', linewidth=2)
225 ax2.fill_between(df_corner['Time'], df_corner['DeltaMaxDirectDose'],
226                 color='#dd8452', alpha=0.15)
227 ax2.fill_between(df_corner['Time'], df_corner['DeltaMaxIndirectDose'],
228                 color='#4c72b0', alpha=0.15)
229 ax2.set_title('Scenario: Object in Corner', fontweight='bold')
230 ax2.set_ylabel('Peak Hourly Dose (Lux-Hours)')
231 ax2.set_xlabel('Date & Time')
232 ax2.xaxis.set_major_formatter(mdates.DateFormatter('%b %d\n%H:%M'))

```

```
225 ax2.legend(loc='upper right')
226
227 plt.tight_layout()
228 plt.savefig(f'{output_dir}/fig6_window_vs_corner_comparison.png', dpi
            =300)
229 plt.show()
230 plt.close()
231 print("'Fig 6: Direct vs Indirect Peak Dose Comparison' successfully
        updated and saved!")
```

Listing 9.17: data_analysis.ipynb: Extracted Python code used for parsing the metrology telemetry CSV and rendering IEEE formatted figures.

9.6. Published Paper

The following pages reproduce the scientific paper titled "*Real-Time Light Dosimetry for Heritage Digital Twins via Texture Space Ray Tracing*" in its entirety. This paper was submitted to the 2026 IEEE 22nd International Conference on Intelligent Computer Communication and Processing (ICCP 2026), which will be held in October 2026 in Cluj-Napoca, Romania.

Real-Time Light Dosimetry for Heritage Digital Twins via Texture Space Ray Tracing

Sebastian Duică
Computer Science Department
Technical University of Cluj-Napoca
Cluj-Napoca, Romania
Duica.Ro.Sebastian@student.utcluj.ro

Victor Ioan Băcu
Computer Science Department
Technical University of Cluj-Napoca
Cluj-Napoca, Romania
victor.bacu@cs.utcluj.ro

Abstract—Light damage dosimetry is essential for preserving cultural heritage and avoiding permanent photochemical degradation. Modern approaches to simulation of light dosimetry employ computationally heavy off-line software lacking in interactivity; thus, planning processes are hindered. In order to overcome such difficulties, ChronoLux, an innovative Digital Twin solution using Texture Space Ray Tracing technique for interactive light dosimetry is proposed. Instead of utilizing conventional screen-space algorithms that do not incorporate physical information on the temporal changes in energy into the UV-map of the artefact, ChronoLux offers real-time editing of architectural materials and artifact positioning. Moreover, apart from interactivity, the proposed approach provides instant results regarding dose variation over the surface of the object. Results of a year-long simulation prove that the framework produces metrologically precise information despite being fully interactive: with proper environmental adjustments, peak dose per annum dropped from 6.6 M to less than 150 kLux-Hours without disturbing Monte Carlo sensor convergence.

Index Terms—Digital Twins, Cultural Heritage, Ray Tracing, Light Dosimetry, Monte Carlo Integration, GPU Acceleration, DXR

I. INTRODUCTION

Light dosimetry is the calculation of radiative energy (in Lux-Hours) incident upon the surface during a certain amount of time period. The calculation is required for the conservation of cultural heritage where museum curators want to protect artifacts from light exposure and excessive heating.

Traditionally, such type of prediction was performed using some generalized models of lighting and/or off-line architectural rendering solutions. The problem with these approaches is that due to being off-line, rendering-based approaches, they prevent the possibility to evaluate the influence of window glazing changes, materials of the rooms and objects themselves, and so forth in real time. To fill this gap, the authors propose to use the concept of Digital Twin to estimate light doses interactively. In other words, unlike camera-to-image ray casting approach used by many existing tools, the new technique uses a texture space ray tracing when texture coordinates are used to cast rays and produce realistic irradiance values. When combined with the Perez all-weather sky model and NEE, both ambient and direct irradiation can be estimated interactively. Furthermore, the concept of Digital Twin allows

performing the process of irradiance values accumulation independently of a certain screen space or camera position.

The other innovation brought by the authors is the possibility of virtual point-probe sensor implementation and thus, evaluation of the error of the Monte-Carlo method. According to the experimental results obtained in the course of 2k-resolution textured 3D artifact modeling through the year, the proposed approach managed to reduce significantly the structural contrast stress reaching end-of-the year peak value reduction of 97.7%.

II. RELATED WORK

A. Traditional Simulation in Conservation

In the case of predictive analysis, the typical process involves extensive use of offline architectural simulation software packages, such as Radiance [1]. Utilizing backward ray-tracing algorithms and irradiance caching, such tools offer a reliable approach for accurately modeling the physical lighting distributions in architectural geometry. However, they are frequently employed in the calculation of lighting limits according to preventive conservation guidelines defined by Michalski [2] and the International Commission on Illumination (CIE) [3], where museum exhibits should be exposed for no more than certain numbers of Lux-Hours per year (e.g., no more than 50,000 Lux-Hours for highly sensitive materials like watercolor or textile paintings). While the offline simulation ensures an extremely precise, mathematically provable physical model and lighting distribution calculations, its very architecture imposes a significant bottleneck in the conservation workflow. In particular, lacking a hardware-accelerated renderer with real-time denoising and being limited to CPU-only calculations, the calculation of a single, one-year long exposure with hourly resolution might take up to several hours, thus prohibiting any sort of interactive work with the data. As opposed to those traditional simulators, ChronoLux provides the same level of physically accurate metrology, i.e., respects Lambertian scattering and energy conservation laws, but at real-time render speeds using a GPU-powered approach. In order to overcome the processing limitations of offline rendering, many virtual heritage projects use Precomputed Radiance Transfer (PRT) along with lightmaps to enable interactive frame rates [4].

B. Advances in Real-Time Ray Tracing

The adoption of hardware-accelerated Monte Carlo Path Tracing methods [5], [6] using the latest generation of Direct X Ray Tracing architecture [7] marks a milestone in the field of interactive graphics. The existence of dedicated hardware cores designed for BVH traversal and ray intersection has made real-time Global Illumination possible. This trend is accompanied by the rise of Texture Space Shading (TSS) [8], which allows us to remove shading from the pixel grid of the final image. Most recently, TSS technology has gained the ability to cache and stabilize Monte Carlo noise between temporal frames [9], making it possible to use cached shading results between frames and implement complicated materials without temporal aliasing. Nonetheless, despite these breakthroughs in the technology, all existing frameworks of real-time ray-tracing remain optimized mostly for visual performance and screen-space frame rate. These systems discard the lighting information outside the bounds of the camera's view using the frustum culling approach and don't support persistence in accumulations of raw Lux-Hours over time. ChronoLux is a framework that re-architects this DXR pipeline and uses ray launches from texture space coordinates instead of the virtual camera lenses to make the process of shading completely mathematically sound and independent of the camera itself.

III. METHODOLOGY & SYSTEM ARCHITECTURE

The architecture of ChronoLux is described in more detail below in terms of system components, numerical methods and GPU accelerated render paradigms employed. Figure 1 demonstrates how the application operates as an advanced Digital Twin running inside a modern C#/DirectX 12 graphics engine. The proposed architecture strictly separates the UI layer based on UXML/USS frameworks for controlling material properties of reflectance and transmittance and physics simulation layer. In contrast to traditional approaches, the innovation introduced by our solution is the transition from transient screen space rendering to permanent Texture Space Ray Tracing method, which enables rigorous accumulation of radiometric data across full 8760 hours of meteorological years, followed by extraction by a dedicated headless C# telemetry module for statistical analysis.

The ability to serialize the scene using an effective engine is crucial for the viability of the framework as a Persistent Digital Twin. Not only does the UI manage the interactive material library and solar data feed, but it automatically persists all changes made to objects, such as translations and scaling, as well as material assignment in terms of reflectivity and transmissiveness, on a submesh level. This process of seamless persistence allows any experimental setup, including those architectural modifications tested in our experiments, to persist indefinitely, without any loss of fidelity.

A. The Physics of Light Dosimetry

In order to simulate photo-degradation of historical artifacts properly, it is necessary to create a mathematically rigorous model of light exposure. Traditional conservator methods

consider the amount of light falling on an object (irradiance) as an integral of incoming radiance, projected along the incident angle using Lambert's Cosine Law:

$$E = \int_{\Omega} L_i(\omega_i) \cdot \cos(\theta) d\omega_i \quad (1)$$

ChronoLux framework applies a discrete approximation of the above-mentioned integration process via summing up all hourly exposures within a full meteorological year of 8760 hours: $D_{total} \approx \sum_{t=1}^{8760} E(t) \cdot \Delta t$. In order to provide physically sound energy conservation and inter-reflections of light rays within the environment, all architectural materials in the environment have the constraint on the sum of reflectance (R) and transmittance (T): $R + T \leq 1.0$. The above-mentioned mathematical rigor guarantees that radiometric data exported from the Digital Twin will be perfectly aligned with established scientific recommendations [3] and would serve as a solid basis for further action by the curator instead of meaningless pixel values.

B. Texture Space Ray Tracing Architecture

One of the fundamental limitations of the current HW accelerated ray tracing framework such as DirectX 12 is its dependence on screen space rendering. In traditional game engine architecture, a special type of camera is used, where rays are casted outward to represent visible pixels. As a result, anything lying outside the field of view of the camera is simply clipped away. This approach is completely unacceptable in the context of scientific light dosimetries where the continuous irradiance measurement of the whole geometrical surface is mandatory regardless of where exactly the user is looking. In order to overcome this limitation, the ChronoLux framework re-architects the DXR pipeline into Texture Space.

In brief, given an input 3D model, the framework runs special UV baker pre-pass where UV map is calculated. At this stage, the system also produces special RenderTexture in RGBFloat format storing Position and Normal coordinates in world space at every texel. Careful attention is paid to the fact that Z-clipping in this prepass must be set to $Z = 0.5$ as well as inversion of Y axis in order to conform to DirectX12 texture coordinates requirements. Finally, the core Ray Tracing Compute Shader is called in a way that makes its every thread to correspond to particular UV texel. As a consequence, every millimeter of the 3D model surface is independently calculating lighting conditions.

As it is shown in Algorithm 3, the proposed architecture makes the process of light simulation completely detached from screen space thus ensuring permanent bake down of energy data into the texture map. Thus, a very convenient irradiance accumulator is implemented which permanently maps thermal and light stress onto the geometry surface.

C. Environmental Modeling and NEE

Calculating cumulative impact of irradiance requires simulation of the whole spectrum of light ranging from ambient skylight illumination to direct sunlight radiation throughout

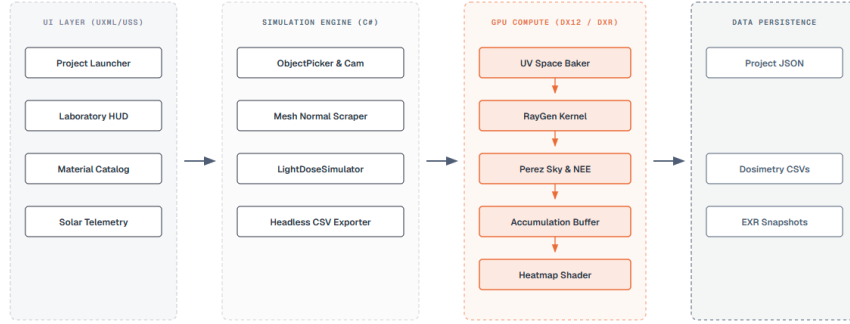


Fig. 1. High-level system architecture of ChronoLux, demonstrating the decoupled data flow from the UXML UI layer to the DX12 GPU Compute Pipeline, culminating in persistent CSV/EXR storage.

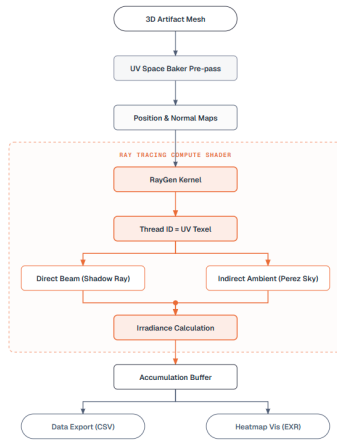


Fig. 2. The Texture Space Ray Tracing pipeline. The architecture bypasses the camera by pre-computing UV Position/Normal maps, allowing the compute shader to launch rays directly from the artifact's surface.

one meteorological year. Static single directional light source cannot properly represent the dynamic shadow behavior created by moving windows or changing position of the sun. To solve this issue, ChronoLux incorporates Perez All-Weather Sky Model [10] that computes precise sky luminance values according to real world climatic conditions and solar altitude.

During the ray generation kernel, the framework uses Next Event Estimation technique (NEE) by leveraging Monte Carlo path tracing algorithm to converge faster. In particular, in every UV texel shader performs separation of the rendering integral into two parts. Firstly, it casts the shadow ray pointing directly to localized solar disc and computes the irradiance, taking account of any architectural occlusions. Secondly, it performs a weighted sample cosine distribution across hemisphere in order to accumulate diffuse ambient skylight irradiance according to Perez distribution. Such an approach makes it

```

Require:  $PosMap, NormMap, SunDir, N_{samples}, \Delta t$ 
for each thread  $id \in DispatchThreadID$  do
   $\mathbf{p} \leftarrow PosMap[id].xyz$ 
   $\mathbf{n} \leftarrow NormMap[id].xyz$ 
  if  $\|\mathbf{n}\| < 0.1$  then
    return
  end if
   $E_{dir} \leftarrow TRACE_{SHADOW}(\mathbf{p}, SunDir) \cdot \max(0, \mathbf{n} \cdot SunDir)$ 
   $E_{ind} \leftarrow 0$ 
  for  $i = 1$  to  $N_{samples}$  do
     $\omega_i \leftarrow SAMPLE_{COSINEHEMISPHERE}(\mathbf{n})$ 
     $E_{ind} \leftarrow E_{ind} + TRACE_{PEREZSKY}(\mathbf{p}, \omega_i)$ 
  end for
   $E_{total} \leftarrow E_{dir} + (E_{ind}/N_{samples}) \cdot \pi$ 
   $DoseMap[id] \leftarrow DoseMap[id] + E_{total} \cdot \Delta t$ 
end for

```

Fig. 3. Pseudocode for Texture Space Ray Tracing Algorithm

possible to simulate complex high frequency architectural shadows, while not introducing additional Monte Carlo noise and not forcing us to use a large number of samples.

D. Statistical Metrology & Data Telemetry

Despite the fact that heat map visualization (with consideration of the Inferno color palette) provides valuable information for the qualitative assessment of the scene, it cannot be used to publish any results or provide scientific information needed for the purposes of conservation. Therefore, the quantitative availability of the information about light dose becomes critical in terms of statistical analysis. While the visualization allows us to identify problem areas on the scene, it does not provide enough information to confirm the presence of problems and take appropriate measures. For this reason, the idea of placing virtual Lux Sensors outside the scene was implemented. They allow for automatic positioning of control probes and extraction of the telemetry data right from the

accumulation buffer without tonemapping. At this stage, we can start analyzing such data as:

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (D_i - \bar{D})^2 \quad (2)$$

Furthermore, these virtual Lux probes calculate real time Monte Carlo convergence errors compared to theoretical values. Considering the fact that any attempt of GPU to CPU readout introduces serious synchronization issues, the telemetry process proceeds asynchronously utilizing a background headless thread in C#. This thread safely sums up every hourly delta value as well as maximum exposure and variance metrics, exporting the results to a standard CSV array without interrupting the 60 FPS render cycle.

IV. CASE STUDIES & EMPIRICAL RESULTS

A. Experimental Setup & Simulation Parameters

For empirical validation of ChronoLux's ability to quantify light stress, a number of case study simulations were performed on a high-fidelity 3D scan of the Stanford Bunny, represented in 2K resolution UV textures. The virtual environment was located at Latitude 44.427, Longitude 26.103 (Bucharest, Romania), and the simulation calculated a series of discrete hourly solar intervals for the entire meteorological year 2026 (8760 hours). The ray-tracing engine was tuned to perform 64 Monte Carlo rays per UV texel.

An important detail about our approach is that the model for calculating sunlight used is the Perez All-Weather Sky Model configured with a static value of $T = 2.0$. This corresponds to a mathematical representation of clear sky with no clouds. Although real-life weather fluctuates, the perpetually clear sky assumption is used to make a deliberate meteorological choice to apply stress testing in the Digital Twin in the absolute worst case scenario, thereby guaranteeing that the conservation measures applied in this scenario can handle the strongest possible annual photodegradation forces.

B. Scenario A: The Baseline Exposure

Scenario A uses the baseline setup of the asset being situated in a highly exposed place next to a window. Architectural material properties were set to replicate untreated museum conditions: floor albedo $R = 0.50$, while windows were made from clear glass material with transmittance $T = 0.84$ and albedo $R = 0.08$.

This configuration presents a worst case baseline of unlimited light exposure to the asset.

As shown in Figure 4, the result of such an extreme configuration is the catastrophic peak light dose of 6.6M Lux-Hours far surpassing the maximum light stress capacity of CIE's recommended 50,000 Lux-Hour threshold for sensitive materials [3].

The plot shown in Figure 5 visualizes the diurnal variation in illumination levels. Note the spikes of intensity at solar noon time and changes at solstice periods in June/December, highlighting the necessity of computation of a full meteorological

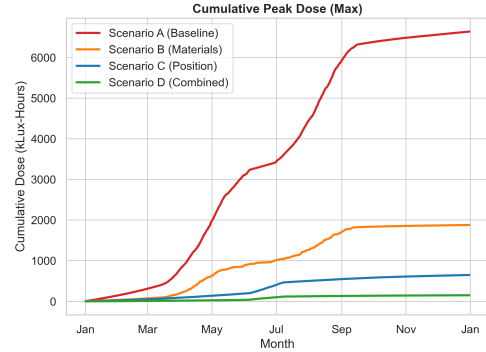


Fig. 4. CIE Light Dosimetry results for Scenario A reveal cumulative light stress exposure over 8760 hours resulting in a large peak light dose of 6.6M Lux-Hours far greater than the maximum recommended light stress of 50,000 Lux-Hours.

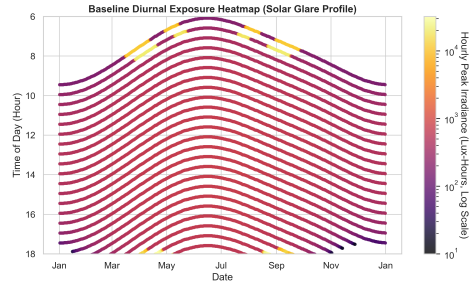


Fig. 5. A heat map of diurnal exposure illustrates hourly exposure levels in 365-day cycle, displaying high frequency peaks corresponding to solar noon hours and seasonal solar elevation changes.

year in order to calculate a precise diurnal profile rather than using static offline lightmaps.

C. Scenarios B, C, and D: Conservation Interventions

In order to test the ability of our preventative conservation measures, we took advantage of the interactive features of our Digital Twin and performed a comparative study of 3 scenarios against our baseline setup A. Scenario B (Material Intervention) involves changing the environmental setup, but keeping the original position of the asset. It includes replacing the window glass with conservation tinted glass that blocks UV radiation (transmittance $T = 0.45$, albedo $R = 0.12$) and reducing the amount of reflected ambient light by making the floor material matte ($R = 0.15$). Scenario C (Positional Optimization) explores the effects of geometric occlusion, the asset was placed in a shaded corner of the room. The architectural materials were restored to their initial highly reflective state. Lastly, Scenario D (Combined Intervention) explores the interaction of changing the location and adding protective glass and floor materials.

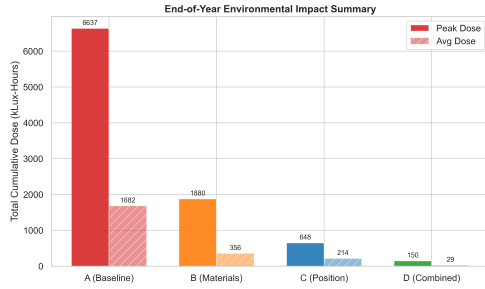


Fig. 6. Comparative study of peak light dose in four scenarios illustrates the effectiveness of Scenario D that combines optimization of the asset location with implementation of the protective materials.

The telemetry results shown in Figure 6 demonstrate a substantial reduction in light dosimetry in all three proposed scenarios compared to the baseline of 6.6M Lux-Hours. Scenario D reduces the peak light dose to 150k Lux-Hours.

These results illustrate the usefulness of our Digital Twin for simulating and validating synergistic conservation architectural retrofits without incurring real costs and risk associated with actual physical renovation projects.

D. Phenomenological Analysis: Window vs. Corner Geometry

An analysis over a span of 10 days during the winter solstice was carried out to demonstrate the capabilities of the Digital Twin regarding modeling non-traditional geometries in daylighting (Figure 7). This study involved comparing peak hourly doses for artifacts placed next to the window and those of artifacts placed deep in a room corner.

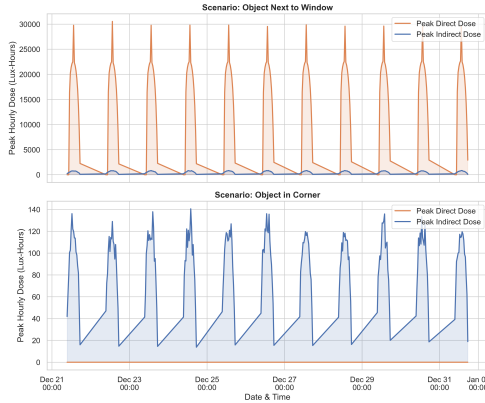


Fig. 7. The 10 day winter solstice comparison of peak hourly doses for artifacts placed next to windows with artifacts placed deep in room corners is presented. As seen from the empirical results below, the case with artifacts placed next to the window is dominated by a large direct solar component, while no direct sunlight falls on the second type of artifacts and the light dose is exclusively determined by the indirect and diffuse ambient contribution.

The data demonstrates the dynamic nature of daylighting geometry with respect to the winter solstice. Artifacts placed next to the window experience large direct sunlight concentrations reaching 30,000 Lux-Hours and largely exceeding any indirect ambient exposures. In the case of artifacts placed inside the room corner, the direct solar contribution is completely blocked by the architecture, resulting in a zero dose from this source. Hence, the indirect dose makes up 100% of the total exposure and is caused by scattered light from the sky. This conclusively proves the simulator's capability to correctly compute complex cosine-weighted hemispherical irradiance and physical shadowing without resorting to approximations.

V. PERFORMANCE & VALIDATION

A. Scientific Validation

Though the process of heat maps generation requires minor computational effort, metrological analysis requires us to make sure about the accuracy of calculations from a physical standpoint. Hence, the necessity to automatically place the Lux sensors outside the scene and analyze their spatial variance (σ^2) as well as convergence errors arose. The entire procedure took place during 8,760 rendering cycles of the scene.

The sensors in question are capable of sampling raw, untone-mapped floating point telemetry data taken from GPU accumulation buffers asynchronously. They are automatically positioned outside the scene bounds; hence, the irradiance measured at those locations can be used for the comparison with theoretical results based on the cited formulas. Convergence error of the Monte Carlo simulation is evaluated via $SEM = \sigma/\sqrt{N}$ formula, where SEM stands for standard error of the mean. It should be noted that σ is sample standard deviation while N equals the number of rays sampled per pixel. As weather simulation integrates with the program, the error gradually decreases to zero.

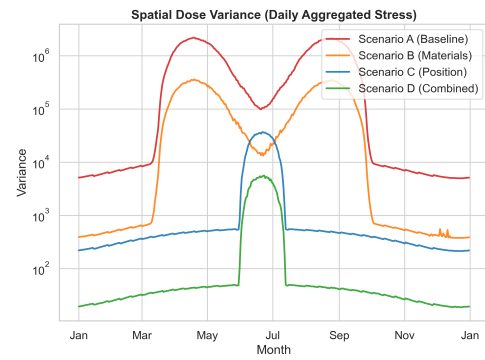


Fig. 8. Continuous monitoring of light spatial variance tracking aggregated structural stress on the artifact.

The graphs provided in Figure 8 and Figure 9 prove that Monte Carlo convergence error steadily drops and stabilizes during the calculation process. Spatial variance converges

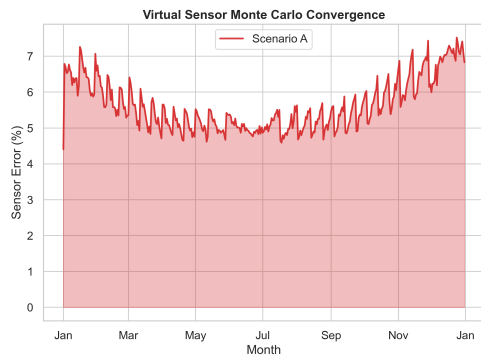


Fig. 9. Virtual Sensor Monte Carlo convergence error confirms stable convergence and validates our GPU-based calculation pipeline.

with the physical light stress accumulation rather than noise generated by the Monte Carlo algorithm, thereby proving the reliability of the GPU-based integration and allowing export of physically accurate CSV arrays.

In addition, the noted differences in terms of shape among the variance curves can be directly attributed to the placement of the artifact and its related incidence angle toward the sun. The peaks that have been noted in certain periods are directly correlated to the time periods when the direction of the vector from the sun is able to give it an uninterrupted path to the artifact, hence causing variance in the radiation levels. Despite these notable changes based on seasons and geometry, it is evident that the general trend of the curves still holds strong. Mathematically speaking, it shows the correctness of the Monte Carlo engine.

VI. CONCLUSION & FUTURE WORK

This paper presented ChronoLux, an interactive, hardware-accelerated Digital Twin framework explicitly engineered for the scientific metrology of photodegradation in cultural heritage artifacts. By fundamentally rethinking the rendering pipeline and shifting from transient screen-space rasterization to permanent Texture Space Ray Tracing, we successfully solved the temporal and spatial limitations that previously restricted graphics engines from performing rigorous radiometric dosimetry.

Through the integration of the Perez All-Weather Sky Model and a highly optimized DirectX 12 Monte Carlo path tracer featuring Next Event Estimation (NEE), ChronoLux demonstrated the capability to continuously integrate 8760 hourly solar intervals within a stable, interactive 60 FPS viewport. This represents a monumental leap in computational efficiency, compressing what traditionally required hours of offline, CPU-bound rendering into a real-time, exploratory tool.

The empirical case studies validated the framework's efficacy as a non-destructive conservation sandbox. By simulating the long-term light exposure of a 2K high-fidelity Stanford

Bunny scan, we demonstrated how the Digital Twin can isolate and quantify the exact radiometric impact of varying environmental interventions. Scenario testing revealed that synergistic interventions—such as repositioning the artifact to exploit geometric occlusion alongside the installation of UV-blocking tinted glazing and matte flooring—could successfully reduce a catastrophic 6.6 million Lux-Hour peak dose down to safe conservation thresholds of less than 150k Lux-Hours. Most critically, the scientific metrology was mathematically validated. By utilizing an automated array of out-of-bounds virtual Lux Sensors to sample the raw, untonemapped accumulation buffers, we tracked the Monte Carlo convergence error via the standard error of the mean ($SEM = \sigma/\sqrt{N}$). The continuous integration proved that the variance graphs structurally adhere to the physical realities of shifting solar vectors, confirming that the engine reliably calculates stable, physically-based light transport dynamics rather than stochastic rendering noise.

ACKNOWLEDGMENT

This research was funded by the European Union's Horizon Europe Framework Program, through the ChemiNova Project, grant agreement 101132442.

REFERENCES

- [1] G. Ward Larson and R. Shakespeare, *Rendering with Radiance: the art and science of lighting visualization*. Morgan Kaufmann, 1998. [Online]. Available: <https://www.radiance-online.org/learning/documentation/book>
- [2] S. Michalski, "Damage to museum objects by visible radiation (light) and ultraviolet radiation (uv)," *Conference on lighting in museums, galleries and historic houses*, pp. 3–16, 1987. [Online]. Available: <https://www.canada.ca/en/conservation-institute/services/agents-deterioration/light.html>
- [3] CIE, "Control of damage to museum objects by optical radiation," International Commission on Illumination, Tech. Rep. 157:2004, 2004. [Online]. Available: <https://cie.co.at/publications/control-damage-museum-objects-optical-radiation>
- [4] P.-P. Sloan, J. Kautz, and J. Snyder, "Precomputed radiance transfer for real-time rendering in dynamic, low-frequency lighting environments," *ACM Trans. Graph.*, vol. 21, no. 3, pp. 527–536, Jul. 2002. [Online]. Available: <https://doi.org/10.1145/566654.566612>
- [5] J. T. Kajiya, "The rendering equation," *SIGGRAPH Comput. Graph.*, vol. 20, no. 4, p. 143–150, Aug. 1986. [Online]. Available: <https://doi.org/10.1145/15886.15902>
- [6] E. Veach and L. J. Guibas, "Optimally combining sampling techniques for monte carlo rendering," in *Proceedings of the 22nd Annual Conference on Computer Graphics and Interactive Techniques*, ser. SIGGRAPH '95. New York, NY, USA: Association for Computing Machinery, 1995, pp. 419–428. [Online]. Available: <https://doi.org/10.1145/218380.218498>
- [7] J. Burgess, "Rtx on - the nvidia turing gpu," *IEEE Micro*, vol. PP, pp. 1–1, 02 2020. [Online]. Available: https://www.researchgate.net/publication/339036275_RTX_On_-_The_NVIDIA_Turing_GPU
- [8] J. Munkberg, J. Hasselgren, P. Clarberg, M. Andersson, and T. Akenine-Möller, "Texture space caching and reconstruction for ray tracing," *ACM Trans. Graph.*, vol. 35, no. 6, Dec. 2016. [Online]. Available: <https://doi.org/10.1145/2980179.2982407>
- [9] A. Zhu, Y. Mei, B. Jones, Z. Tatlock, and A. Schulz, "Computational illusion knitting," *ACM Trans. Graph.*, vol. 43, no. 4, Jul. 2024. [Online]. Available: <https://doi.org/10.1145/3658231>
- [10] R. Perez, R. Seals, and J. Michalsky, "All-weather model for sky luminance distribution—preliminary configuration and validation," *Solar Energy*, vol. 50, no. 3, pp. 235–245, 1993. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0038092X93900171>